UNIVERSITÁ DEGLI STUDI DI ROMA TOR VERGATA
Macroarea di Ingegneria

MASTER'S DEGREE PROGRAMME IN
ICT and Internet Engineering

FINAL THESIS IN
Information Theory and Data Science

WiFi Sensing Through Digital Receive Beamforming and CSI

Supervisor:
Prof. Mauro De Sanctis

Candidate:
Matricola: 0334623
Assel Ussenova

Academic Year 2024/2025

Contents

Chapter 1

Introduction

1.1 Motivation

Human activity recognition (HAR) aims to automatically identify human actions and behavioral patterns from sensor observations. HAR systems are increasingly used in healthcare, assisted living, smart homes, industrial monitoring, security, and human computer interaction. Typical applications include fall detection for elderly care, occupancy monitoring, gesture recognition, sleep monitoring, and tracking movement inside indoor environments. The growing demand for intelligent and context-aware systems has therefore motivated extensive research into reliable sensing technologies capable of continuously monitoring human activity.

Traditional HAR approaches mainly rely on wearable sensors or camera-based systems. Wearable devices such as accelerometers and gyroscopes provide accurate motion measurements and can capture detailed body dynamics. However, these systems require users to continuously wear dedicated devices, which limits their practicality in long term monitoring scenarios. Devices may be forgotten, removed, or discharged, especially in elderly care and passive monitoring applications. Camera-based systems provide detailed spatial information but introduce important limitations related to privacy, deployment cost, lighting conditions, and occlusions. These concerns become particularly significant in private indoor environments such as homes, hospitals, and offices.

As a result, there is a growing interest in sensing technologies capable of passive, low-cost, and privacy-preserving activity recognition. Radio frequency sensing has emerged as one of the most promising alternatives. Unlike cameras, radio signals do not require direct visibility and can operate in darkness and moderate occlusion conditions. Existing WiFi infrastructure continuously transmits signals that propagate through indoor spaces, reflect from walls, furniture, and human bodies, and arrive at the receiver through multiple propagation paths. Human movement modifies these multipath components and produces measurable variations in the wireless channel over time. By analyzing these variations, it becomes possible to infer

human activity without requiring the monitored person to carry devices or be visually observed [1].

Over the last decade, commodity WiFi sensing has attracted significant research attention due to its low deployment cost and passive sensing capabilities. Existing studies have demonstrated that WiFi signals can be used for activity recognition, gesture recognition, occupancy estimation, respiration monitoring, and gait analysis [1], [2]. Compared to specialized radar systems, WiFi sensing can operate using commercial off-the-shelf devices already available in indoor environments, making it practical for real world deployment.

Among different WiFi sensing approaches, Channel State Information (CSI) has become one of the most widely used signal representations for HAR. CSI provides fine-grained information about the wireless channel for each OFDM subcarrier and antenna pair. Unlike coarse channel metrics such as Received Signal Strength Indicator (RSSI), CSI captures detailed amplitude and phase information across both frequency and spatial dimensions. This additional resolution allows CSI to reveal subtle channel variations produced by human movement and environmental changes.

Modern WiFi systems employ Orthogonal Frequency Division Multiplexing (OFDM) and Multiple Input Multiple Output (MIMO) communication techniques. Consequently, CSI measurements consist of complex channel responses obtained across multiple subcarriers and multiple transmi-receive antenna combinations. Each CSI sample therefore provides a detailed spectro spatial representation of the indoor propagation environment. Small body movements affect the amplitude and phase of different multipath components, generating characteristic patterns that can be exploited for activity recognition [3].

Despite its advantages, CSI-based sensing introduces several important challenges. Indoor wireless channels are highly complex due to multipath propagation. The received signal is formed by the superposition of many reflections originating from both static and dynamic objects inside the environment. Static reflections from walls and furniture often dominate the received signal, while human-induced reflections appear as smaller time-varying perturbations superimposed on this static background. Consequently, the same human activity performed in different rooms can produce substantially different CSI measurements due to variations in room geometry, furniture arrangement, and propagation conditions.

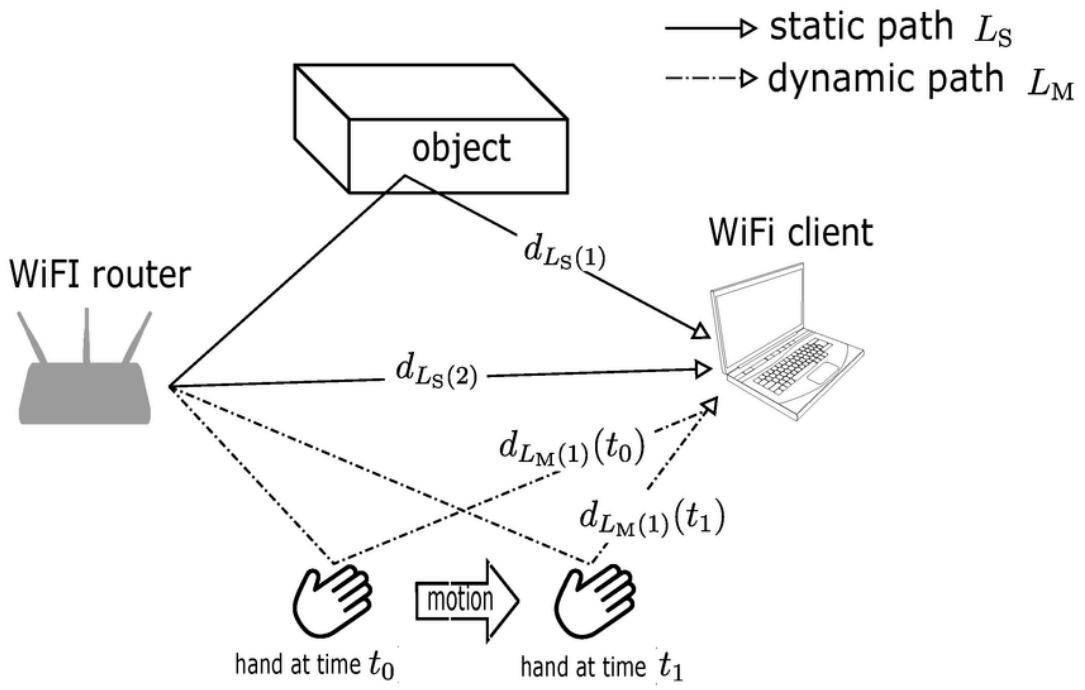


Figure 1.1. WiFi signal propagation in an indoor multipath environment; reflections from static objects and dynamic human bodies create time-varying channel responses. [5]

Another challenge arises from the phase component of CSI measurements obtained from commodity WiFi devices. In theory, CSI phase contains valuable information about signal propagation and motion dynamics. However, practical wireless hardware introduces several distortions related to synchronization errors and internal radio frequency processing. These effects generate unstable phase measurements that cannot be directly used for sensing without additional preprocessing [4], [5]. As a result, robust CSI-based HAR systems require dedicated signal processing techniques capable of mitigating hardware induced distortions while preserving motion related channel information.

At the same time, existing research has shown that spatial processing techniques such as digital receive beamforming can significantly improve the sensitivity of CSI-based sensing systems. Beamforming combines signals from multiple receive antennas using complex weights in order to spatially enhance reflections arriving from specific directions. This allows motion related channel variations to become more distinguishable from static environmental clutter. In addition, beamforming derived features are generally more robust to hardware related phase distortions because they rely on relative spatial information between antennas rather than on unstable absolute phase measurements [6], [7].

These observations motivate the approach developed in this thesis. The proposed work combines CSI phase preprocessing, digital receive beamforming, CSI distance analysis, and machine learning based classification for WiFi-based human activity recognition. The thesis investigates how commodity CSI measurements can be processed to suppress hardware distortions and emphasize motion sensitive channel variations suitable for robust indoor activity recognition.

1.2 Objectives and contributions

The main objective of this thesis is to investigate robust WiFi-based human activity recognition using beamformed Channel State Information measurements obtained from commodity wireless devices. The work focuses on improving the quality of CSI phase measurements, enhancing motion related channel variations through digital beamforming, and evaluating the effectiveness of the resulting features for indoor activity classification.

To achieve this objective, a complete processing framework is developed starting from raw CSI measurements and ending with machine learning based activity recognition. The proposed methodology combines CSI phase preprocessing, beamforming based signal enhancement, temporal motion analysis, and feature based classification.

The first objective of the thesis is to analyze the impact of hardware induced distortions on CSI phase measurements obtained from commodity WiFi devices. Particular attention is given to phase-locked loop bias, sampling frequency offset, symbol timing offset, and phase discontinuities appearing in 80 MHz CSI measurements. A preprocessing pipeline is developed in order to mitigate these distortions and recover spatially consistent CSI phase information suitable for beamforming applications.

The second objective is to investigate digital receive beamforming for indoor human activity sensing. The work studies how beamforming can spatially emphasize dynamic reflections generated by human movement while reducing the influence of static environmental clutter. Different receive antenna pairs and beamforming configurations are evaluated in order to analyze their impact on sensing performance.

The third objective is to construct robust motion sensitive features from beamformed CSI measurements. In particular, the thesis investigates Euclidean CSI distance analysis and temporal periodicity analysis of walking activity. The work studies how repeated motion patterns appear in CSI distance signals and how dominant motion periods can be extracted through peak detection and temporal analysis.

The final objective is to evaluate the proposed framework under multiple classification scenarios. Machine learning experiments are performed using Random Forest classification under both intra room and cross room conditions in order to evaluate the robustness and generalization capability of the extracted features.

The main contributions of this thesis can be summarized as follows:

1. A CSI phase preprocessing pipeline is developed for commodity 80 MHz WiFi measurements, including pilot subcarrier reconstruction, PLL bias mitigation, and segmentwise linear regression transformation for SFO and STO compensation.
2. The work analyzes the impact of subband stitching discontinuities in wideband CSI measurements and demonstrates the importance of segmentwise phase correction for beamforming applications.
3. A beamforming based CSI distance framework is implemented for motion sensitive feature construction and activity analysis.
4. A temporal periodicity analysis framework is proposed for walking activity characterization using CSI distance peak detection and motion period estimation.
5. The proposed methodology is experimentally evaluated using datasets collected in multiple indoor environments under different human activities and training configurations.

1.3 Structure of the Thesis

The remainder of this thesis is organized as follows.

Chapter 2 introduces the theoretical background necessary for understanding WiFi-based human activity recognition and CSI-based sensing. The chapter presents the fundamentals of OFDM and MIMO communication systems, defines Channel State Information, and discusses

the characteristics of CSI measurements obtained from commodity WiFi devices. The principles of digital receive beamforming and their relevance for RF sensing applications are also introduced.

Chapter 3 presents the proposed CSI phase preprocessing framework. The received CSI phase model is analyzed together with the main hardware impairments affecting commodity WiFi devices. The structure of the utilized dataset and the characteristics of the 80 MHz CSI measurements are then described. The preprocessing pipeline including pilot subcarrier reconstruction, PLL bias mitigation, and segmentwise linear regression transformation for SFO and STO compensation is subsequently introduced. Finally, preprocessing results are presented through CSI visualizations before and after correction.

Chapter 4 introduces the beamforming-based human activity recognition methodology. The mathematical formulation of digital receive beamforming is presented together with the implementation details used in this work. The chapter further explains the construction of Euclidean CSI distance features from beamformed CSI measurements and investigates the impact of antenna pair selection, beamforming direction, and window size on motion sensitivity. Experimental CSI distance results obtained for different activities and antenna configurations are also analyzed.

Chapter 5 focuses on the temporal analysis of walking activity using CSI distance signals. The chapter investigates the quasi periodic structure of walking induced CSI variations and introduces a peak-based periodicity analysis framework. Motion periods are estimated from CSI distance peaks and compared across different indoor environments using statistical and temporal analysis techniques.

Chapter 6 presents the feature extraction and activity classification framework. Statistical features are extracted from CSI distance and beamforming related measurements using multiple temporal window configurations. A Random Forest classifier is then trained and evaluated under different experimental scenarios including same room and cross room conditions. The obtained classification results are analyzed in terms of robustness, environmental sensitivity, and generalization capability.

Finally, Chapter 7 summarizes the main findings of the thesis, discusses the limitations of the proposed methodology, and outlines possible directions for future work.

Chapter 2

WiFi CSI and Beamforming Background

This chapter introduces the theoretical background required for understanding WiFi-based sensing using Channel State Information (CSI) and digital receive beamforming. Modern IEEE 802.11 wireless systems employ Orthogonal Frequency Division Multiplexing (OFDM) and Multiple-Input Multiple-Output (MIMO) communication techniques, which provide fine-grained measurements of the wireless propagation channel across frequency, time, and spatial dimensions. These measurements, known as CSI, form the basis of the sensing framework developed in this thesis.

The chapter first reviews the principles of OFDM and MIMO communication systems together with CSI estimation in IEEE 802.11 networks. The properties of CSI and its relevance for device-free human activity recognition are then discussed. Finally, the chapter introduces the fundamentals of digital receive beamforming, which forms the basis of the proposed spatial sensing methodology used in the following chapters.

2.1 IEEE 802.11 OFDM Systems

Modern IEEE 802.11 wireless local area networks employ Orthogonal Frequency Division Multiplexing (OFDM) together with Multiple-Input Multiple-Output (MIMO) transmission techniques in order to achieve high data rates and reliable communication in multipath indoor environments. OFDM converts a frequency-selective wideband channel into multiple orthogonal narrowband subchannels, while MIMO exploits spatial diversity and multipath propagation through multiple antennas.

In WiFi sensing applications, these communication mechanisms become highly relevant because they provide fine-grained measurements of the wireless propagation channel across both frequency and spatial dimensions. The resulting Channel State Information (CSI)

captures amplitude and phase variations over OFDM subcarriers and antenna pairs, enabling the observation of environmental dynamics caused by human motion.

2.1.1 OFDM Fundamentals

Orthogonal Frequency Division Multiplexing (OFDM) converts a wideband, frequency-selective channel into a set of narrowband flat-fading subcarriers by mapping a block of complex symbols $\{X_k\}$ onto N orthogonal tones and transmitting their inverse discrete Fourier transform; the discrete baseband OFDM symbol is therefore

$$s[n] = \frac{1}{N}\sum_{k=0}^{N-1} X_k e^{j2\pi\frac{k}{N}n}, \quad n = 0, \ldots, N-1,$$

where the inverse discrete Fourier transform (IDFT) is implemented in practice by an IFFT. A cyclic prefix of length L_{CP} samples is prepended to each OFDM symbol to absorb multipath delays up to L_{CP} and avoid inter-symbol interference; after transmission through a linear multipath channel and removal of the cyclic prefix, the receiver computes an N-point FFT and, under the usual narrowband assumption per subcarrier, the received complex sample on subcarrier k is modeled as

$$y_k = h_k x_k + n_k,$$

where x_k is the transmitted symbol on subcarrier k, h_k is the complex channel frequency response (CFR) at that subcarrier and time, and n_k is additive complex Gaussian noise. The CFR vector $H = [H_0, \ldots, H_{N-1}]^\top$ is the frequency-domain representation of the channel for that OFDM symbol; its discrete time dual, the channel impulse response (CIR) $h[\ell]$, is obtained by the IDFT (IFFT) via

$$h[\ell] = \frac{1}{N}\sum_{k=0}^{N-1} H_k e^{j2\pi\frac{k}{N}\ell}, \quad \ell = 0, \ldots, N-1,$$

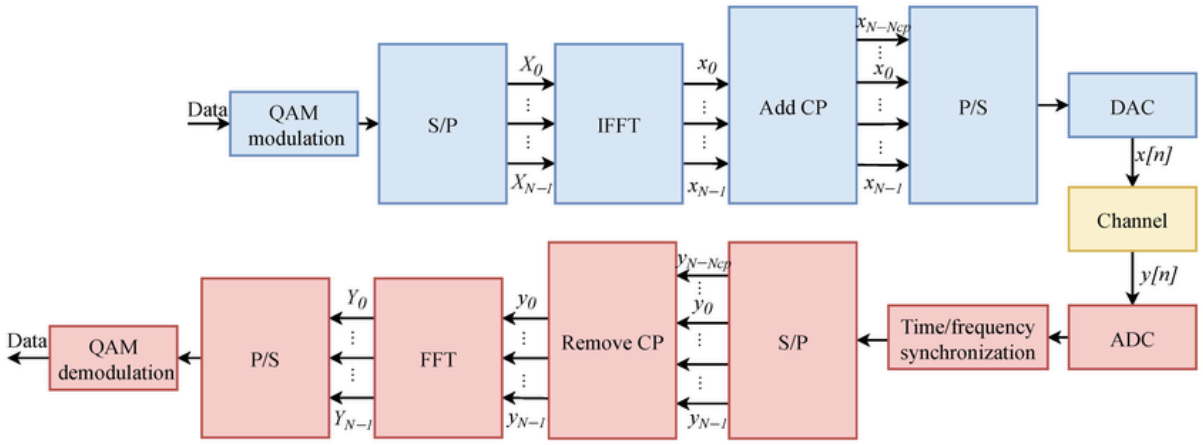


Figure 2.1. Block diagram of an OFDM communication system with a cyclic prefix [8].

Practical OFDM receivers estimate the CFR per subcarrier using known pilot/training symbols: if p_k is a known pilot on subcarrier k then the received pilot sample satisfies $y_k^p = h_k p_k + n_k$, and an unbiased single-tone least-squares estimate of the CFR is

$$\hat{h}_k = \frac{y_k^p}{p_k}, \quad (p_k \neq 0),$$

so that, for the n-th OFDM frame and receive antenna m, the estimated CSI vector is written

$$\hat{h}_m(n) = [\hat{h}_{m,1}(n), \hat{h}_{m,2}(n), \ldots, \hat{h}_{m,N_{sub}}(n)],$$

with each complex element decomposed as

$$\hat{h}_{m,k}(n) = A_{m,k,n} e^{j\theta_{m,k,n}},$$

the amplitude $A_{m,k,n}$ reflects attenuation and constructive/destructive multipath combining on that subcarrier while the phase $\theta_{m,k,n}$ encodes propagation delay, relative inter-antenna phase for array processing, and temporal Doppler, making CSI useful for sensing.

2.1.2 MIMO basics

Modern WiFi systems, including IEEE 802.11n/ac/ax, use Multiple-Input Multiple-Output (MIMO) communication to enhance throughput, reliability, and spatial resolution by transmitting data simultaneously over multiple antennas. MIMO technology exploits the spatial diversity created by multipath propagation, treating the wireless channel as a matrix-valued system rather than a single-channel scalar. This enables the simultaneous transmission of multiple data streams as shown in Figure 2.2, increasing data rates and improving link robustness in complex indoor environments.

In the context of MIMO-OFDM, the received signal at the m-th receive antenna on subcarrier k and time index n can be modeled as:

$$y_k^{(m)}(n) = \sum_{k=0}^{N-1} H_k^{(m,j)}(n) \cdot x_k^{(j)}(n) + n_k^{(m)}(n),$$

where:

- $x_k^{(j)}(n)$ is the symbol transmitted by the j-th transmit antenna on subcarrier k,
- $H_k^{(m,j)}(n)$ is the channel frequency response between the j-th transmit and m-th receive antenna on subcarrier k,
- $n_k^m(n)$ represents additive white Gaussian noise.

This formulation can be compactly represented using matrix notation as:

$$y_k(n) = H_k(n) \cdot x_k(n) + n_k(n),$$

where $H_k(n) \in \mathbb{C}^{N_r \times N_t}$ is the MIMO channel matrix at subcarrier k and time n, $x_k(n) \in \mathbb{C}^{N_t \times 1}$ is the transmitted symbol vector, and $y_k(n) \in \mathbb{C}^{N_r \times 1}$ is the received signal vector.

The spatial diversity provided by MIMO systems is particularly important for WiFi sensing applications because motion-induced channel variations appear differently across antenna pairs. This spatial information enables advanced processing techniques such as digital receive

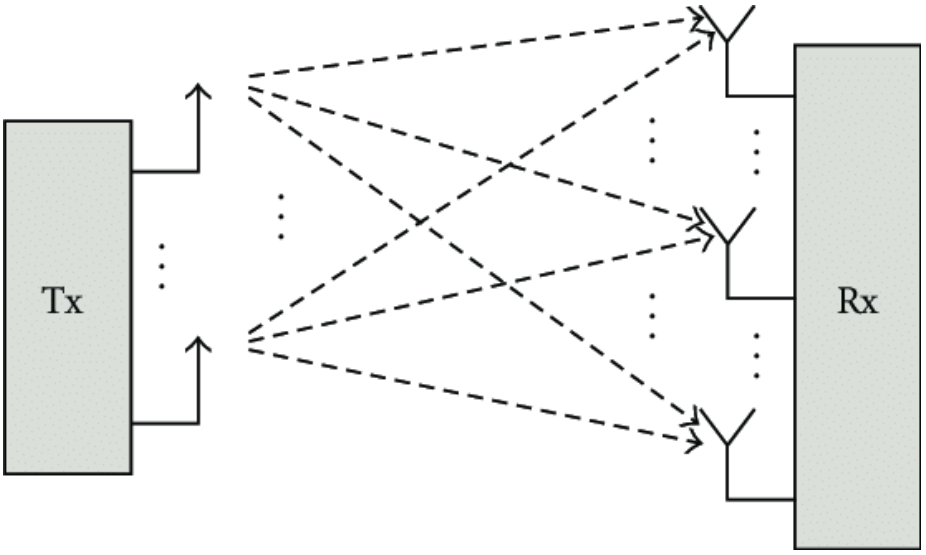


Figure 2.2. MIMO communication system with multiple transmit and receive antennas [9].

beamforming, where CSI measurements from multiple antennas are combined to enhance reflections arriving from specific directions.

2.2 Channel State Information

CSI describes the properties of the channel through which OFDM wireless signals propagate. These channel properties depend on the geometry and materials of the environment and therefore carry signatures of the scene: moving people, furniture, doors and other objects modify propagation paths and cause measurable perturbations in CSI. This is the reason why CSI has become a popular sensing signal in device-free human activity recognition (HAR): its high temporal, spectral and spatial resolution allows detection of subtle motion-induced changes while preserving privacy compared with video-based systems [1, 2].

The elementary CSI measurement is the complex frequency-domain channel gain on a single OFDM subcarrier and a single OFDM symbol (or packet). For the s-th OFDM symbol and the k-th subcarrier we denote the estimated CSI as

$$\widehat{h}_{s,k} = \left|\widehat{h}_{s,k}\right| e^{j\theta_{s,k}}$$

where $|\widehat{h}_{s,k}|$ is the amplitude and $\theta_{s,k}$ the phase. Stacking complex estimates across S successive symbols and K subcarriers yields the estimated CSI matrix:

$$\widehat{\mathbf{H}}_{S\times K} = \begin{pmatrix} \hat{h}_{1,1} & \hat{h}_{1,2} & \cdots & \hat{h}_{1,K} \\ \hat{h}_{2,1} & \hat{h}_{2,2} & \cdots & \hat{h}_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{h}_{S,1} & \hat{h}_{S,2} & \cdots & \hat{h}_{S,K} \end{pmatrix}.$$

The phase matrix $\Phi_{S\,x\,K}$ is formed by the angles $\theta_{s,k}$ and is often the primary object for phase-based sensing, while the amplitude matrix $\widehat{H}$ provides robust magnitude features and Doppler energy derived from short-time FFTs over packets.

Commodity CSI extractors report per-antenna CSI. In MIMO systems each receive chain produces its own $\widehat{H}$, so the dataset is conventionally organised as a tensor with axes (antennas, packets, subcarriers). Typical post-processing pipelines therefore operate along the subcarrier axis (to remove frequency-dependent distortions), along the antenna axis (to cancel per-chain constants or compute inter-antenna differences) and along the packet/time axis (to estimate packet biases and to compute Doppler spectra) [3, 4].

Although the complex CSI contains propagation information, the measured phase reported by commodity devices is corrupted by a mixture of deterministic and random distortions introduced by the RF front-end, clocks and packet processing.

Although CSI contains rich propagation information, commodity WiFi hardware introduces several impairments that distort the measured phase response. These distortions include synchronization offsets, hardware phase inconsistencies between RF chains, and subband stitching artifacts in wideband CSI measurements. Since digital receive beamforming relies heavily on accurate inter-antenna phase relationships, dedicated CSI preprocessing becomes necessary before reliable spatial processing can be applied.

The next chapter therefore introduces the proposed CSI phase preprocessing framework developed in this thesis.

2.3 Digital Receive Beamforming

2.3.1 Beamforming

Beamforming is a spatial signal processing technique used to direct the transmission or reception of electromagnetic waves toward a specific direction by combining signals from multiple antennas with appropriate phase and amplitude adjustments. Unlike omnidirectional reception, where signals arriving from all directions are treated equally, beamforming allows the antenna array to enhance signals arriving from selected directions while attenuating unwanted components and interference. This capability is particularly important in wireless communication and sensing systems operating in rich multipath indoor environments. [7]

In indoor WiFi propagation, transmitted signals interact with surrounding objects such as walls, furniture, and human bodies, generating multiple reflected and scattered propagation paths. As illustrated in Figure 3.1, the received signal is therefore composed of the superposition of several copies of the transmitted waveform arriving with different delays, attenuations, and phases. Human motion modifies these propagation paths dynamically, causing time-varying changes in the wireless channel response. Beamforming techniques exploit the spatial diversity provided by antenna arrays in order to selectively enhance specific propagation components and improve sensitivity to environmental changes. [5], [9]

For a uniform linear array (ULA) composed of M receive antennas, the received signals contain phase differences that depend on the direction of arrival (AoA) of the incoming wavefront. By properly compensating these phase differences through complex weighting, the received signals can be combined constructively for signals arriving from a desired direction while suppressing components arriving from other directions.

Beamforming techniques are commonly divided into two categories:

- Analog beamforming, where phase shifting and signal combining are performed in the analog RF domain before analog-to-digital conversion.
- Digital beamforming, where each antenna signal is independently digitized and combined using digital signal processing techniques.

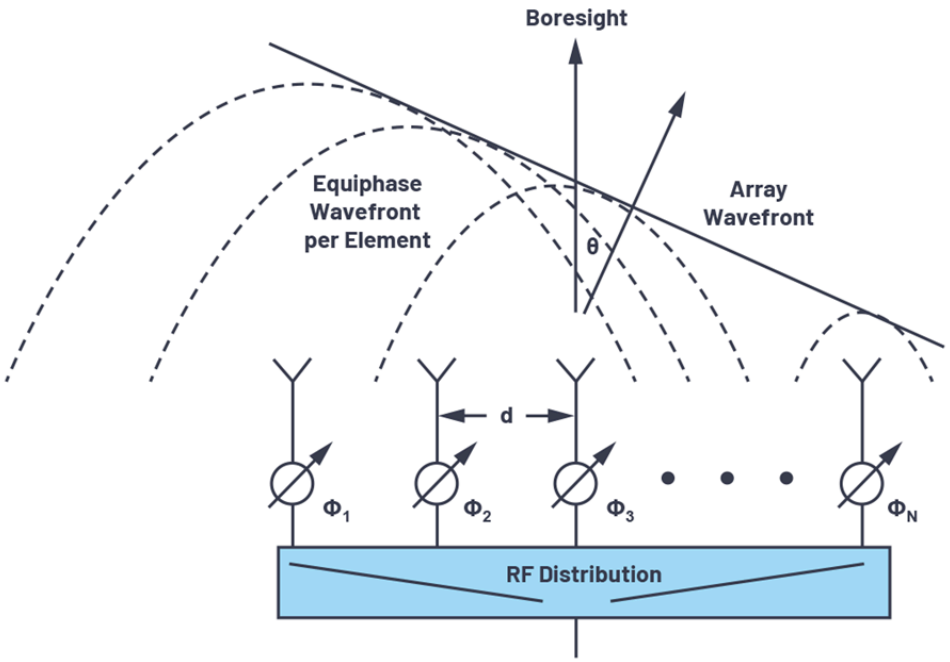


Figure 2.3. Beamforming concept in a multipath indoor environment with directional spatial filtering [12].

Analog beamforming generally has lower hardware complexity and power consumption but provides limited flexibility because only a single beam can usually be formed at a time. Digital beamforming, on the other hand, offers significantly greater flexibility since beam steering and spatial filtering are performed computationally in the digital domain. Multiple virtual beams can therefore be generated adaptively without physically modifying the antenna configuration [7].

2.3.2 Digital Beamforming

Digital beamforming (DBF) is based on independent processing of the signals received at each antenna element after analog-to-digital conversion. The digitized antenna signals are multiplied by complex weights and summed together in order to generate a directional response. Compared to analog beamforming, DBF preserves the complete spatial information captured by the antenna array and provides greater flexibility for adaptive signal processing. [10], [12]

Consider an antenna array composed of M receive antennas. Let the complex signal received at antenna m be represented as

$$x_m(t).$$

The beamformed output signal is obtained as the weighted sum of all antenna signals:

$$y(t) = \sum_{m=1}^{M} w_m^* x_m(t),$$

where $x_m(t)$ is the signal received at antenna m, w_m is the complex beamforming weight, $(\cdot)^*$ denotes complex conjugation, $y(t)$ is the beamformed output signal.

The complex beamforming weights can be represented as

$$w_m = a_m e^{j\phi_m},$$

where a_m controls the amplitude scaling, ϕ_m controls the phase shift applied to antenna m.

The phase shifts are selected such that signals arriving from a desired direction combine constructively at the output. Signals arriving from different directions experience phase mismatches and are therefore attenuated.

For a plane wave impinging on a uniform linear array (ULA) from direction θ, the propagation delay between adjacent antennas produces a phase difference given by

$$\Delta\phi = \frac{2\pi d}{\lambda} sin(\theta),$$

where d is the antenna spacing, λ is the carrier wavelength, θ is the angle of arrival.

The corresponding steering vector of the antenna array becomes

$$a(\theta) = \left[1 \quad e^{j\frac{2\pi d}{\lambda} sin(\theta)} \quad \ldots \quad e^{j(M-1)\frac{2\pi d}{\lambda} sin(\theta)} \right]^T.$$

The steering vector models the spatial phase progression observed across the antenna array for a signal arriving from direction θ. By selecting beamforming weights aligned with this

steering vector, the receiver becomes spatially sensitive to signals arriving from the desired direction. [7], [11]

Digital beamforming is extensively employed in modern wireless communication systems, including MIMO WiFi networks, radar systems, satellite communications, and massive MIMO architectures. In WiFi sensing applications, DBF enables spatial analysis of multipath propagation variations generated by human motion. [6]

2.3.3 Receive Digital Beamforming

Receive digital beamforming refers specifically to the application of beamforming at the receiver side, where signals collected by multiple receive antennas are combined to improve signal reception and spatial selectivity. Instead of processing each antenna independently, receive beamforming exploits the spatial correlation between antenna signals in order to enhance desired components and mitigate interference and noise. [10]

For a narrowband received signal $s(t)$, the received signal vector across the antenna array can be modeled as

$$\overline{x}(t) = \overline{a}(\theta)\, s(t) + \overline{n}(t),$$

where $\overline{a}(\theta)$ is the steering vector, $s(t)$ is the transmitted signal, $\overline{n}(t)$ is additive noise.

The beamformed output signal is expressed as

$$y(t) = \overline{w}^{H} \overline{x}(t),$$

where w is the beamforming weight vector, $(\cdot)^{H}$ denotes the Hermitian transpose.

Substituting the received signal model into the beamforming equation yields

$$y(t) = \overline{w}^{H} \overline{a}(\theta)\, s(t) + \overline{w}^{H} \overline{n}(t).$$

The term

$$\overline{w}^{H} \overline{a}(\theta)$$

represents the spatial response of the antenna array toward direction θ. By appropriately selecting the beamforming weights, signals arriving from the desired direction are coherently combined while signals arriving from other directions are attenuated.

In indoor WiFi environments, receive digital beamforming becomes particularly relevant because multipath components arrive from multiple spatial directions. Human motion dynamically modifies these propagation paths, producing time-varying spatial signatures in CSI measurements. Spatial filtering through beamforming can therefore improve sensitivity to human-induced channel variations.

2.3.4 Receive Digital Beamforming for MIMO-OFDM Systems

Modern IEEE 802.11 WiFi systems employ MIMO-OFDM communication architectures, where CSI measurements are estimated independently for each subcarrier and antenna pair. In such systems, receive digital beamforming can be applied directly in the frequency domain after OFDM demodulation. This approach is commonly referred to as post-FFT beamforming. [10]

After cyclic prefix removal and FFT processing, the received signal on subcarrier k can be expressed as

$$\overline{Y[k]} = \overline{H[k]}\ \overline{X[k]} + \overline{N[k]},$$

where $H[k]$ is the MIMO channel matrix, $\overline{X[k]}$ is the transmitted signal vector, $N[k]$ is additive noise.

The CSI matrix $H[k]$ contains the complex channel response for each transmit-receive antenna pair and each OFDM subcarrier. Since CSI preserves both amplitude and phase information, it also contains spatial information associated with signal propagation geometry.

Applying receive beamforming in the frequency domain consists of combining CSI measurements from multiple receive antennas through complex weighting:

$$Y_{BF}[k] = \overline{w}^{H}\,\overline{H[k]}.$$

Because beamforming relies on coherent phase combination across antennas, accurate inter-antenna phase consistency is essential. However, practical CSI measurements obtained from commodity WiFi devices are affected by hardware impairments such as synchronization errors, carrier frequency offset (CFO), sampling frequency offset (SFO), symbol timing offset (STO), and independent RF chain phase offsets. These impairments distort the measured CSI phase and can significantly degrade beamforming performance if not properly compensated. [4, 5]

For this reason, CSI phase preprocessing and calibration become fundamental steps before reliable digital receive beamforming can be applied in WiFi sensing systems. The next chapter therefore introduces the CSI preprocessing techniques adopted in this work in order to mitigate phase distortions and stabilize inter-antenna phase relationships prior to spatial processing.

Chapter 3

CSI Phase Preprocessing

The phase component of Channel State Information (CSI) contains valuable information about wireless propagation dynamics and human motion. In ideal OFDM systems, CSI phase reflects the propagation delay and multipath characteristics of the wireless channel. However, CSI measurements obtained from commodity WiFi devices significantly deviate from this ideal model due to multiple hardware and synchronization impairments introduced by the RF front-end and OFDM processing chain. These impairments generate unstable and distorted phase measurements that cannot be directly used for reliable sensing and beamforming applications. [4, 13]

In particular, digital receive beamforming relies on coherent phase relationships between receive antennas. Small phase inconsistencies between RF chains may therefore severely degrade spatial filtering performance and produce incorrect beamforming responses. Additional distortions such as carrier frequency offset (CFO), sampling frequency offset (SFO), symbol timing offset (STO), and subband stitching discontinuities further corrupt the phase response across OFDM subcarriers. As a consequence, CSI phase preprocessing becomes a fundamental step before applying beamforming-based sensing techniques [5, 13, 14].

This chapter introduces the proposed CSI phase preprocessing framework developed for commodity 80 MHz IEEE 802.11ac CSI measurements. First, the received CSI phase model and the main hardware-induced impairments affecting commodity WiFi devices are analyzed. Then, the structure of the adopted dataset and the characteristics of the CSI measurements are presented. Finally, the preprocessing pipeline including pilot subcarrier reconstruction, PLL bias mitigation, and linear phase correction for SFO/STO compensation is introduced.

3.1 Received Phase Modeling

The phase component of CSI carries rich information about multipath propagation, including path length variations and motion-induced changes. Unlike amplitude, phase is highly sensitive to small environmental dynamics, which makes it particularly valuable for fine-grained human activity recognition [4,13]. For this reason, understanding the structure of the received phase and distinguishing physical propagation effects from hardware-induced distortions is essential for building an environment-insensitive sensing pipeline.

In classical OFDM communication theory, the received phase at subcarrier k and symbol s is typically modeled as

$$\hat{\theta}_{s,k} = \theta_{s,k} + 2\pi \frac{m_k}{N} \Delta t + \gamma_s + Z_{s,k}$$

Here, $\hat{\theta}_{s,k}$ denotes the observed phase and $\theta_{s,k}$ represents the true channel-induced phase caused by multipath propagation. The term $2\pi\frac{m_k}{N}\Delta t$ models the phase error caused by imperfect timing synchronization between the transmitter and the receiver. m_k is the index of the k-th subcarrier and N is the FFT size. Δt is the time lag due to sampling frequency offset (SFO) and symbol timing offset (STO). γ_s represents the carrier frequency offset (CFO) caused by a mismatch between the transmitter and receiver carrier frequencies and results in an unknown phase rotation. The term $Z_{s,k}$ captures residual noise and unmodeled effects [14,16,17].

This classical phase error model accounts for synchronization impairments such as SFO/STO and CFO. However, it implicitly assumes a single receiver phase reference and does not explicitly include per-RF-chain constant phase offsets.

In commodity WiFi devices with multiple RF chains, each receive antenna has its own phase-locked loop (PLL). During device initialization, each PLL locks independently, introducing an antenna-specific constant phase offset [15]. As a result, the received phase at antenna n, subcarrier k, and packet s is more accurately described as

$$\widehat{\theta}_{s,k}^{n} = \theta_{s,k}^{n} + 2\pi \frac{m_k}{N} \Delta t + \gamma_s + \phi^n + Z_{s,k}^{n}$$

The additional term ϕ^n denotes the constant phase bias associated with RF chain n.

This distinction becomes critical when beamforming is applied, since it relies on accurate phase differences between antennas to enhance signals from specific directions. If each antenna carries an unknown constant bias ϕ^n, the relative phase relationships are distorted, and spatial combining may reflect hardware offsets rather than true propagation effects.

Although beamforming weights could theoretically absorb these constant offsets, the biases are unknown and device-dependent. Without explicit calibration, the spatial filter may adapt to hardware-specific phase rotations, reducing robustness across sessions and environments.

By explicitly modeling and compensating the antenna-specific phase bias ϕ^n, the proposed framework separates hardware impairments from propagation-induced phase variations and stabilizes the spatial processing stage, ensuring that beamforming captures environmental dynamics rather than device artifacts.

3.2 Dataset and CSI Structure Overview

The dataset used in this work was collected using the Nexmon CSI extraction framework running on an Asus RT-AC86U IEEE 802.11ac WiFi router configured as a monitoring device. The adopted setup operates in Very High Throughput (VHT) mode with a channel bandwidth of 80 MHz, allowing the extraction of CSI measurements from commodity WiFi transmissions. CSI traces were collected from packets exchanged between a transmitting access point and a client device while human activities were performed inside indoor office environments. The overall experimental setup and acquisition methodology follow the framework presented in the thesis by Rebecca Fallani.

In OFDM-based IEEE 802.11 systems, the available bandwidth is divided into multiple orthogonal subcarriers, each carrying a portion of the transmitted information. The number of active subcarriers depends on the selected channel bandwidth and physical layer standard. IEEE 802.11ac extends the bandwidth supported by previous IEEE 802.11a/n systems and

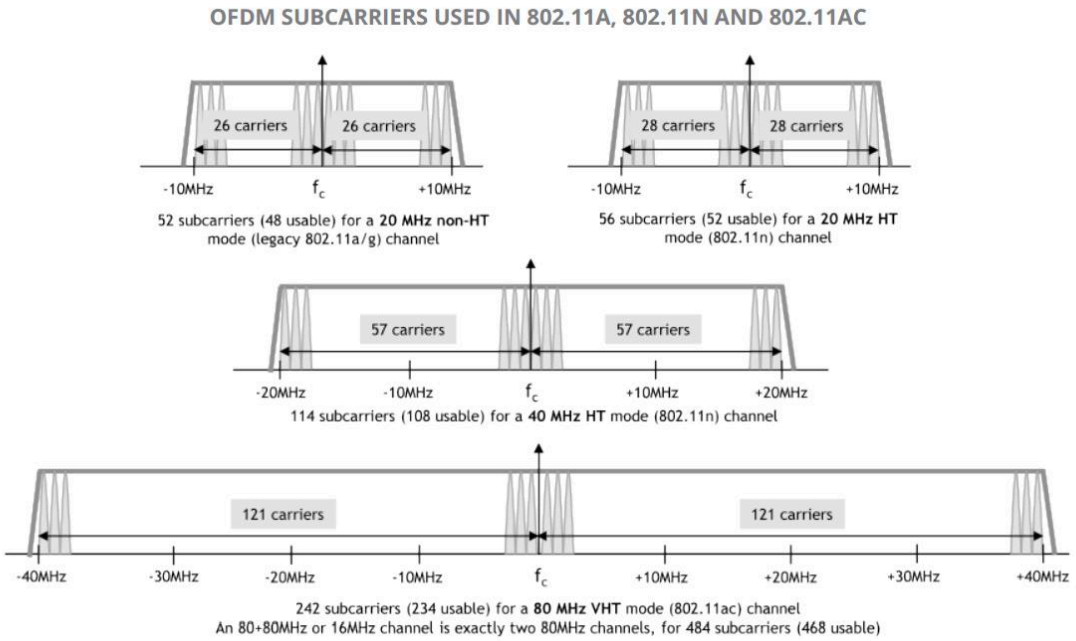


Figure 3.1. OFDM subcarrier allocation in IEEE 802.11a/n/ac systems for different channel bandwidths [18].

allows operation up to 80 MHz and 160 MHz channels. For an 80 MHz VHT channel, the OFDM structure is based on a 256-point FFT. However, not all FFT tones are used for data transmission since several subcarriers are reserved for pilot tones, guard bands, and the DC carrier around the center frequency. The general OFDM subcarrier allocation for IEEE 802.11a, IEEE 802.11n, and IEEE 802.11ac systems is illustrated in Figure 3.1 [18].

For the 80 MHz IEEE 802.11ac configuration adopted in this work, the OFDM system contains 256 FFT subcarriers, among which 234 correspond to active data subcarriers while the remaining tones are associated with pilots, guard bands, and null subcarriers around the DC component. The CSI extraction process performed by Nexmon initially provides CSI values corresponding to the active OFDM tones. During the offline preprocessing stage described in the referenced dataset processing workflow, pilot, guard, and DC subcarriers were removed in order to retain only the usable data-bearing subcarriers for subsequent processing and analysis.

Each CSI sample therefore consists of four complex-valued CSI vectors corresponding to the four receive antennas of the Asus RT-AC86U monitoring router. Every vector contains CSI measurements across the available OFDM subcarriers, providing both amplitude and phase

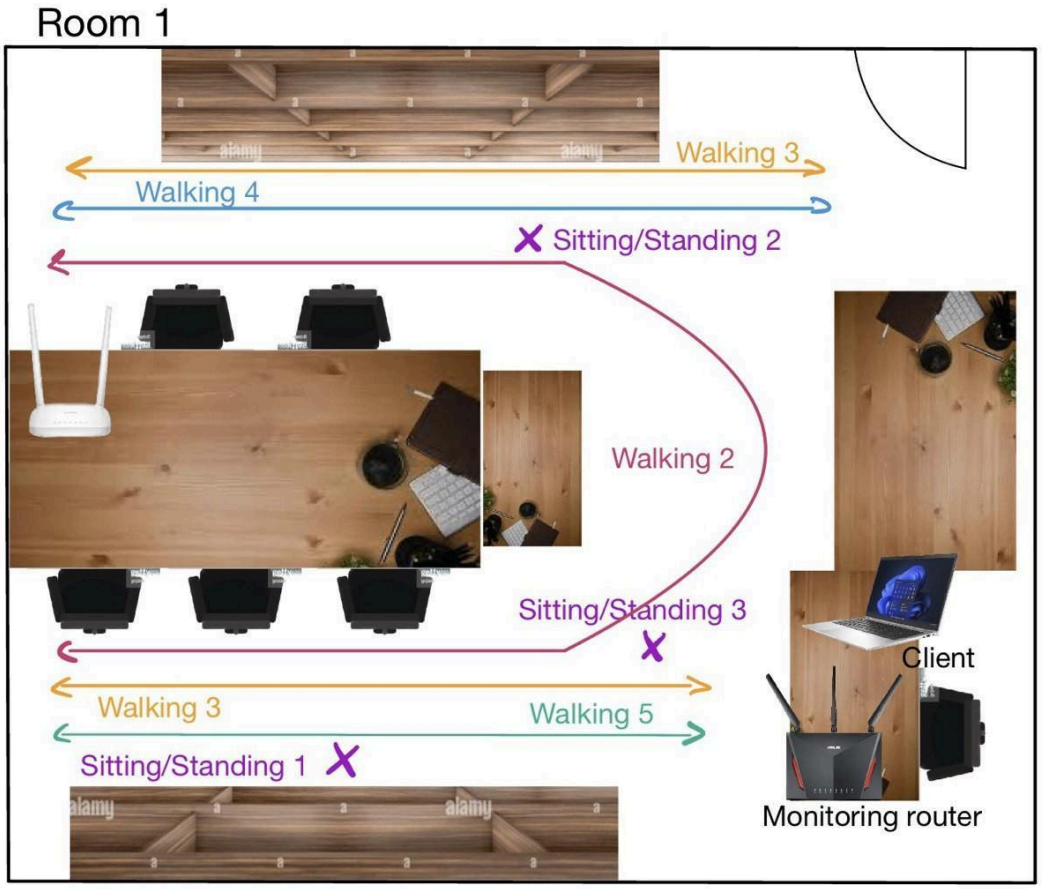


Figure 3.2. Experimental setup and monitored activity trajectories in Room 1 [19].

information for each receive antenna and subcarrier index. Since CSI is extracted independently for each antenna chain, the resulting measurements preserve spatial information required for digital receive beamforming and multi-antenna processing techniques.

The CSI acquisition process was performed in two different indoor office environments, referred to as Room 1 and Room 2, in order to evaluate the robustness of the proposed sensing framework under different propagation conditions and room geometries. The environments contain multiple reflecting objects including desks, chairs, computers, and walls, producing rich multipath propagation conditions typical of indoor WiFi scenarios. The positions of the transmitter, monitoring router, client device, and human activity trajectories are shown in Figures 3.2 and 3.3 adapted from [19].

The measurement setup reflects a realistic indoor deployment scenario where WiFi traffic is already present in the environment and CSI can be collected passively through a monitoring

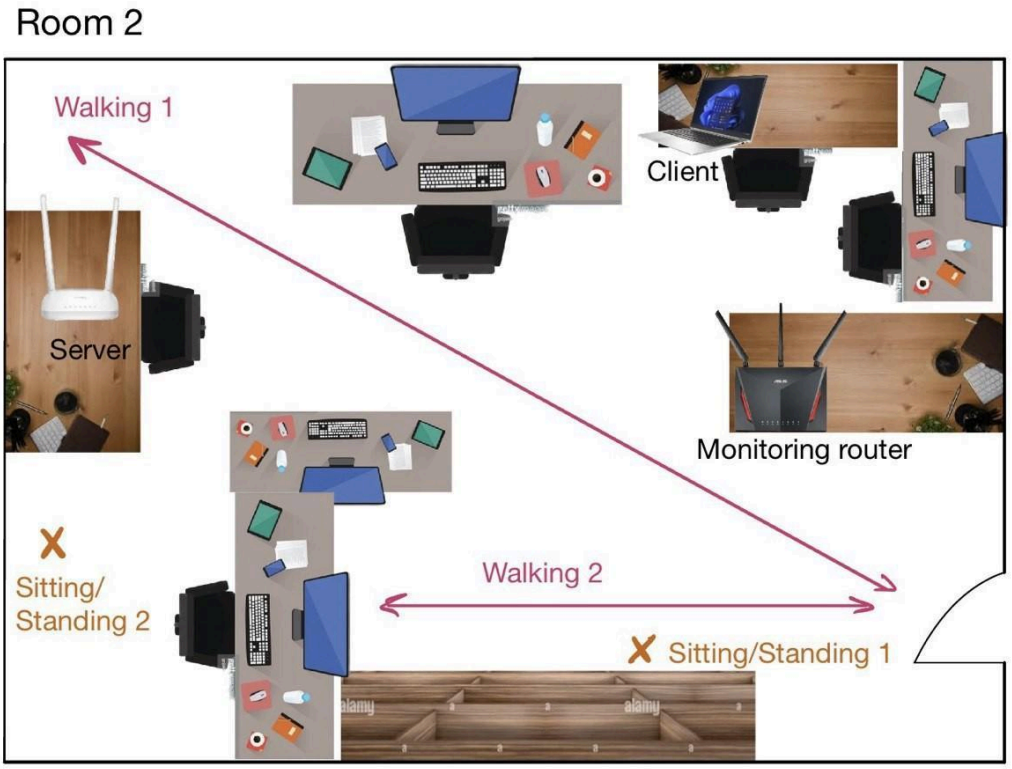


Figure 3.3. Experimental setup and monitored activity trajectories in Room 2 [19].

node. A traffic stream was generated between a transmitter access point and a client laptop using an iPerf3 session, while the Asus monitoring router continuously captured CSI packets transmitted over the wireless channel. The monitoring router employed four receive antennas, enabling multi-antenna CSI acquisition suitable for beamforming-based sensing approaches.

Human activities were performed inside the monitored rooms following predefined trajectories and positions. The monitored actions included walking along multiple paths as well as stationary sitting and standing activities at different positions inside the room. In Room 1, five different walking trajectories and three stationary positions were considered, while Room 2 included two walking paths and two sitting/standing positions. These different movement patterns generate distinct perturbations in the wireless multipath channel, which are reflected in the temporal evolution of CSI amplitude and phase.

Each acquisition session lasted approximately ten minutes per activity, producing a CSI sampling rate of nearly five packets per second. Consequently, the dataset contains long CSI sequences capturing both static and dynamic propagation conditions over time. The relatively

low sampling frequency is sufficient for capturing human-scale motion dynamics while maintaining manageable storage and processing complexity [19].

The CSI structure adopted in this work strongly influences the subsequent preprocessing methodology. Since beamforming techniques rely on coherent phase relationships across receive antennas and OFDM subcarriers, the CSI phase must first be corrected for hardware-induced impairments and synchronization distortions before reliable spatial filtering can be applied. In particular, the discontinuities introduced by removed pilot and null subcarriers, together with antenna-dependent phase offsets and OFDM synchronization errors, motivate the dedicated preprocessing stages introduced in the following sections.

3.3 Overview of the Proposed Phase Preprocessing Pipeline

The proposed CSI preprocessing framework is designed to mitigate the main hardware-induced phase distortions affecting commodity 80 MHz CSI measurements while preserving motion-related propagation information required for beamforming and sensing applications.

The preprocessing pipeline consists of the following stages:

1. Reconstruction of missing pilot subcarrier positions in order to recover a continuous OFDM frequency structure and correctly preserve subcarrier indexing.
2. Removal of constant per-antenna phase offsets caused by independent RF chains and phase-locked loops.
3. Estimation and removal of linear phase distortions across subcarriers caused by timing and sampling synchronization mismatches.

The preprocessing stages are applied sequentially to all CSI packets and antenna streams before beamforming operations. The following sections describe each preprocessing stage in detail.

3.4 Pilot Subcarrier Reconstruction

Pilot and null subcarrier removal introduces discontinuities in the CSI frequency structure, which become problematic for phase-based preprocessing techniques relying on smooth phase evolution across subcarriers. In particular, synchronization impairments such as SFO and STO generate approximately linear phase distortions over the OFDM frequency axis. Missing subcarrier positions may therefore negatively affect subsequent phase regression and beamforming preprocessing stages.

To restore frequency continuity, missing pilot subcarrier positions were reconstructed using linear interpolation between neighboring valid CSI tones. Linear interpolation is widely employed in OFDM channel estimation and pilot-aided reconstruction due to its low computational complexity and its ability to preserve smooth channel variations across adjacent subcarriers [20, 21].

Let the CSI vector corresponding to one packet and one receive antenna be represented as

$$\overline{H} = [H(k_1), H(k_2), \ldots, H(k_N)]$$

where $H(k)$ denotes the complex CSI value at subcarrier index k.

The interpolation procedure was applied independently to CSI amplitude and phase.

For a missing subcarrier position k_m located between neighboring valid subcarriers k_{m-1} and k_{m+1}, linear interpolation was performed according to

$$\hat{x}(k_m) = x(k_1) + \frac{k_m - k_{m-1}}{k_{m+1} - k_{m-1}} [x(k_{m+1}) - x(k_{m-1})]$$

where $x(k)$ represents either CSI amplitude or unwrapped phase.

The OFDM subcarrier indexing was represented using a centered frequency axis ranging from negative to positive subcarrier indices around the DC carrier. In this representation, the DC component corresponds to subcarrier $k = 0$, while neighboring subcarriers are indexed symmetrically around the carrier frequency.

Special attention was required around the central DC region of the OFDM spectrum, where the removed tones correspond to

$$k \in \{-1, 0, 1\}.$$

Since these subcarriers form a contiguous missing region around the carrier frequency, interpolation was performed jointly using the closest valid neighboring subcarriers on both sides of the DC null region. This approach preserves phase continuity near the center frequency and avoids abrupt discontinuities in the reconstructed CSI phase.

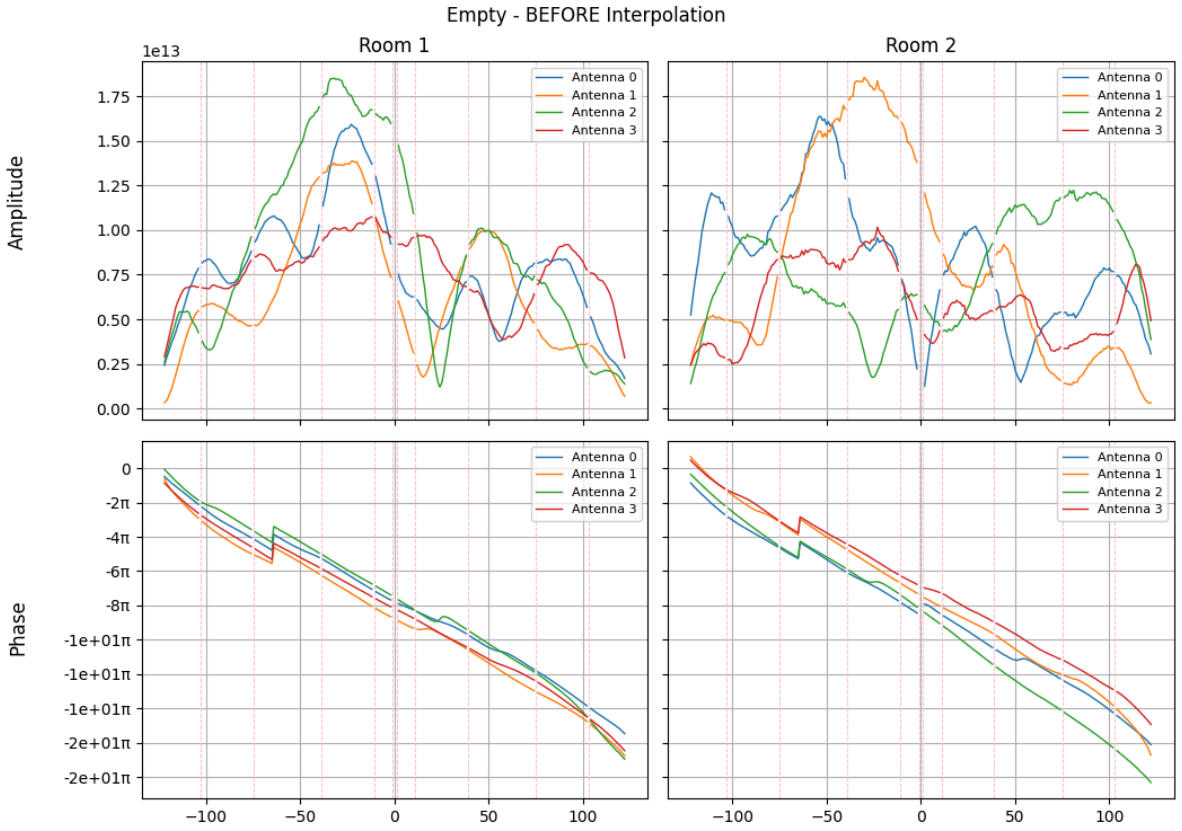
Additional missing pilot tones located inside the OFDM subbands were reconstructed independently using neighboring valid subcarriers while preserving the original OFDM indexing geometry. The reconstructed CSI structure therefore contains 244 subcarriers, including the interpolated pilot positions while still excluding guard-band tones located at the spectrum edges.

The complete implementation of the interpolation procedure is provided in Appendix A.1 (interpolation.py). The reconstruction algorithm first restores the OFDM indexing structure, performs phase unwrapping, reconstructs missing pilot positions through linear interpolation, and finally generates the reconstructed CSI tensors used in the subsequent preprocessing stages.

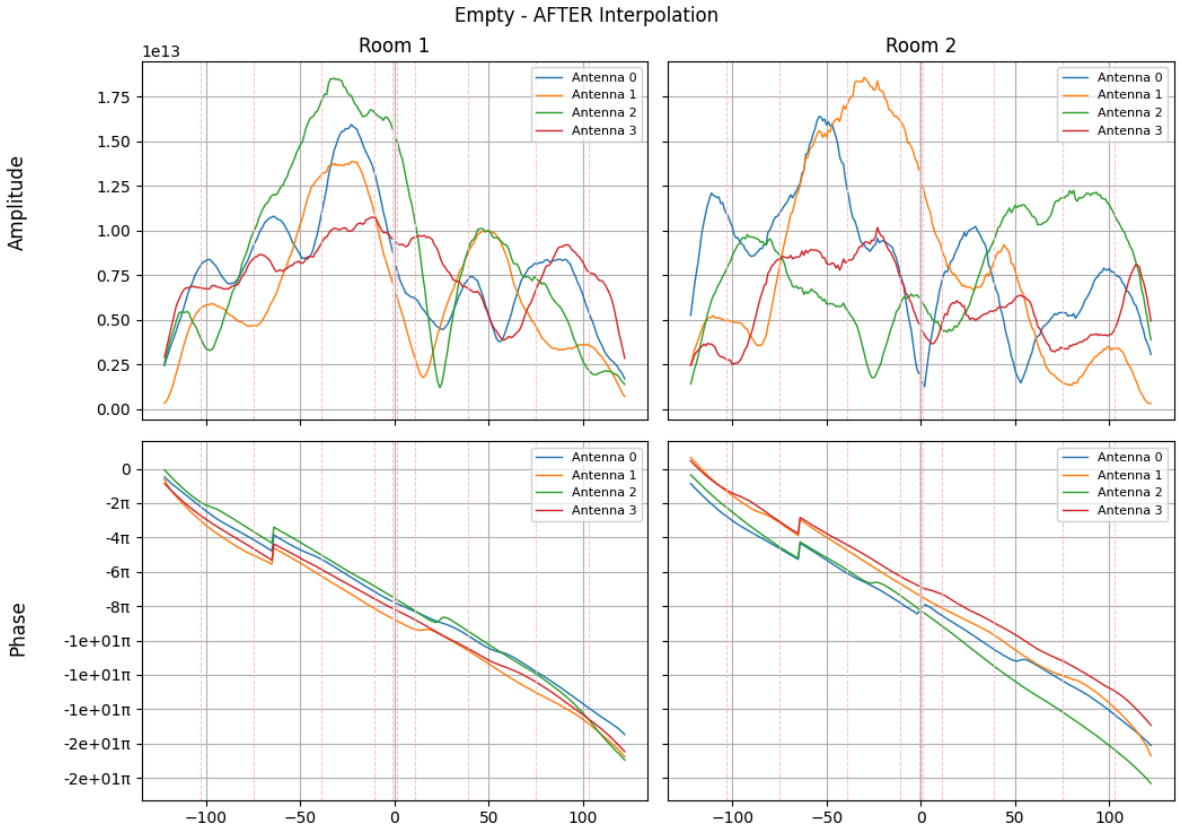
Figure 3.4 (a-h) illustrate representative CSI amplitude and phase responses before and after pilot reconstruction for different activities and environments.

Before interpolation (Figure 3.4 (a, c, e, g)), visible discontinuities appear at the removed pilot and null subcarrier locations. These discontinuities are particularly evident in the CSI phase response, where abrupt gaps interrupt the otherwise smooth linear phase evolution across subcarriers. Such gaps negatively affect subsequent linear regression operations used for SFO/STO compensation.

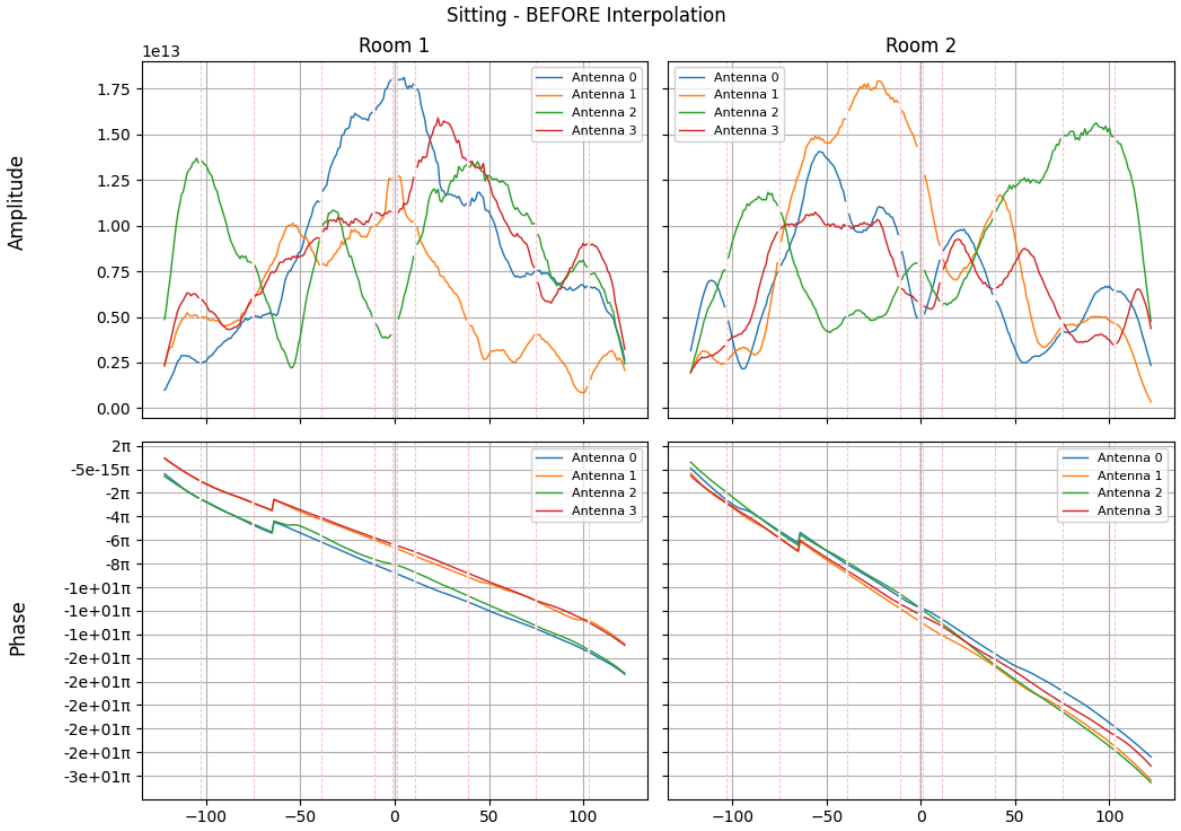
After interpolation (Figure 3.4 (b, d, f, h), the reconstructed CSI responses exhibit significantly improved continuity across the OFDM frequency axis. The reconstructed pilot positions preserve the smooth phase evolution required for later synchronization correction and beamforming operations.


Empty - BEFORE Interpolation
Room 1
Room 2
1e13
Amplitude
Phase
Antenna 0
Antenna 1
Antenna 2
Antenna 3
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
-2π
-4π
-6π
-8π
-1e+01π
-1e+01π
-1e+01π
-2e+01π
-2e+01π
−100
−50
0
50
100

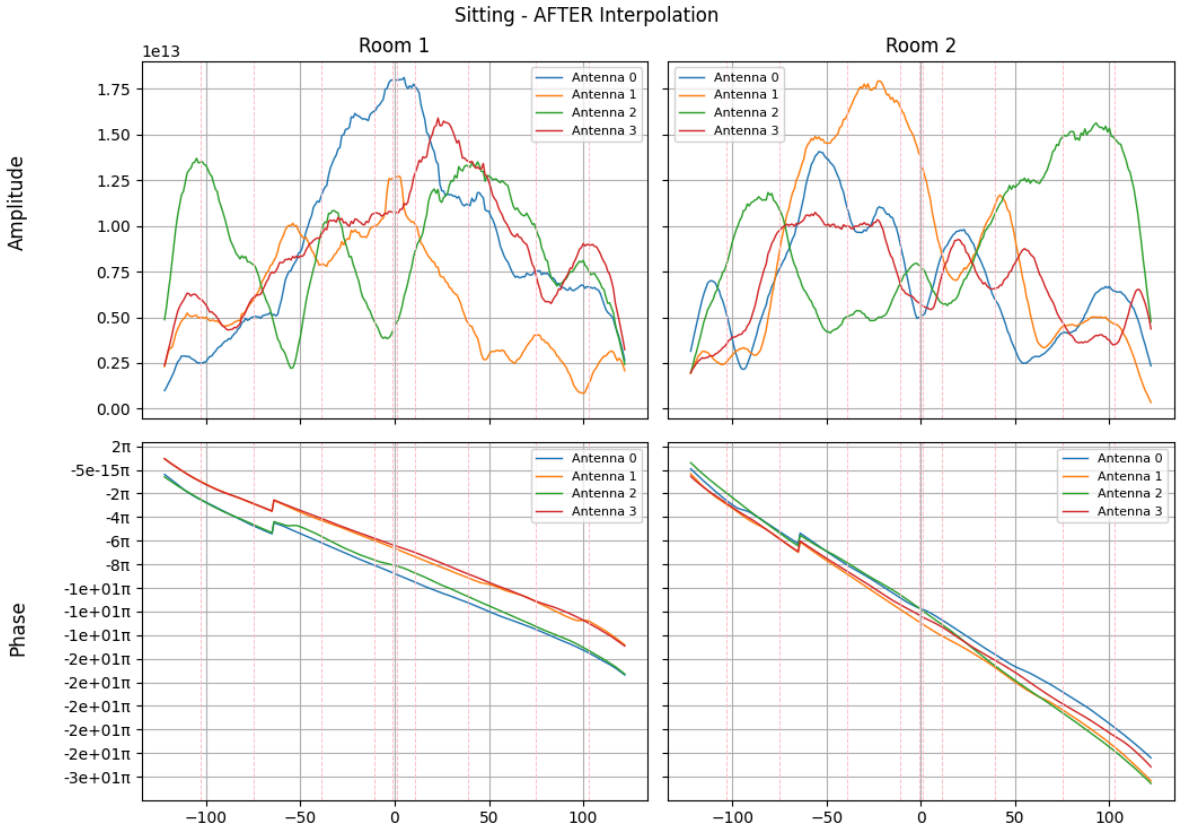
(a)


Empty - AFTER Interpolation
Room 1
Room 2
1e13
Amplitude
Phase
Antenna 0
Antenna 1
Antenna 2
Antenna 3
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
-2π
-4π
-6π
-8π
-1e+01π
-1e+01π
-1e+01π
-2e+01π
-2e+01π
−100
−50
0
50
100

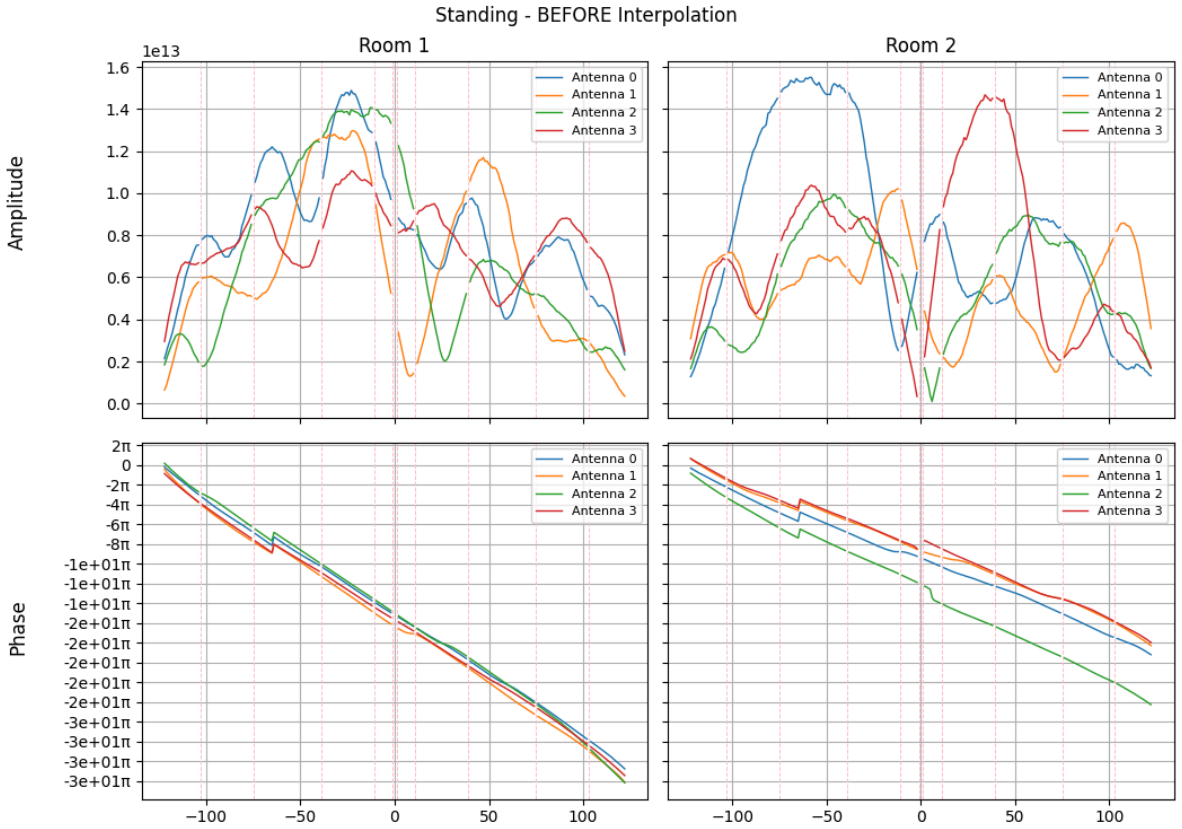
(b)


Sitting - BEFORE Interpolation
Room 1
Room 2
1e13
Amplitude
Phase
Antenna 0
Antenna 1
Antenna 2
Antenna 3

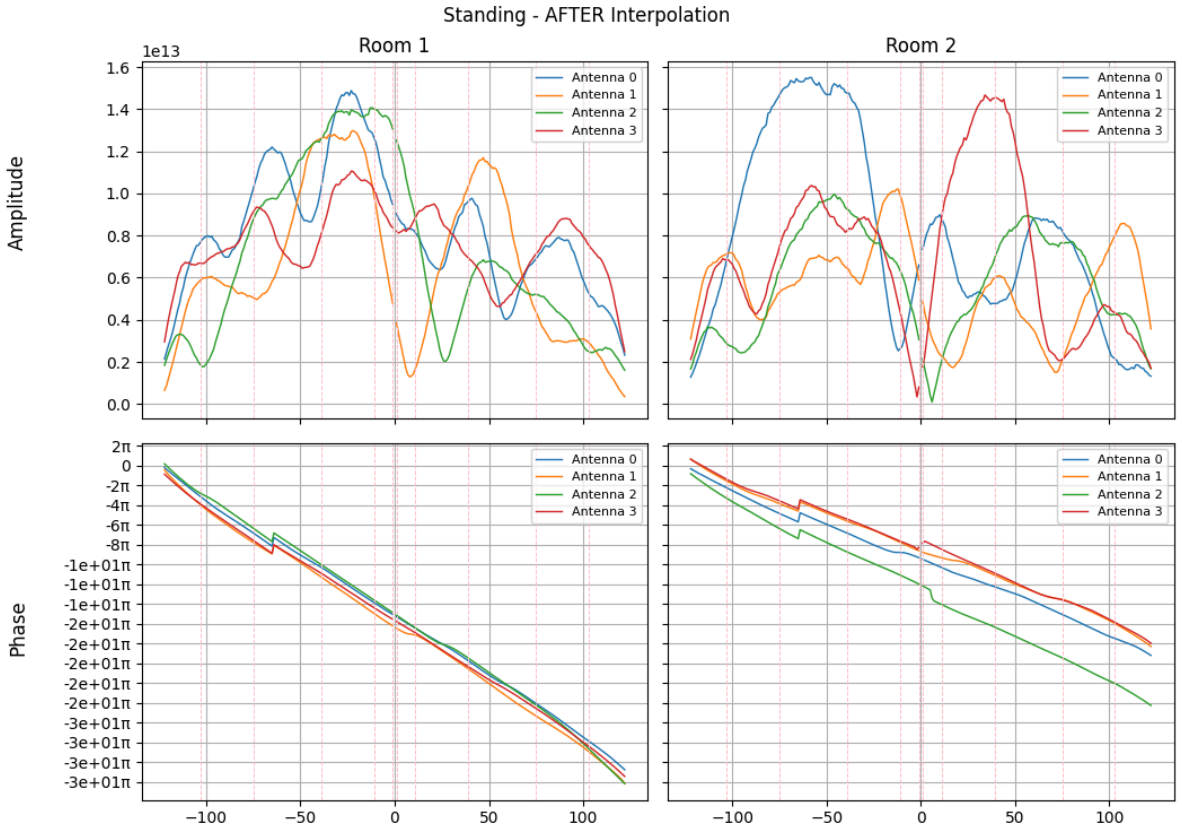
(c)


Sitting - AFTER Interpolation
Room 1
Room 2
1e13
Amplitude
Phase
Antenna 0
Antenna 1
Antenna 2
Antenna 3

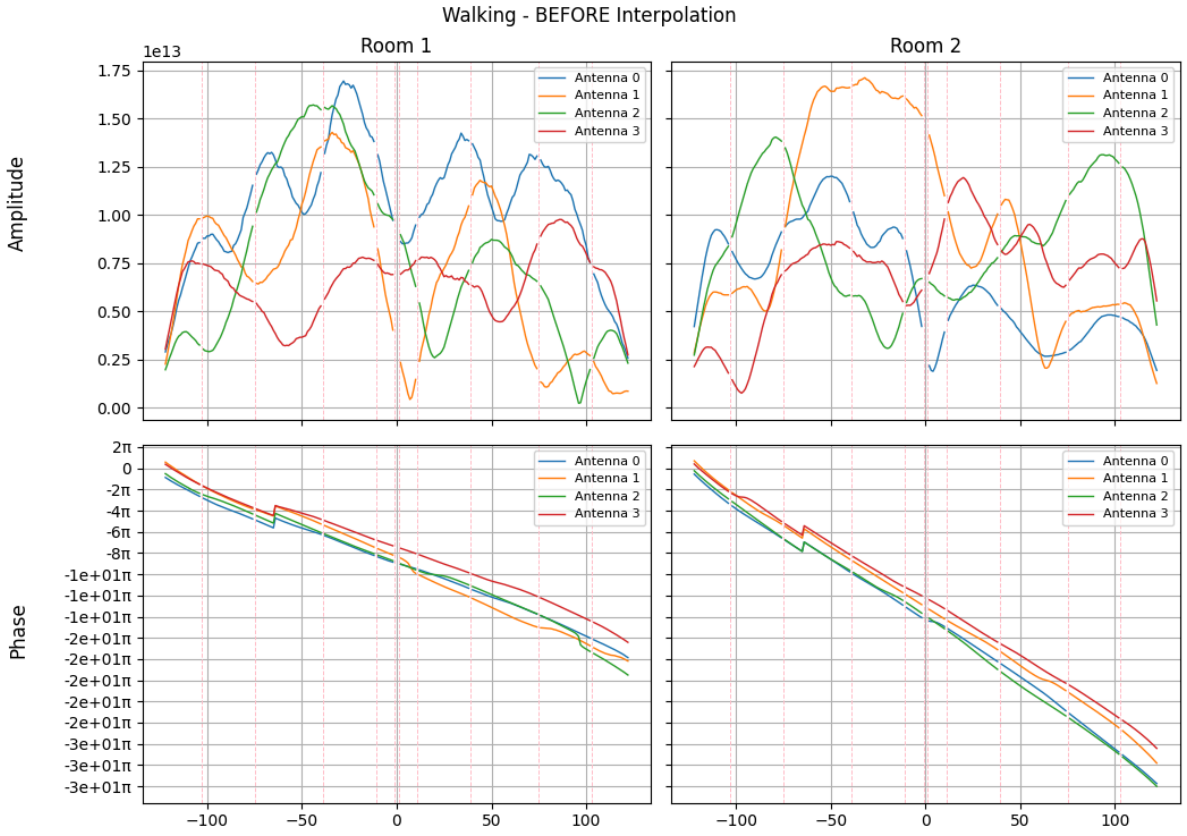
(d)


Standing - BEFORE Interpolation
Room 1
Room 2
Amplitude
Phase
1e13
Antenna 0
Antenna 1
Antenna 2
Antenna 3

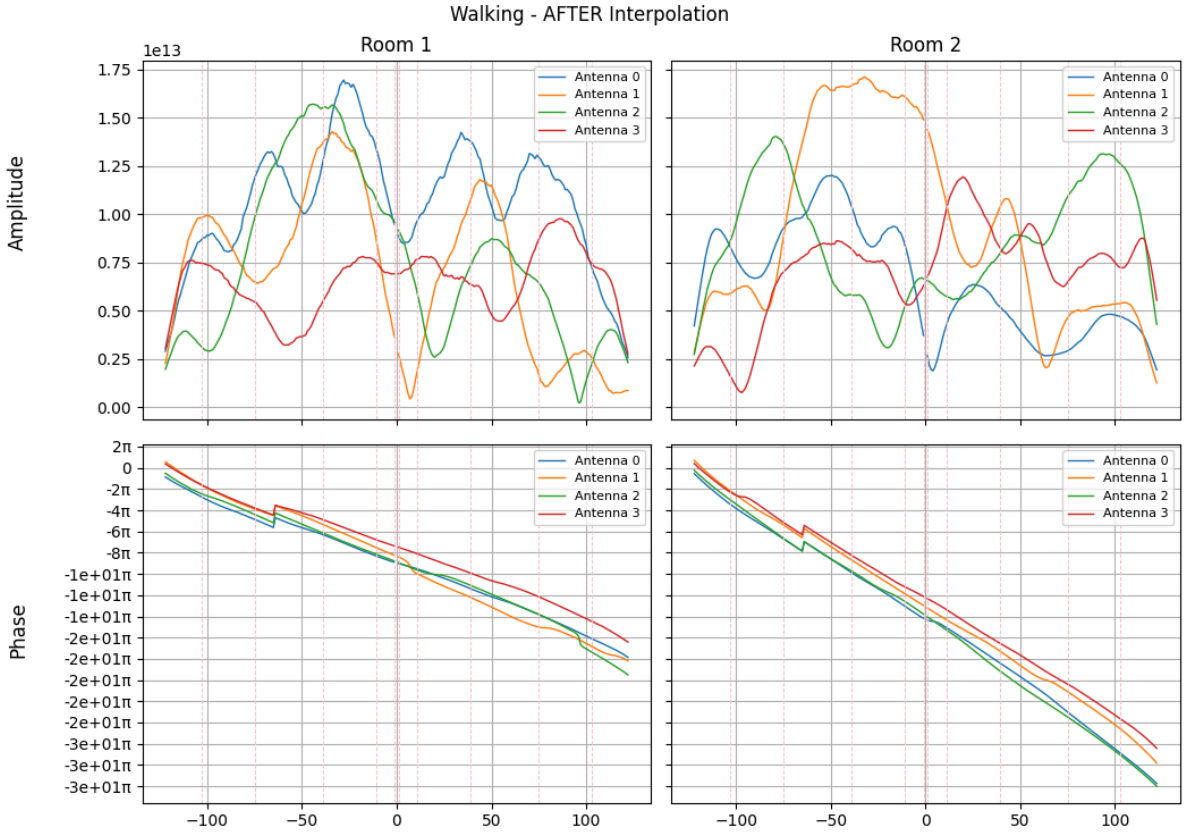
(e)


Standing - AFTER Interpolation
Room 1
Room 2
Amplitude
Phase
1e13
Antenna 0
Antenna 1
Antenna 2
Antenna 3

(f)


Walking - BEFORE Interpolation
Room 1
Room 2
Amplitude
Phase
1e13
Antenna 0
Antenna 1
Antenna 2
Antenna 3

(g)


Walking - AFTER Interpolation
Room 1
Room 2
Amplitude
Phase
1e13
Antenna 0
Antenna 1
Antenna 2
Antenna 3

(h)

Figure 3.4. CSI amplitude and phase responses for packet No 1000 before and after pilot interpolation for different activities in Room 1 and Room 2.
(a) Empty room before interpolation, (b) Empty room after interpolation,
(c) Sitting activity before interpolation, (d) Sitting activity after interpolation,
(e) Standing activity before interpolation, (f) Standing activity after interpolation,
(g) Walking activity before interpolation, (h) Walking activity after interpolation.

The interpolation results show that the proposed reconstruction procedure successfully restores continuity across the OFDM frequency structure without introducing noticeable distortions in either amplitude or phase. Since the interpolated tones are estimated from neighboring subcarriers, the reconstructed CSI preserves the smooth spectral behavior characteristic of indoor wireless channels. This continuity becomes particularly important for the subsequent linear regression preprocessing stage, where accurate estimation of phase slopes across subcarriers is required for SFO and STO mitigation.

3.5 Mitigating Per-Antenna Constant Phase Offset

Commodity WiFi devices employ independent RF chains and phase-locked loops (PLLs) for each receive antenna. As a consequence, CSI measurements obtained from different antennas exhibit constant inter-antenna phase offsets that are unrelated to physical wireless propagation. These hardware-induced phase biases significantly affect phase consistency across receive antennas and may severely degrade beamforming performance and spatial processing accuracy. [15, 22]

Several previous works proposed dedicated calibration procedures to mitigate CSI phase offsets. Existing approaches include wired back-to-back calibration channels, controlled line-of-sight (LoS) reference measurements, external reference antennas, and explicit RF-chain calibration techniques [15, 22, 23].

In contrast, this section presents a lightweight compensation strategy that removes per-antenna constant phase bias directly from the measured CSI without requiring external calibration hardware, controlled LoS references, wired synchronization channels, or modifications to commodity WiFi devices.

3.5.1 Methodology

For each antenna n and packet s, we select a reference subcarrier $k = 0$ and compute:

$$\Theta_{s,k}^{n} = \hat{\theta}_{s,k}^{n} - \hat{\theta}_{s,k=0}^{n}$$

Substituting the model:

$$\Theta_{s,k}^{n} = (\theta_{s,k}^{n} + 2\pi\frac{m_k}{N}\Delta t_s + \gamma_s + \phi^n + Z_{s,k}^{n}) - (\theta_{s,k0}^{n} + 2\pi\frac{m_{k0}}{N}\Delta t_s + \gamma_s + \phi^n + Z_{s,k0}^{n}) =$$

$$= (\theta_{s,k}^{n\,true} - \theta_{s,k0}^{n\,true}) + 2\pi\frac{m_k - m_{k0}}{N}\Delta t_s + (Z_{s,k}^{n} - Z_{s,k0}^{n}).$$

Both the per-antenna constant bias ϕ^n and packet-wide CFO γ_s are eliminated without explicit estimation. In this step we neglect $(Z_{s,k}^{n} - Z_{s,k0}^{n})$.

$$\Theta_{s,k}^{n} = (\theta_{s,k}^{n\,true} - \theta_{s,k0}^{n\,true}) + 2\pi\frac{m_k - m_{k0}}{N}\Delta t_s.$$

The term $2\pi\frac{m_k - m_{k0}}{N}\Delta t_s$ will be addressed in the Step 2.

Now take two antennas n and m. Their calibrated phase difference at subcarrier k is:

$$\Delta\Theta_{s,k}^{n,m} = \Theta_{s,k}^{n} - \Theta_{s,k}^{m}$$

Substitute the definition:

$$= (\theta_{s,k}^{n\,(true)} - \theta_{s,k0}^{n\,(true)}) - (\theta_{s,k}^{m\,(true)} - \theta_{s,k0}^{m\,(true)}) =$$

$$= (\theta_{s,k}^{n\,(true)} - \theta_{s,k}^{m\,(true)}) - (\theta_{s,k0}^{n\,(true)} - \theta_{s,k0}^{m\,(true)})$$

The first bracket is exactly the true inter-antenna phase difference at subcarrier k:

$$\Delta\Theta_{s,k}^{n,m\,(true)}.$$

The second bracket is the true inter-antenna phase difference at the reference subcarrier $k0$:

$$\Delta\Theta_{s,k0}^{n,m\ (true)}.$$

The calibrated inter-antenna difference becomes:

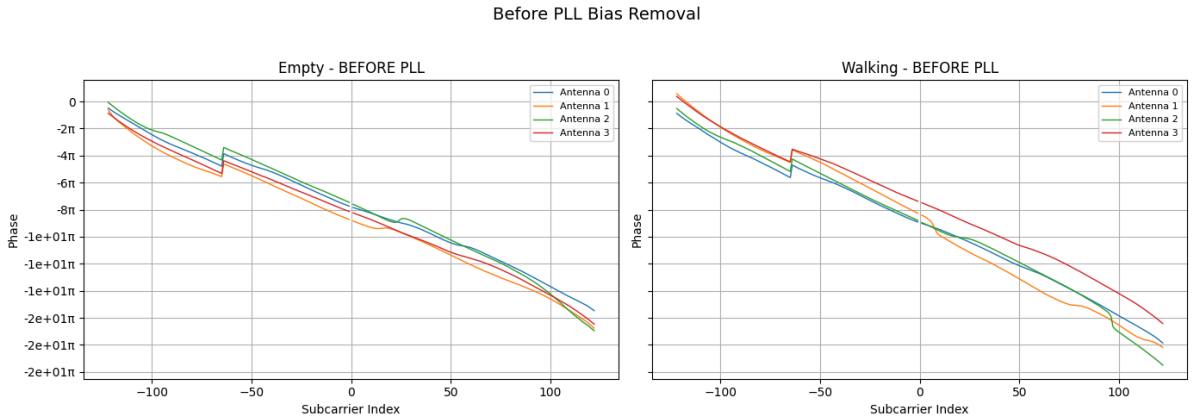
$$\Delta\Theta_{s,k}^{n,m} = \Delta\Theta_{s,k}^{n,m\ (true)} - \Delta\Theta_{s,k0}^{n,m\ (true)}$$

The second term is independent of k and therefore represents a constant spatial offset across subcarriers.

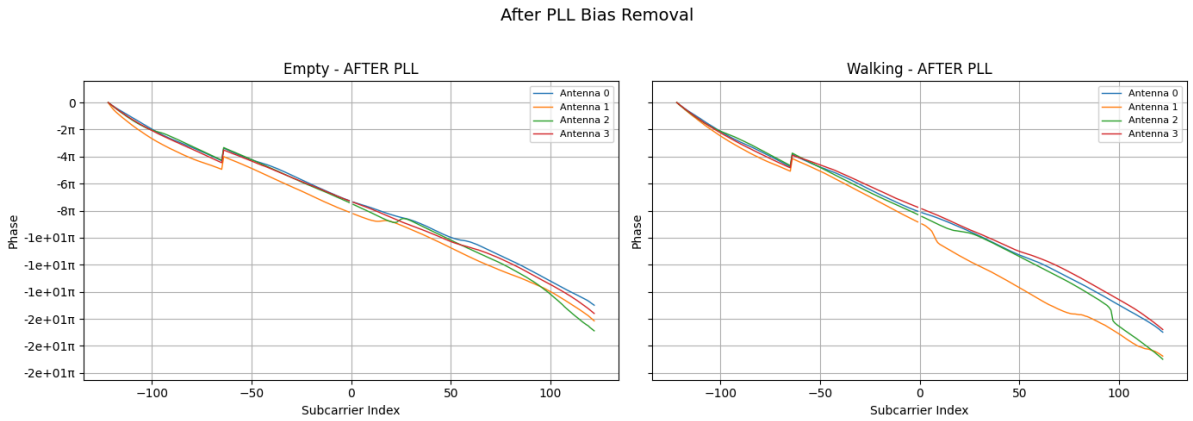
The implemented PLL compensation procedure estimates a reference phase offset independently for each receive antenna and packet, then subtracts this constant bias from all valid subcarriers belonging to the corresponding antenna stream. In the proposed implementation, the first valid subcarrier of each packet is used as the phase reference for estimating the antenna-dependent offset. The correction is then applied across the complete CSI phase vector in order to align inter-antenna phase responses while preserving the relative phase evolution across subcarriers. The complete implementation of the PLL compensation stage is provided in Appendix A.1 (PLL_bias.py).

3.5.2 Results and Discussion

As shown in Figure 3.5 (a), before compensation the phase curves of different antennas are vertically shifted relative to each other. After reference subtraction Figure 3.5 (b), these vertical offsets disappear and the curves align at the reference subcarrier. Importantly, the shape of each curve remains unchanged. Since the physically meaningful information is encoded in the phase variation across the OFDM subcarriers rather than in the absolute phase, re-referencing the phase across subcarriers effectively removes per-antenna constant bias while preserving the wideband spatial structure required for beamforming.



(a)



(b)

Figure 3.5. CSI phase response for packet 1000 in Room 1 before and after PLL bias compensation during the empty room activity.
(a) Before PLL bias removal and (b) after PLL bias removal.

3.6 SFO and STO Mitigation Using Linear Regression Transformation

Sampling Frequency Offset (SFO) and Symbol Timing Offset (STO) introduce a frequency-dependent phase distortion across OFDM subcarriers. According to the received phase model presented in Section 3.1, these synchronization impairments generate an approximately linear phase component with respect to the subcarrier index. As a result, the

measured CSI phase exhibits a deterministic slope superimposed on the true channel-dependent phase response.

To mitigate this effect, a Linear Regression Transformation (LRT) was applied independently to each CSI packet and receive antenna. Similar linear phase sanitization approaches have been widely adopted in CSI-based sensing and human activity recognition literature for compensating synchronization-induced phase distortions [4].

3.6.1 Linear Regression Transformation

Let $\widehat{\Theta} \in \mathbb{R}^{S\times K}$ denote the measured CSI phase matrix, where S is the number of OFDM symbols (or packets) and K is the number of subcarriers. The element in the s-th row and k-th column is the measured phase $\widehat{\theta}_{s,k}$.

The dominant synchronization impairments due to Sampling Frequency Offset (SFO) and Sampling Time Offset (STO) introduce a phase distortion that is linear with respect to the subcarrier index k. Therefore, for each OFDM symbol s, the measured phase across subcarriers can be modeled as a linear trend plus residual components.

For a fixed symbol s, we consider the phase vector

$$\widehat{\Theta}_s = [\widehat{\theta}_{s,1}, \widehat{\theta}_{s,2}, \ldots, \widehat{\theta}_{s,K}]^T.$$

Since SFO and STO generate a linear phase rotation across subcarriers, the dominant distortion can be approximated by a first-order model of the form

$$r_s(k) = \varepsilon_s k + \tau_s,$$

where ε_s represents the phase slope associated with SFO/STO and τ_s represents the constant phase offset term.

To estimate these parameters, a linear regression is performed independently for each symbol s. The goal is to find the coefficients ε_s and τ_s that minimize the squared error between the

measured phase and the linear model:

$$min(\varepsilon_s, \tau_s) \sum_{k=1}^{K} (\widehat{\theta}_{s,k} - (\varepsilon_s k + \tau_s))^2.$$

Let $\bar{k}$ denote the mean value of the subcarrier indices,

$$\bar{k} = \frac{1}{K} \sum_{k=1}^{K} k,$$

and let $\overline{\theta_s}$ denote the average phase of symbol s,

$$\overline{\theta_s} = \frac{1}{K} \sum_{k=1}^{K} \widehat{\theta}_{s,k}.$$

The least-squares solution for the slope ε_s is then given by

$$\varepsilon_s = \frac{\sum_{k=1}^{K} (\widehat{\theta}_{s,k} - \bar{\theta}_s)(k - \bar{k})}{\sum_{k=1}^{K} (k - \bar{k})^2}.$$

Once ε_s is obtained, the offset τ_s is computed as

$$\tau_s = \overline{\theta_s} - \varepsilon_s \bar{k}.$$

The linear regression function that captures the SFO/STO-induced distortion for symbol s is therefore

$$r_s(k) = \varepsilon_s k + \tau_s,$$

After estimating this linear trend, the sanitized phase matrix $\widehat{\Theta} \in \mathbb{R}^{S \times K}$ is obtained by subtracting the regression line from the measured phase:

$$\Theta_{s,k} = \widehat{\theta}_{s,k} - (\varepsilon_s k + \tau_s).$$

In matrix form, for each row s, this operation corresponds to removing the best linear approximation of $\widehat{\Theta}_s$ with respect to the subcarrier index. As a result, the linear phase component introduced by SFO and STO is mitigated, and the residual phase $\Theta_{s,k}$ is approximately free from deterministic frequency-dependent distortion [4].

3.6.2 Subband Stitching in the 80 MHz CSI Dataset

After pilot reconstruction, the CSI dataset contains 244 active OFDM subcarriers corresponding to the usable tones of the 80 MHz IEEE 802.11ac channel. Although these subcarriers form a single wideband CSI vector, the 80 MHz channel is internally composed of multiple stitched OFDM subbands.

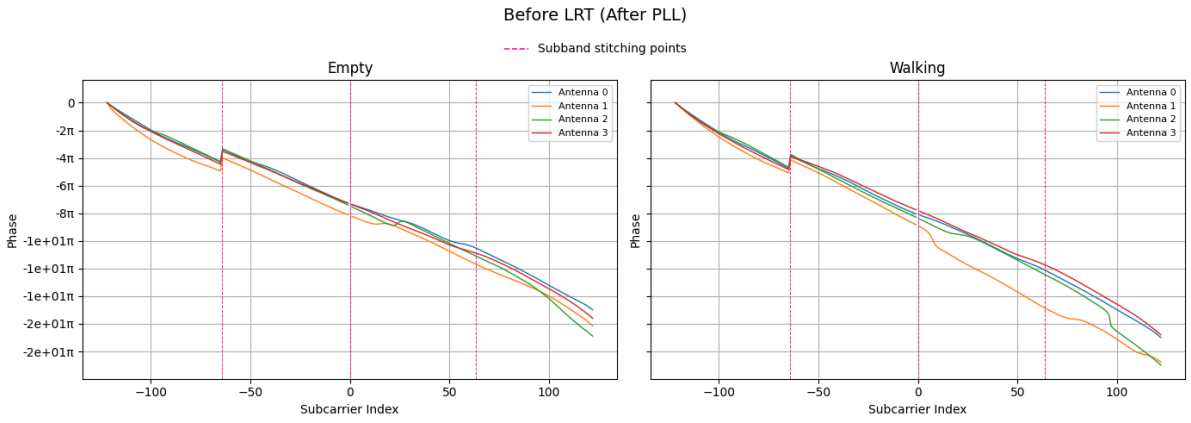
In practice, commodity WiFi chipsets process wideband CSI through multiple frequency segments that are later combined into a single CSI measurement. As a consequence, discontinuities may appear at the boundaries between adjacent subbands due to independent synchronization and hardware processing effects. These discontinuities become clearly visible in the CSI phase response after PLL bias compensation.

Figure 3.6 illustrates the measured CSI phase before LRT correction for the empty-room and walking scenarios in Room 1. Clear phase discontinuities can be observed near the stitching boundaries, particularly at the subcarrier index $k = -64$, where a pronounced phase jump appears across all receive antennas. Similar discontinuities are also visible near the other stitching points highlighted in the figure.

Because these discontinuities break the global linearity of the phase response, applying a single regression model over the entire 244-subcarrier CSI vector would lead to inaccurate slope estimation and imperfect SFO/STO compensation. For this reason, the proposed framework applies the Linear Regression Transformation independently within each OFDM subband segment.

The adopted segmentation divides the CSI vector into four contiguous subcarrier regions corresponding to the stitched subbands of the reconstructed 80 MHz CSI structure. Linear regression is then performed separately on each segment, allowing the synchronization-induced phase slope to be estimated locally while preserving the subband discontinuities.

The complete implementation of the segmentwise LRT preprocessing procedure is provided in Appendix A.1 (LRT.py).



(a) (b)

Figure 3.6. CSI phase after PLL bias compensation and before Linear Regression Transformation for packet 1000 in Room 1: (a) empty environment and (b) walking activity. Dashed vertical lines indicate subband stitching boundaries within the reconstructed 80 MHz CSI structure.

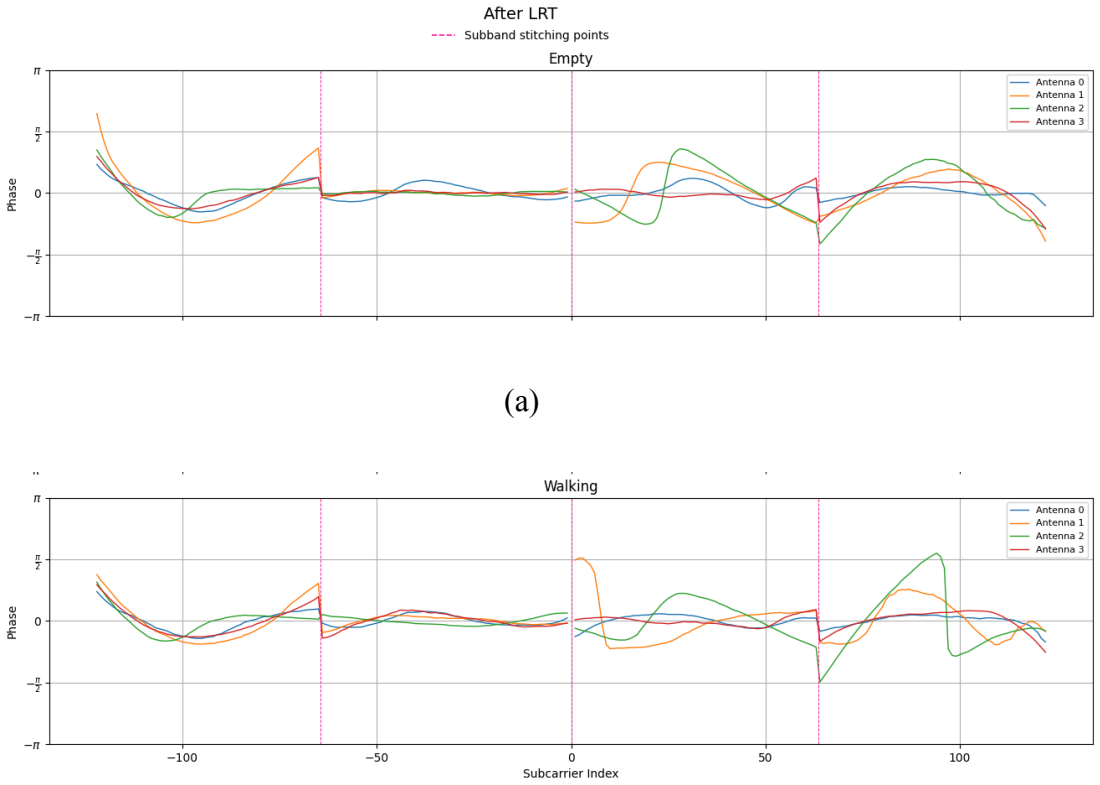
3.6.3 Segmentwise LRT Results

Figure 3.7 illustrates the effect of the segmentwise Linear Regression Transformation on the CSI phase measured across OFDM subcarriers for the empty-room and walking activities in Room 1.

Before correction, the CSI phase exhibits a clear linear trend with respect to the subcarrier index within each subband segment. This behavior is mainly caused by synchronization impairments such as Sampling Frequency Offset (SFO) and Symbol Timing Offset (STO), which introduce deterministic frequency-dependent phase distortions.

After subtracting the estimated regression line from each segment independently, the corrected phase no longer presents a dominant linear slope, as shown in Figure 3.7. Instead, the phase oscillates around zero while preserving the intrinsic channel-dependent variations associated with the wireless propagation environment.

In both scenarios, the segmentwise LRT effectively mitigates the synchronization-induced phase distortion while maintaining the relative phase dynamics across antennas and



(a)

(b)

Figure 3.7. CSI phase after segmentwise Linear Regression Transformation for packet 1000 in Room 1: (a) empty environment after LRT, and (b) walking activity after LRT. Dashed vertical lines indicate subband stitching boundaries.

subcarriers. At the same time, the residual phase fluctuations differ between the empty and walking cases, reflecting the impact of human motion on the wireless channel response. This demonstrates that the proposed preprocessing stage removes the deterministic synchronization component while preserving motion-sensitive phase variations relevant for sensing and beamforming applications.

Because the regression is applied independently within each subband, the method also avoids distortion caused by the stitching discontinuities visible in the raw CSI phase. Consequently, the resulting phase becomes substantially more stable and spatially consistent across the reconstructed 80 MHz CSI bandwidth.

3.7 Phase Preprocessing Results

This section presents the overall impact of the proposed CSI phase preprocessing pipeline on the reconstructed 80 MHz CSI measurements. The preprocessing stages include pilot subcarrier reconstruction, per-antenna PLL bias mitigation, and segmentwise Linear Regression Transformation (LRT) for SFO/STO compensation.

Figure 3.8 illustrates the CSI magnitude heatmaps before pilot reconstruction for the empty-room and walking scenarios in Room 1. The horizontal axis corresponds to the OFDM subcarrier index, while the vertical axis represents the packet index. Bright vertical gaps are visible at several subcarrier positions due to the removed pilot tones, null subcarriers, and DC carrier present in the original dataset preprocessing procedure. These discontinuities interrupt the spectral continuity of the CSI measurements and produce fragmented frequency-domain representations.

In addition, the walking scenario exhibits significantly larger temporal fluctuations across packets compared to the empty-room case. Human motion continuously modifies the wireless propagation paths, generating dynamic variations in CSI magnitude over both time and frequency.

After applying the proposed pilot reconstruction procedure, the CSI frequency response becomes substantially more continuous, as shown in Figure 3.9. The previously missing subcarriers are recovered through interpolation, eliminating the large spectral gaps visible in the raw CSI representation. The reconstructed CSI exhibits smoother transitions across neighboring subcarriers while preserving the dominant channel characteristics of the original measurements.

The differences between the empty-room and walking activities also become more visually distinguishable after preprocessing. In the empty-room scenario, the CSI magnitude remains relatively stable across packets, reflecting the static indoor propagation environment. In contrast, the walking activity produces stronger temporal fluctuations and irregular spectral variations caused by continuous motion-induced multipath changes.

Overall, the preprocessing pipeline improves the spatial and spectral consistency of the CSI measurements while preserving motion-related channel dynamics relevant for sensing and beamforming applications. The resulting CSI representations provide a substantially cleaner

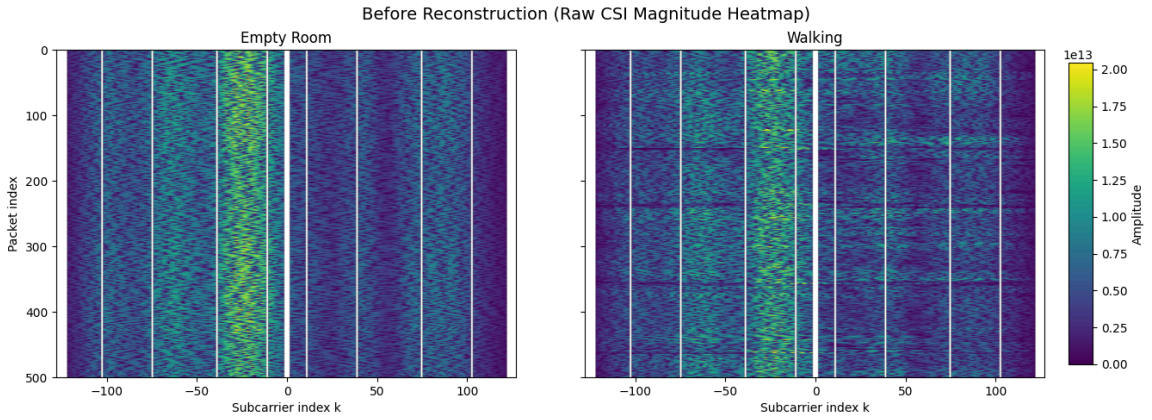


Figure 3.8. Raw CSI magnitude heatmaps before pilot reconstruction for Room 1: (a) empty-room activity and (b) walking activity. Bright vertical lines indicate removed pilot, null, guard and DC subcarriers in the original CSI dataset.

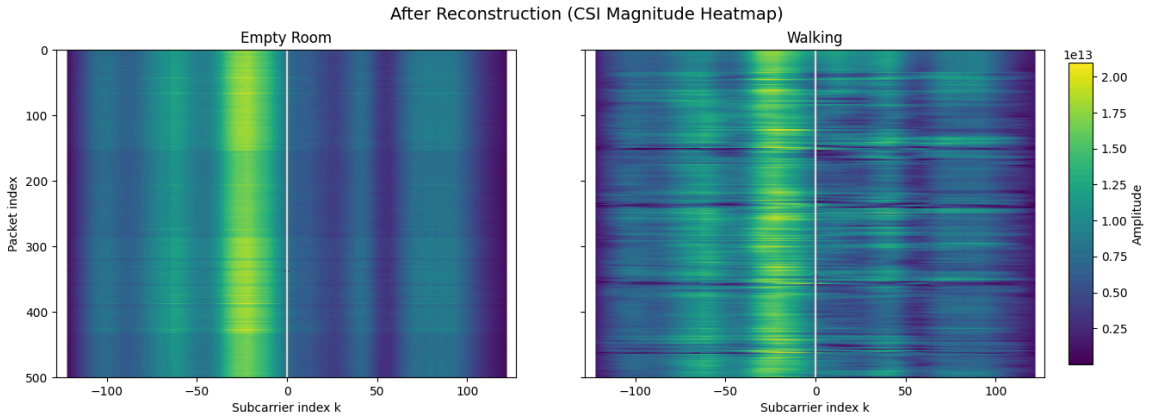


Figure 3.9. CSI magnitude heatmaps after pilot subcarrier reconstruction for Room 1: (a) empty-room activity and (b) walking activity.

basis for subsequent phase correction, digital receive beamforming, and activity recognition stages.

Chapter 4

Beamforming and CSI DistanceBased Motion Analysis

4.1 Beamforming Based HAR Framework

Receive beamforming combines CSI measurements from multiple antennas using complex weights in order to generate virtual directional beams. Instead of physically steering the antenna array, beam steering is performed digitally by modifying the relative phase between antenna signals. This allows the receiver to spatially emphasize reflections arriving from different directions inside the environment.

Let

$$\widehat{h}_m(n) = [\widehat{h}_{m,1}(n), \widehat{h}_{m,2}(n), \ldots, \widehat{h}_{m,N_{sub}}(n)]$$

denote the CSI vector corresponding to receive antenna m at packet index n where N_{sub} is the number of OFDM subcarriers. Each CSI component is complex-valued and contains both amplitude and phase information.

To enhance sensitivity to environmental changes, the approach proposed in [24] applies digital receive beamforming directly to the CSI measurements. Beamforming combines the CSI vectors from multiple antennas using complex weights. This operation acts as a spatial filter that emphasizes signals arriving from certain directions while suppressing others. The beamformed CSI vector at packet n is computed as a weighted combination of the CSI vectors from all receive antennas,

$$y_h^{bf}(n) = w_1\widehat{h}_1(n) + w_2\widehat{h}_2(n) + \ldots w_N\widehat{h}_N(n),$$

where N is the number of receive antennas, $\widehat{h}_i(n)$ represents the CSI vector measured at antenna i, and w_i is the complex beamforming weight applied to that antenna. The resulting vector $y_h^{bf}(n)$ represents the CSI after spatial filtering.

In the experimental implementation considered in this work, beamforming is performed using two receive antennas. In this case the beamforming equation simplifies to

$$y_h^{bf}(n) = w_1\widehat{h}_1(n) + w_2\widehat{h}_2(n).$$

The first antenna weight is fixed as

$$w_1 = 1,$$

while the second antenna weight is defined as a complex phase rotation

$$w_1 = e^{j\theta}.$$

The parameter θ controls the relative phase shift between the antenna signals. By varying θ over the interval

$$0 \leq \theta < 2\pi$$

multiple virtual beams can be generated. Each value of θ corresponds to a different spatial filtering configuration, effectively steering the receiver sensitivity toward different directions in space.

For each packet n and subcarrier k, the beamformed CSI value becomes

$$y_h^{bf}(n,k) = \widehat{h}_{1,k}(n) + e^{j\theta}\widehat{h}_{2,k}(n).$$

Human motion dynamically alters indoor multipath propagation. Depending on the selected beamforming phase θ, reflections generated by moving bodies may combine either constructively or destructively. Consequently, some beam directions become substantially more sensitive to motion-induced channel variations than others.

The objective of the proposed framework is therefore not conventional communication-oriented beamforming optimization such as SNR maximization, but rather motion enhancement. For each temporal window, multiple beamforming directions are evaluated and the beam producing the strongest CSI temporal variation is selected as the optimal sensing beam.

This approach allows the beamformer to spatially emphasize dynamic multipath components associated with human motion while reducing the influence of static environmental reflections. As a result, the beamformed CSI becomes substantially more sensitive to activity-dependent channel fluctuations, enabling more robust motion analysis and activity recognition.

4.2 Beamformed CSI Distance Construction

To quantify how strongly the beamformed CSI changes over time, a distance-based metric is introduced. The Euclidean distance is a natural choice because it measures the magnitude of variation between two complex-valued CSI observations in a direct and computationally efficient way. In the context of WiFi sensing, small distances indicate that the channel remains nearly unchanged between consecutive packets, whereas larger distances indicate stronger temporal variation caused by human movement.

Let $y_h^{bf}(n)$ denote the beamformed CSI vector for packet n. The Euclidean CSI distance between two consecutive beamformed packet indices n_1 and n_2 is defined as

$$d(y_h^{bf}(n_1), y_h^{bf}(n_2)) = \sqrt{\sum_{k=1}^{N} \left[w_i \left(\widehat{h}^i(n_1) - \widehat{h}^i(n_2) \right) \right]^2},$$

where N is the number of receive antennas, w_i is the complex beamforming weight associated with antenna i, and $\widehat{h}^i(n)$ is the CSI vector measured at antenna i for packet n [24].

This distance measures how much the beamformed CSI changes between two packet observations after spatial combining. Since the CSI vectors are complex-valued, the distance

captures variations in both amplitude and phase. When the environment is static, consecutive CSI realizations are nearly similar and the distance remains low. When a person moves inside the environment, the multipath structure changes and the distance increases accordingly.

In the present framework, the distance is computed over consecutive packets inside a temporal window. Let W denote the number of packets in the window. For a fixed beamforming direction, the CSI distance is computed between successive packet pairs in that window and then averaged to obtain a single score for the window. This produces a motion-sensitive measure that reflects the temporal variability of the beamformed channel rather than a single packet-to-packet fluctuation.

The resulting average distance is used as the beam quality criterion: beamforming directions that produce larger mean CSI distances are considered more sensitive to human motion and therefore more suitable for activity analysis. This is consistent with the approach in [24], where receive beamforming weights are selected to maximize sensitivity to channel variations induced by human movement.

4.3 Experimental Beamforming Configuration

The beamforming and CSI distance computation were implemented on the reconstructed CSI measurements obtained after the preprocessing stages described in Chapter 3. Since the monitoring router provides four receive antennas, beamforming was evaluated across all pairwise antenna combinations in order to analyze how spatial diversity influences motion sensitivity.

The following antenna pairs were tested:

$$(0,1), (0,2), (0,3), (1,2), (1,3), (2,3).$$

For each pair, beamforming was performed using the two-antenna formulation introduced in Section 4.1. The first antenna weight was fixed to unity, while the second antenna weight was varied as a complex phase rotation. This generated a family of virtual beams, each corresponding to a different spatial filtering configuration.

Beamforming optimization was performed over temporal sliding windows of CSI packets. The window size W=16 was selected as a compromise between motion sensitivity and robustness against noise fluctuations. Smaller windows tend to produce unstable beam estimates because they rely on too few packets, while excessively large windows smooth out short-term motion dynamics and reduce responsiveness to activity changes.

For each temporal window, the processing pipeline follows a fixed sequence. First, the CSI vectors of the selected antenna pair are combined using a candidate beamforming phase θ. Second, the beamformed CSI distance is computed between consecutive packets using the Euclidean metric defined in the previous section. Third, the distances are averaged over the window to obtain a mean motion score for that beam direction. Finally, the beamforming phase that maximizes the mean CSI distance is selected as the optimal sensing beam for that window.

This procedure is repeated independently for all candidate antenna pairs. The selected beam directions and corresponding distance values are then stored for later analysis. In addition to the distance values, the evolution of the optimal beam phase over time is also retained, since it provides useful information about how the most motion-sensitive spatial direction changes across different activities and environments.

To avoid abrupt beam switching caused by isolated noisy measurements, the implementation favors temporal continuity between consecutive windows. In practice, the selected beam is encouraged to remain close to the previously chosen optimal beam unless a different direction produces a clearly stronger CSI variation. This makes the beamforming response more stable while preserving its sensitivity to motion-induced channel changes.

The complete implementation of the beam search, windowed CSI distance computation, and optimal beam selection is provided in Appendix A.2 (csi_distance.py).

4.4 CSI Distance Analysis Across Activities

Figure 4.1 compares the beamformed CSI distance traces for the four activity classes in Rooms 1 and 2 across all antenna pairs. The general trend is consistent in every subplot: walking produces the highest CSI distance values and the strongest temporal fluctuations,

while the empty-room condition remains at the lowest and most stable level. This is exactly the expected behavior for a motion-sensitive beamformed feature, since the Euclidean distance grows when the channel varies more strongly between consecutive windows. In the receive-beamforming framework of [24], this is the quantity used to identify the beam direction that best captures human motion, which makes the distance signal a natural intermediate representation for activity analysis.

The empty-room traces remain comparatively compact because the propagation environment is dominated by static reflections from walls, furniture, and other fixed objects. Without a moving person in the scene, consecutive CSI windows are similar, so the distance remains low apart from small fluctuations caused by residual noise and hardware imperfection. Sitting and standing produce an intermediate response. They perturb the channel more than the empty-room case because the human body still affects the propagation paths, but the changes are much smaller than during walking, so the distance traces remain noticeably below the walking curves.

Across both rooms, walking is also the most irregular class, with denser peaks and larger short-term variability. This reflects the fact that human locomotion continuously changes the geometry of the multipath environment: reflections shift in delay, phase, and attenuation as the body moves through the room, and the beamformed CSI changes accordingly. Standing is usually more variable than sitting, especially when small posture adjustments or involuntary body motions are present, but the two classes still remain much closer to the empty-room baseline than to walking.

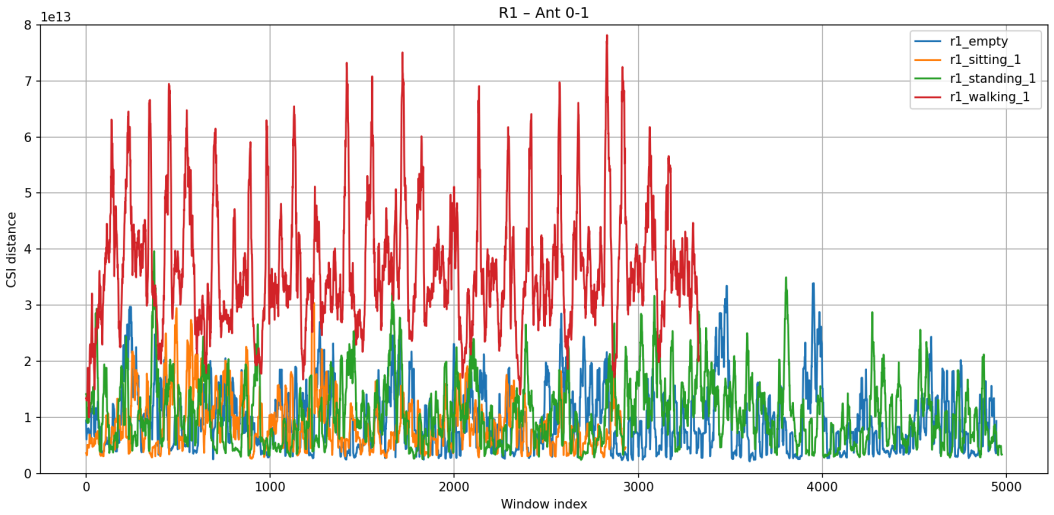
The comparison between antenna pairs shows that pair-specific differences exist, but no pair is uniformly dominant in every room and for every activity. Some pairs produce slightly tighter clustering for the static classes, while others yield stronger peaks for the dynamic class. In Room 1, the separation between walking and the stationary activities is clear for all pairs, while in Room 2 the stationary classes sometimes overlap more and the trace spread depends more strongly on the selected antenna combination. This indicates that the antenna geometry and the virtual beam direction influence the exact magnitude of the distance signal, even though the activity ordering itself remains stable.

Figure 4.2 shows the corresponding optimal beamforming phase θ for the same antenna pairs and activities. These sequences should not be interpreted as physical angles of motion; rather,

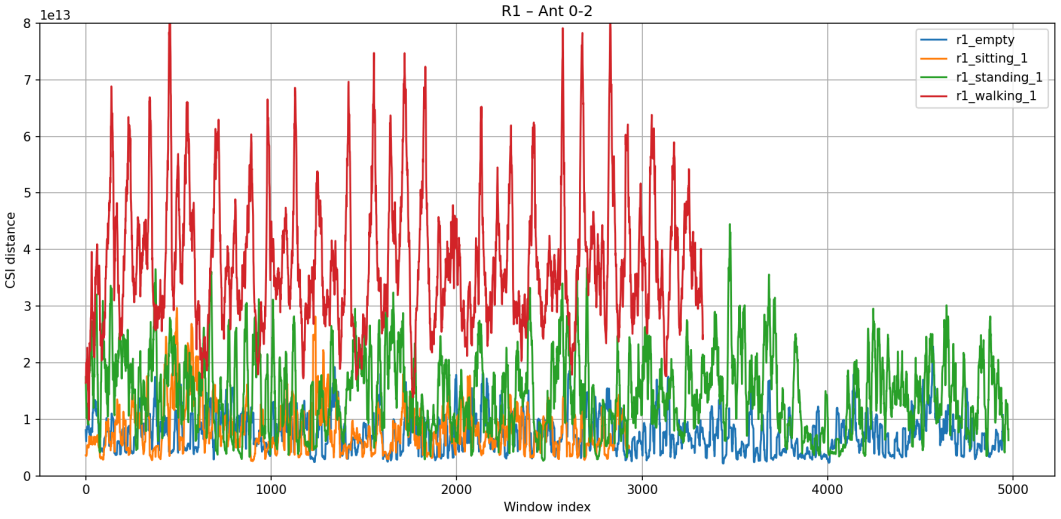
they represent the selected complex beamforming weight. Because the beam search is repeated over sliding windows and constrained for temporal continuity, the selected phase evolves piecewise over time and may jump when a different beam direction becomes more sensitive in the current window. The result is a set of wrapped phase trajectories that encode how the most motion-sensitive virtual beam changes across the recording.

The theta plots reinforce the distance-based analysis. In the static recordings, the selected phase tends to remain within a more limited region for longer intervals, whereas walking produces more frequent transitions and a broader spread of selected phase values. Standing usually falls between these two extremes. In other words, the CSI distance captures how strongly the channel changes, while the optimal θ reveals which virtual beam direction is responsible for that change. The two views are therefore complementary: one measures motion intensity, and the other reflects spatial sensitivity.

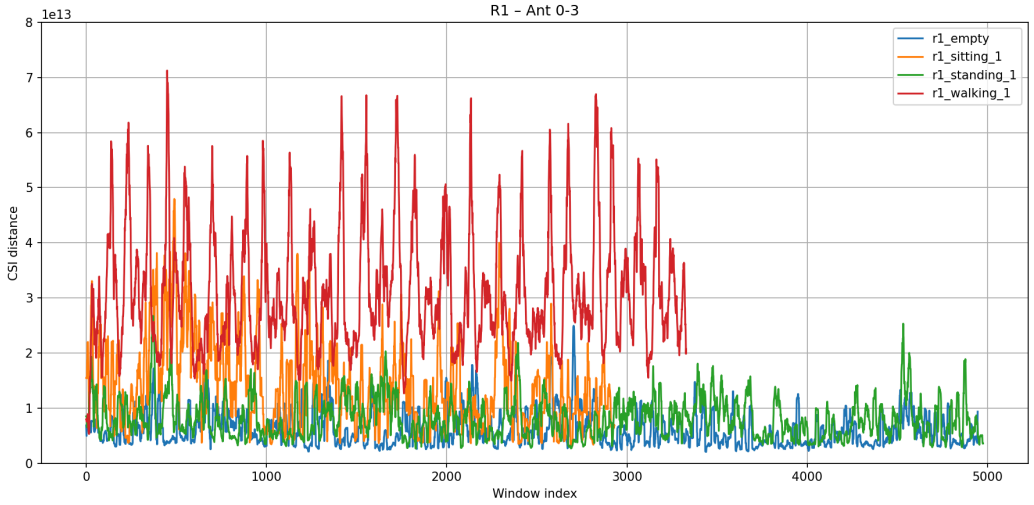
Overall, the results show that the beamformed CSI distance is a robust motion-sensitive feature and that the optimal beam phase varies with both activity and antenna pair. The strongest conclusion at this stage is not that one antenna pair is always best, but that the proposed beamforming pipeline consistently separates walking from stationary behavior and preserves meaningful room-dependent differences. These observations provide a solid basis for the later classification analysis, where the relative contribution of each antenna pair and feature will be evaluated quantitatively.



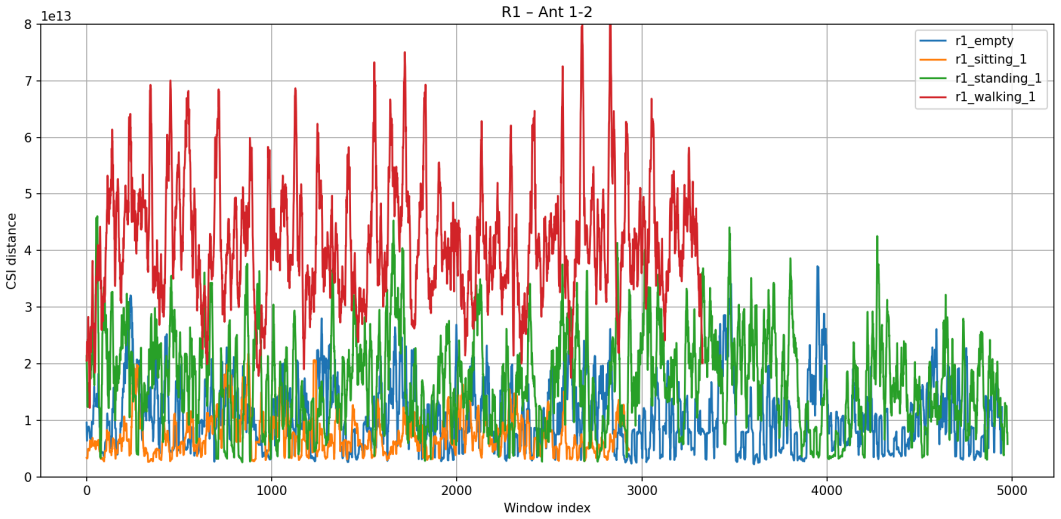
(a)


R1 – Ant 0-2
1e13
CSI distance
Window index
r1_empty
r1_sitting_1
r1_standing_1
r1_walking_1

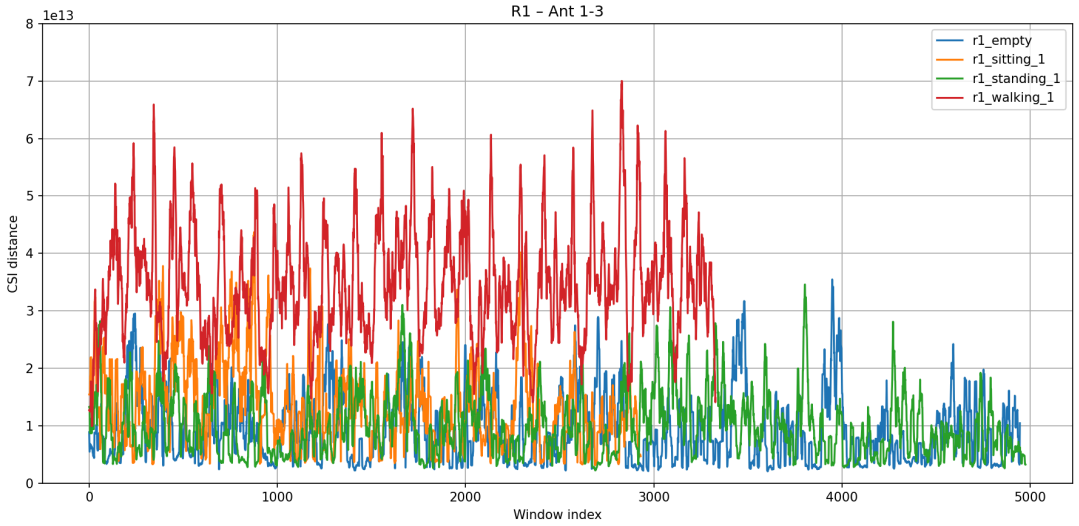
(b)


R1 – Ant 0-3
1e13
CSI distance
Window index
r1_empty
r1_sitting_1
r1_standing_1
r1_walking_1

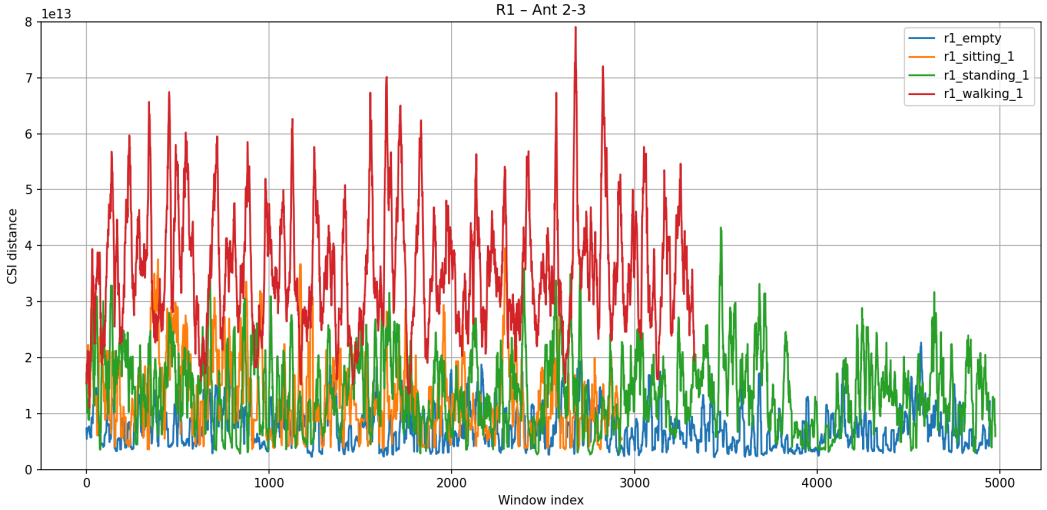
(c)


R1 – Ant 1-2
1e13
CSI distance
Window index
r1_empty
r1_sitting_1
r1_standing_1
r1_walking_1

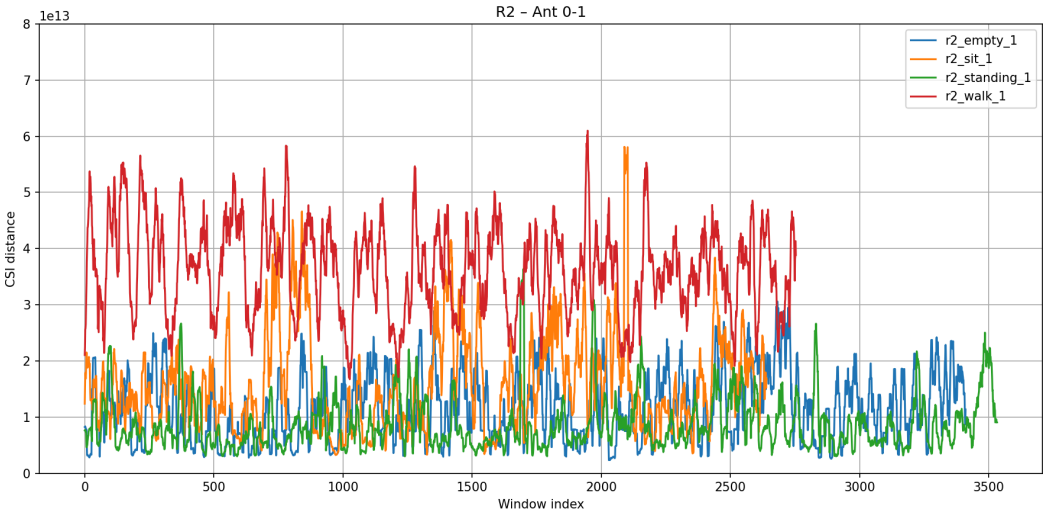
(d)


R1 – Ant 1-3
1e13
CSI distance
Window index
r1_empty
r1_sitting_1
r1_standing_1
r1_walking_1

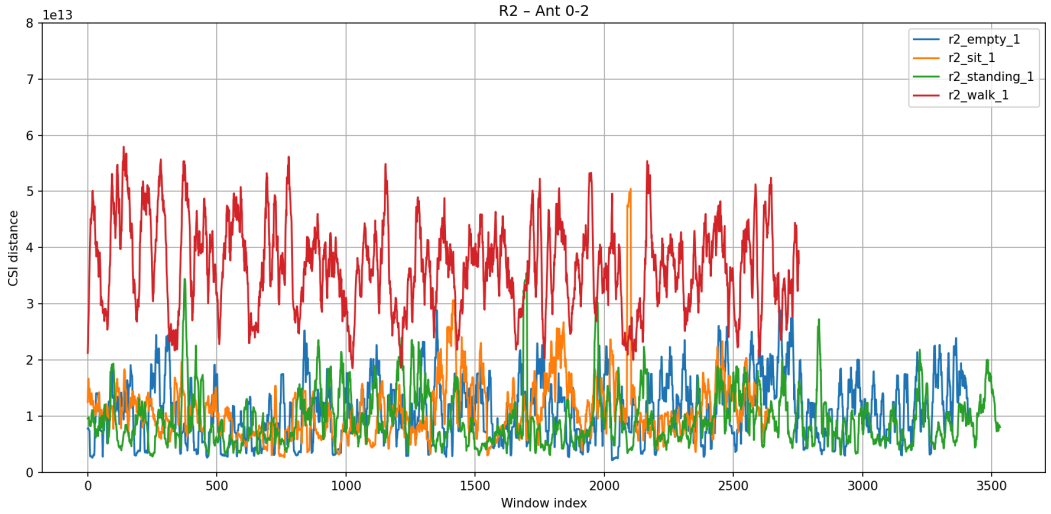
(e)


R1 – Ant 2-3
1e13
CSI distance
Window index
r1_empty
r1_sitting_1
r1_standing_1
r1_walking_1

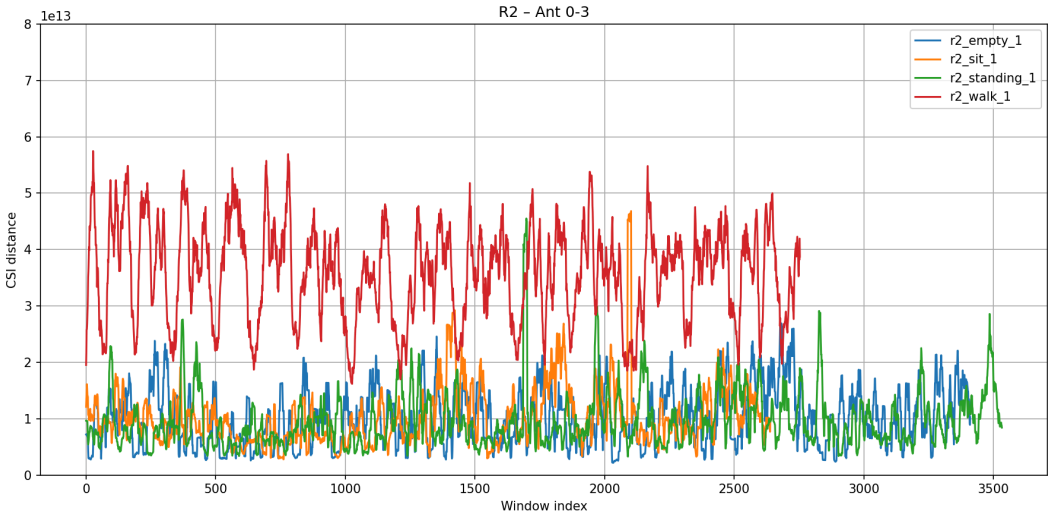
(f)


R2 – Ant 0-1
1e13
CSI distance
Window index
r2_empty_1
r2_sit_1
r2_standing_1
r2_walk_1

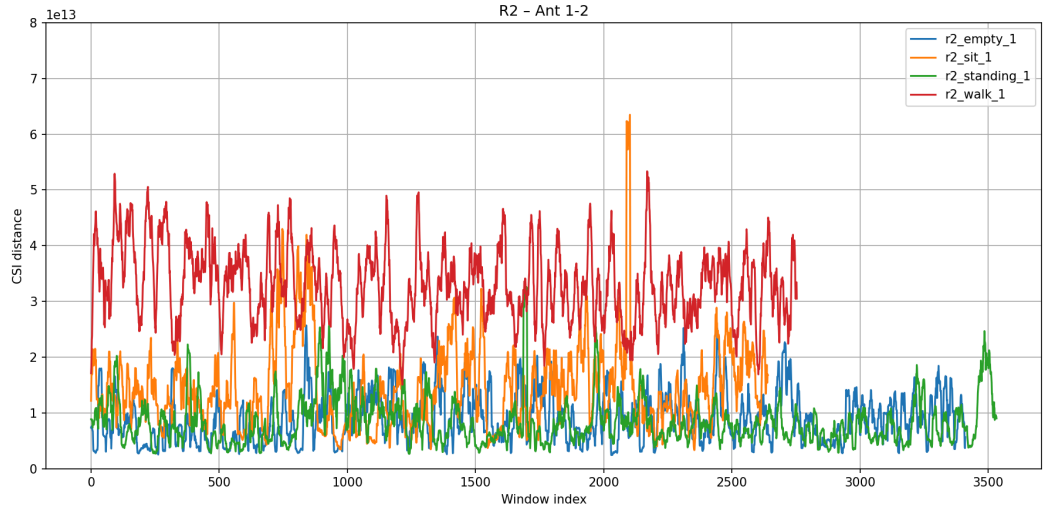
(g)


R2 – Ant 0-2
1e13
CSI distance
Window index
r2_empty_1
r2_sit_1
r2_standing_1
r2_walk_1

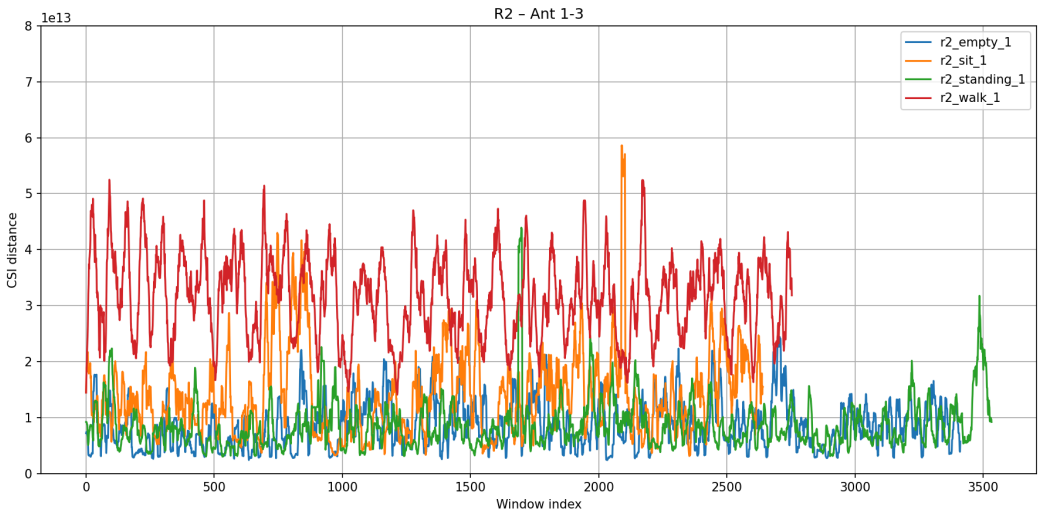
(h)


R2 – Ant 0-3
1e13
CSI distance
Window index
r2_empty_1
r2_sit_1
r2_standing_1
r2_walk_1

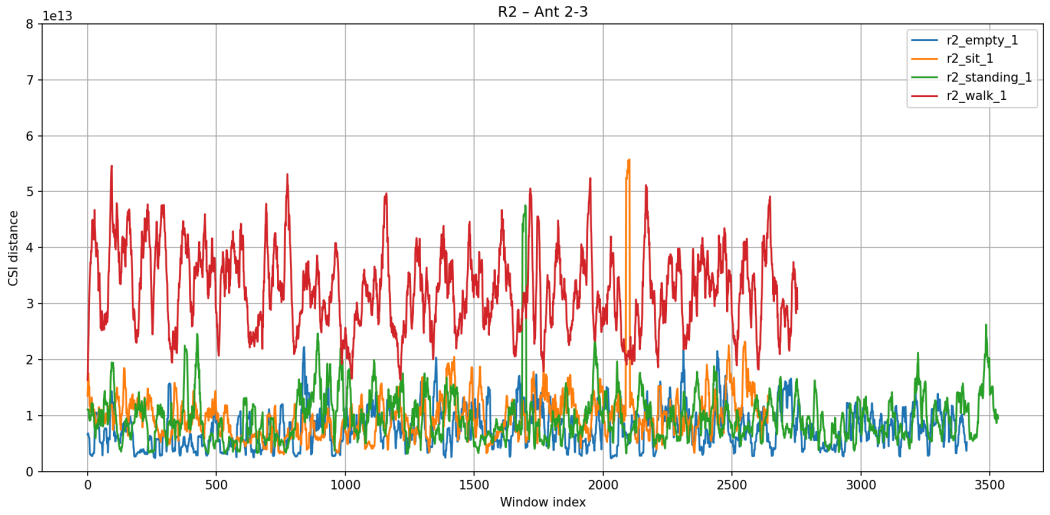
(i)


R2 – Ant 1-2
1e13
CSI distance
Window index
r2_empty_1
r2_sit_1
r2_standing_1
r2_walk_1

(j)

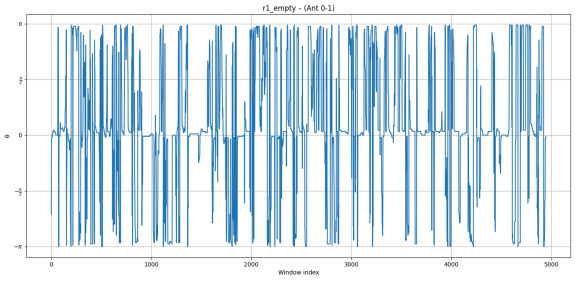


(k)

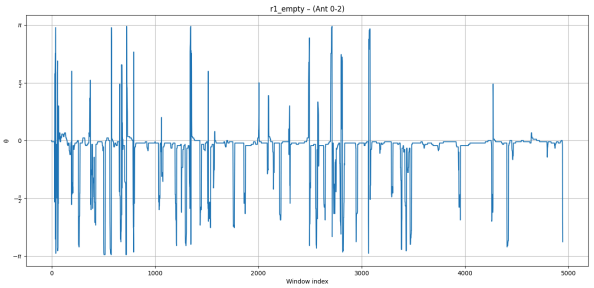


(l)

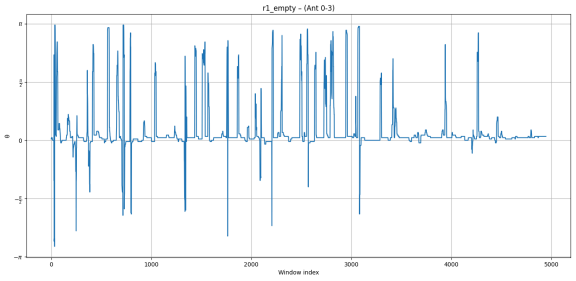
Figure 4.1. CSI distance comparison across activities and antenna pairs in Room 1 and Room 2. (a) Room 1, antenna pair 0–1. (b) Room 1, antenna pair 0–2. (c) Room 1, antenna pair 0–3. (d) Room 1, antenna pair 1–2. (e) Room 1, antenna pair 1–3. (f) Room 1, antenna pair 2–3. (g) Room 2, antenna pair 0–1. (h) Room 2, antenna pair 0–2. (i) Room 2, antenna pair 0–3. (j) Room 2, antenna pair 1–2. (k) Room 2, antenna pair 1–3. (l) Room 2, antenna pair 2–3.



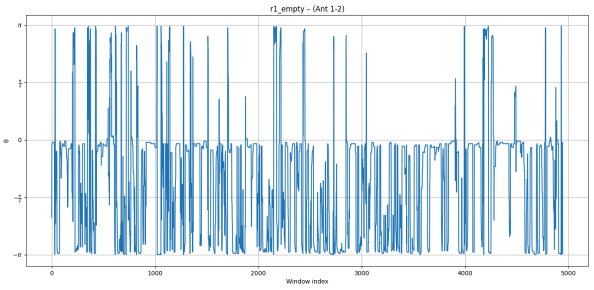
ant 0 - 1



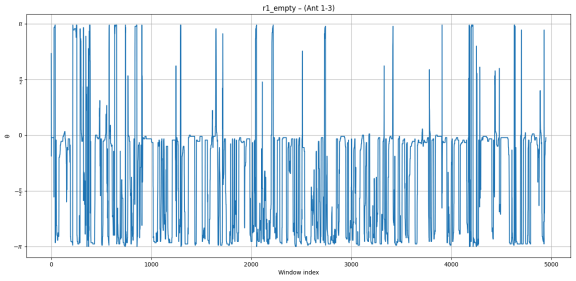
ant 0 - 2



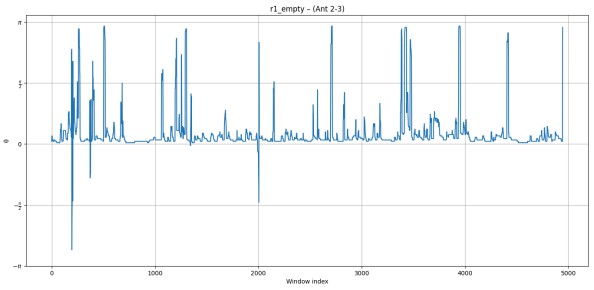
ant 0 - 3



ant 1 - 2

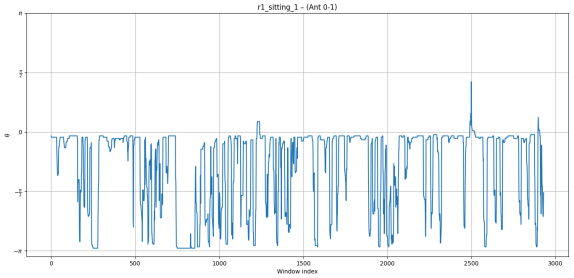


ant 1 - 3

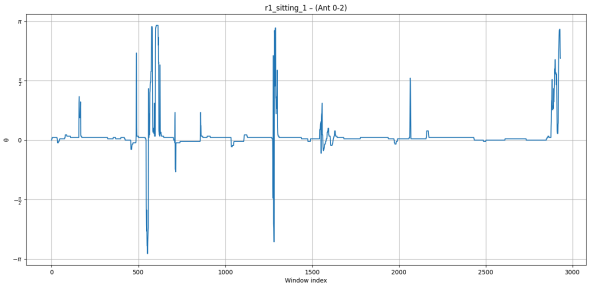


ant 2 - 3

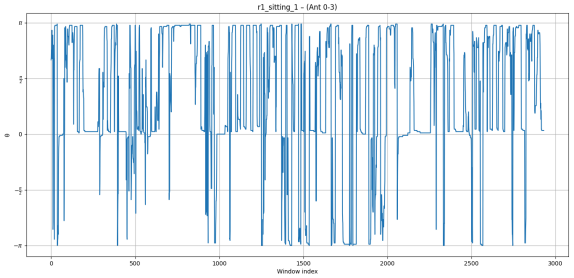
(a) Room 1 - Empty



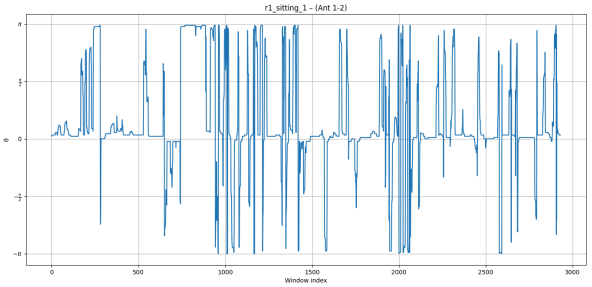
ant 0 - 1



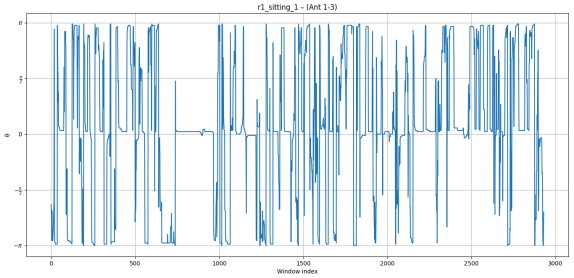
ant 0 - 2



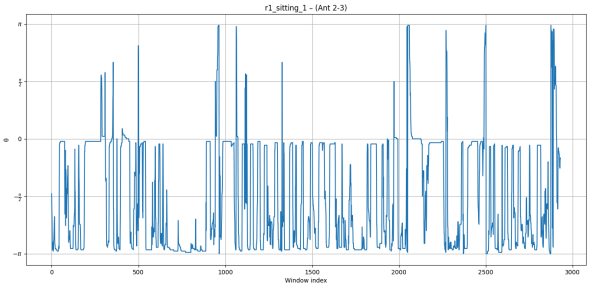
ant 0 - 3



ant 1 - 2

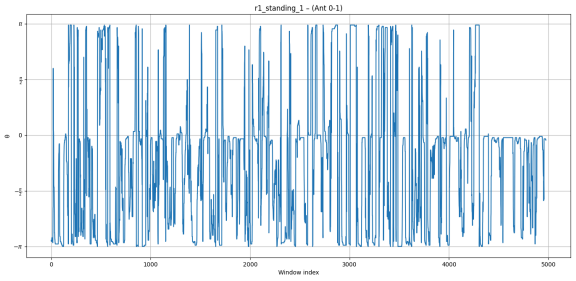


ant 1 - 3

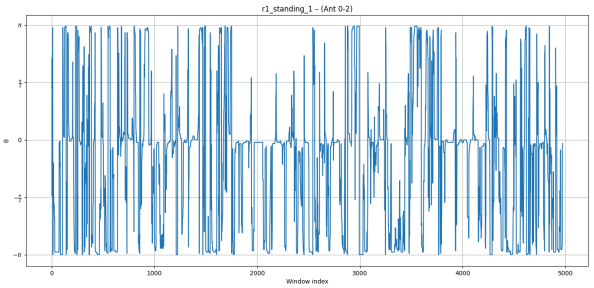


ant 2 - 3

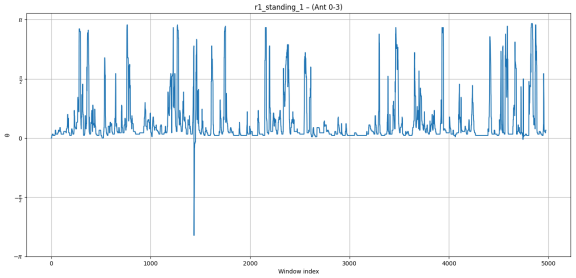
(b) Room 1 - Sitting



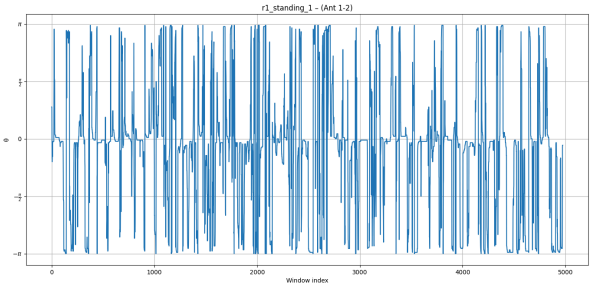
ant 0 - 1



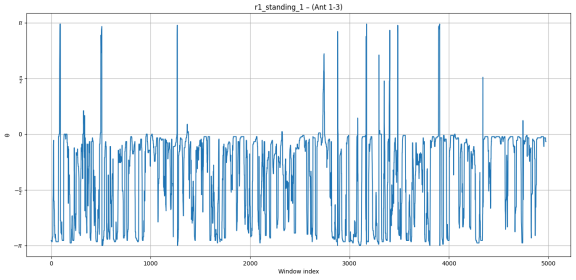
ant 0 - 2



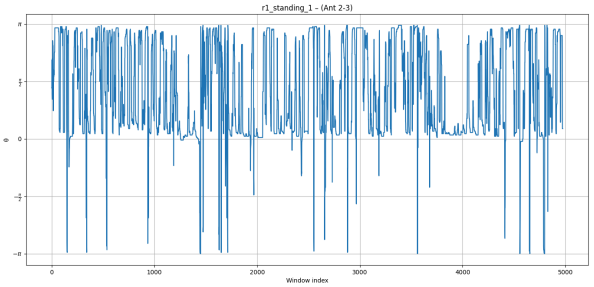
ant 0 - 3



ant 1 - 2

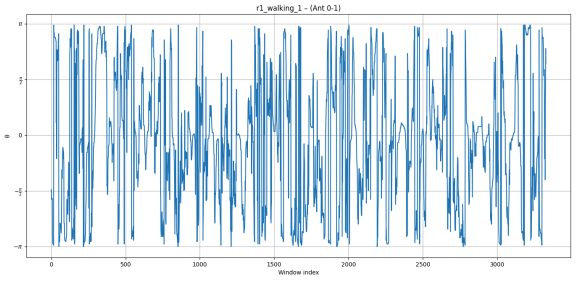


ant 1 - 3

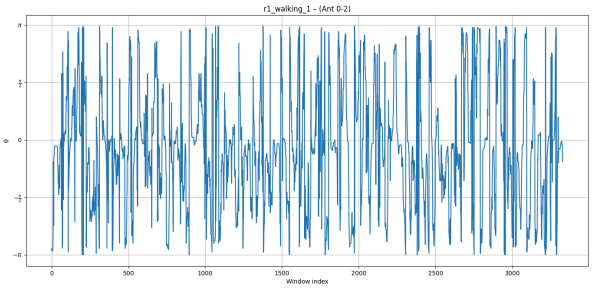


ant 2 - 3

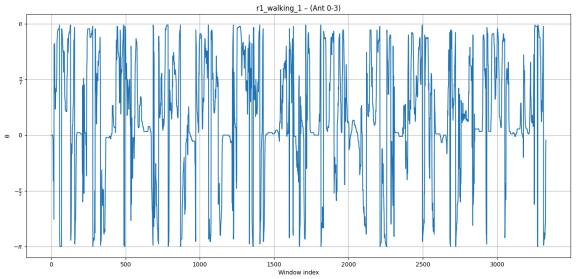
(c) Room 1 - Standing



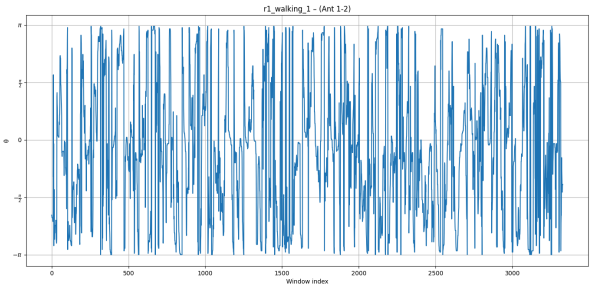
ant 0 - 1



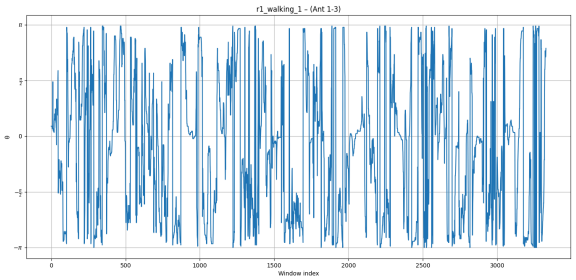
ant 0 - 2



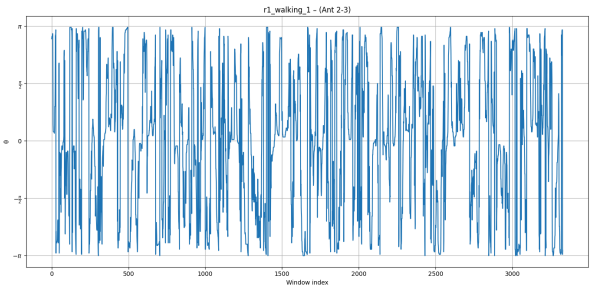
ant 0 - 3



ant 1 - 2



ant 1 - 3



ant 2 - 3

(d) Room 1 - Walking

Figure 4.2. Optimal beamforming phase θ across activities and antenna pairs in Room 1. (a) Empty. (b) Sitting. (c) Standing. (d) Walking.

Chapter 5

Temporal analysis of CSI distance in walking activity

5.1 Temporal Interpretation of CSI Distance During Walking Activity

To further investigate the temporal behavior of walking activity, the CSI distance signals obtained in the previous Euclidean distance analysis were processed through a dedicated periodicity analysis pipeline.

The CSI distance signal obtained in the previous section provides a compact representation of temporal channel variations caused by human motion. While static activities such as sitting or standing produce relatively small fluctuations, walking activity generates repeated changes in the wireless propagation environment due to continuous body movement and changing multipath reflections. As a result, the CSI distance signal exhibits a sequence of pronounced oscillations and local maxima over time.

The CSI distance itself does not directly represent the physical distance between the person and the receiver. Instead, it measures the temporal variation of the wireless channel over consecutive packets. Larger CSI distance values indicate stronger channel perturbations caused by motion, while smaller values correspond to more stable channel conditions. Therefore, the CSI distance can be interpreted as a measure of motion intensity inside the environment. When a person walks, changes direction, approaches the transmitter or receiver, or modifies the multipath propagation paths, the CSI distance increases because the wireless channel changes more rapidly between consecutive observations.

The walking signals shown in Figure 4.1 exhibit repeated peaks over time. These peaks suggest the presence of quasi periodic motion patterns generated by the person repeatedly moving back and forth inside the room. Since the movement in the collected dataset was not perfectly uniform, the signal cannot be considered strictly periodic. However, the repeated

occurrence of local maxima indicates the existence of dominant motion cycles that can be analyzed quantitatively.

To study this behavior, a peak-based periodicity analysis was performed on the CSI distance signals corresponding to walking activity. The analysis was conducted separately for each dataset in order to preserve the temporal characteristics of each recording. In this section, the results for Room 1 and Room 2 using antenna pair 0 to 1 are presented.

5.2 Peak Detection and Motion Period Estimation

The CSI distance sequence is denoted as

$$d[n],$$

where n represents the window index. Since the CSI distance was already computed using a sliding window of size W = 16, the resulting signal is smoother than the original CSI measurements. Nevertheless, small local fluctuations caused by residual noise may still appear. To reduce these short term variations while preserving the dominant motion structure, a moving average smoothing operation was applied:

$$\bar{d}[n] = \frac{1}{M} \sum_{k=0}^{M-1} d[n - k],$$

where M is the smoothing window length. In this work, a smoothing window of 5 samples was used. This value was selected empirically because it reduces high frequency noise while preserving the main walking peaks visible in the signal.

After smoothing, local maxima were extracted from the CSI distance signal. A local maximum corresponds to a point where the CSI distance is larger than its neighboring samples and represents a dominant motion event in the channel dynamics. Since the walking activity produces repeated large fluctuations, these local maxima correspond to periods of strong channel disturbance caused by body movement.

However, not every local fluctuation should be considered a meaningful peak. Small oscillations may appear due to residual channel noise or minor environmental variations. To

avoid detecting insignificant fluctuations, three constraints were imposed during peak detection.

First, a minimum peak height threshold of

$$d[n] > 2 \times 10^{13}$$

was used. This threshold was selected by comparing walking signals with static activity signals. As shown in Figure X, empty, sitting, and standing activities mainly produce CSI distance values below this range, while walking activity consistently exceeds it. Therefore, this threshold helps separate motion induced peaks from background fluctuations.

Second, a minimum prominence value of

$$P_{min} > 0.6 \times 10^{13}$$

was introduced. Peak prominence measures how much a peak stands out relative to its surrounding valleys. This condition ensures that only dominant peaks associated with significant motion variations are retained.

Third, a minimum peak separation distance of 15 samples was applied. This prevents multiple neighboring samples belonging to the same oscillation from being detected as separate peaks. Since walking generates broad motion cycles rather than isolated impulses, this condition allows each dominant motion event to be represented by a single peak.

Once the peaks were identified, the temporal spacing between consecutive peaks was analyzed. Let n_k represent the index of the k-th detected peak. The interval between consecutive peaks:

$$\Delta n_k = n_{k+1} - n_{k'}$$

where Δn_k represents the number of samples between two successive dominant motion events.

The dataset acquisition rate is approximately 5 samples per second. Therefore, the peak intervals can be converted into temporal motion periods:

$$T_k = \frac{\Delta n_k}{f_s},$$

where $f_s = 5$ Hz is the effective sampling frequency and T_k is the motion period in seconds.

The resulting temporal periods provide information about how frequently dominant motion cycles occur during walking. Smaller values of T_k indicate more rapid repetitions of movement, while larger values correspond to slower or less regular motion cycles. Since the subject in the dataset was walking back and forth inside the room, these periods represent repeated large scale motion disturbances in the wireless channel rather than individual walking steps.

To evaluate the stability of the motion pattern, the regularity coefficient was computed using the coefficient of variation:

$$CV = \frac{\sigma_{\Delta n}}{\mu_{\Delta n}},$$

where $\mu_{\Delta n}$ and $\sigma_{\Delta n}$ denote the mean and standard deviation of the peak intervals, respectively. Lower values of CV indicate more stable and regular periodic behavior, while larger values correspond to more irregular motion dynamics.

To summarize, the periodicity analysis of walking activity was performed through the following steps:

- Select the CSI distance signals corresponding to walking activity for antenna pair 0 to 1.
- Apply a moving average smoothing filter with window size $M = 5$ to reduce short term fluctuations while preserving dominant motion structures.
- Detect local maxima from the smoothed CSI distance signal.
- Apply a minimum peak height threshold of 2×10^{13} to suppress low amplitude fluctuations associated with static activities and background noise.

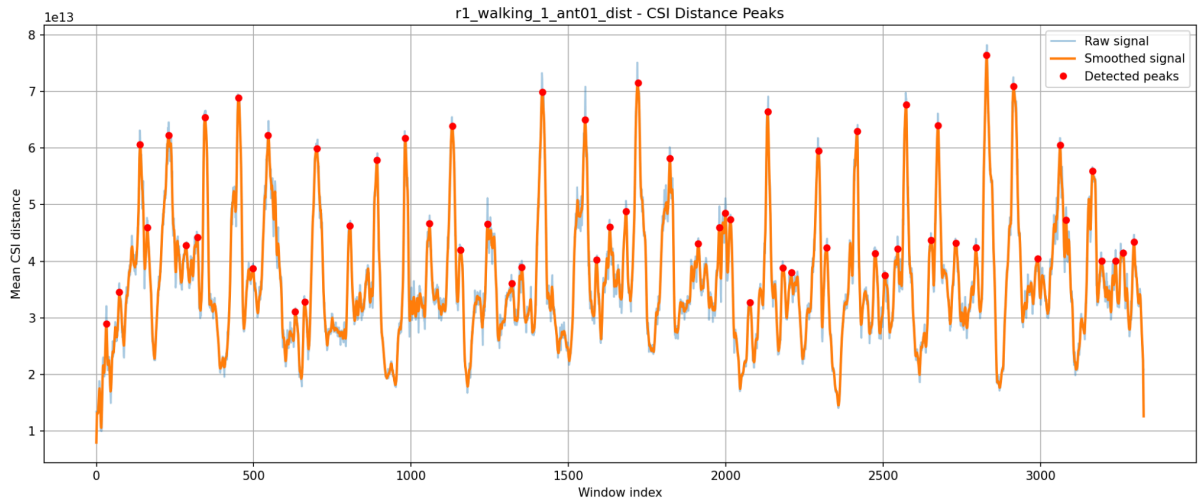
- Apply a minimum prominence threshold of 0.6×10^{13} to retain only dominant motion related peaks.
- Apply a minimum peak separation distance of 15 samples to avoid detecting multiple neighboring samples belonging to the same motion cycle.
- Compute the interval between consecutive detected peaks: Δn_k.
- Convert the peak intervals into temporal motion periods using the effective sampling frequency of approximately 5 samples per second: T_k.
- Compute statistical metrics including the mean motion period, standard deviation, and regularity coefficient to characterize the temporal dynamics of walking activity.
- Analyze the CSI Distance Peaks plots, Peak to Peak Motion Period plots, and interval histograms to study the quasi periodic structure of human motion in both rooms.

The temporal structure of the walking traces was further investigated through an automated peak-based analysis implemented in the script analyze_walking_period.py (see Appendix A.2).

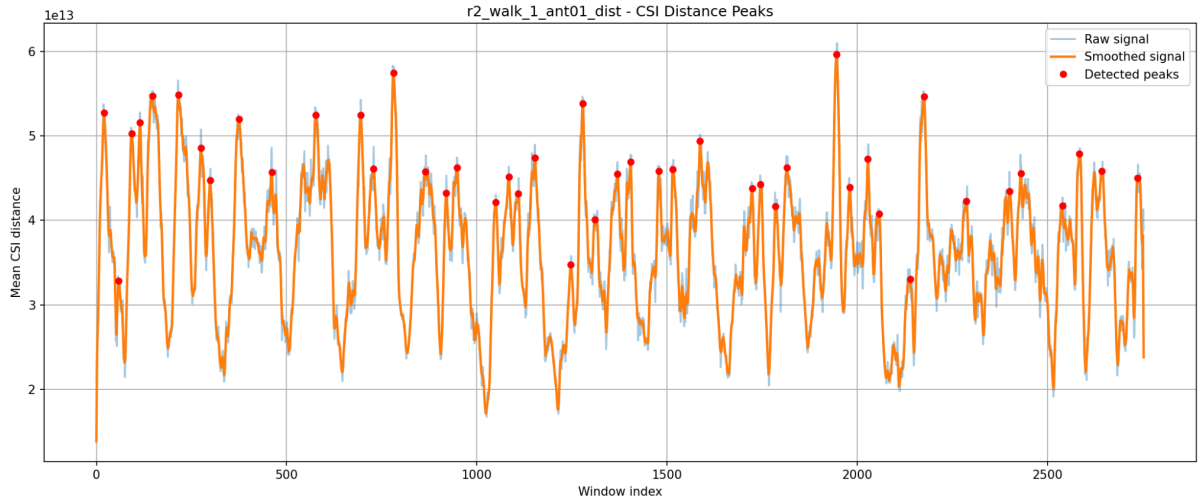
5.3 Analysis of Walking Periodicity

Figure 5.1 (a) presents the detected CSI Distance Peaks for Room 1. The figure shows that the walking activity generates repeated dominant peaks throughout the recording. Although the spacing between peaks is not perfectly uniform, repeated oscillatory structures are clearly visible, confirming that the wireless channel is repeatedly disturbed by the motion of the subject inside the room.

The corresponding Peak to Peak Motion Period plot in Figure 5.2 (a) further illustrates how the temporal spacing between dominant motion events evolves over time. Most motion periods remain approximately between 4 and 18 seconds, with many intervals concentrated around 5 to 8 seconds. This indicates that the walking activity produces repeated channel disturbances at relatively consistent temporal scales. The presence of these repeated periods suggests that the subject followed a recurrent movement trajectory inside the room rather than performing random isolated movements.



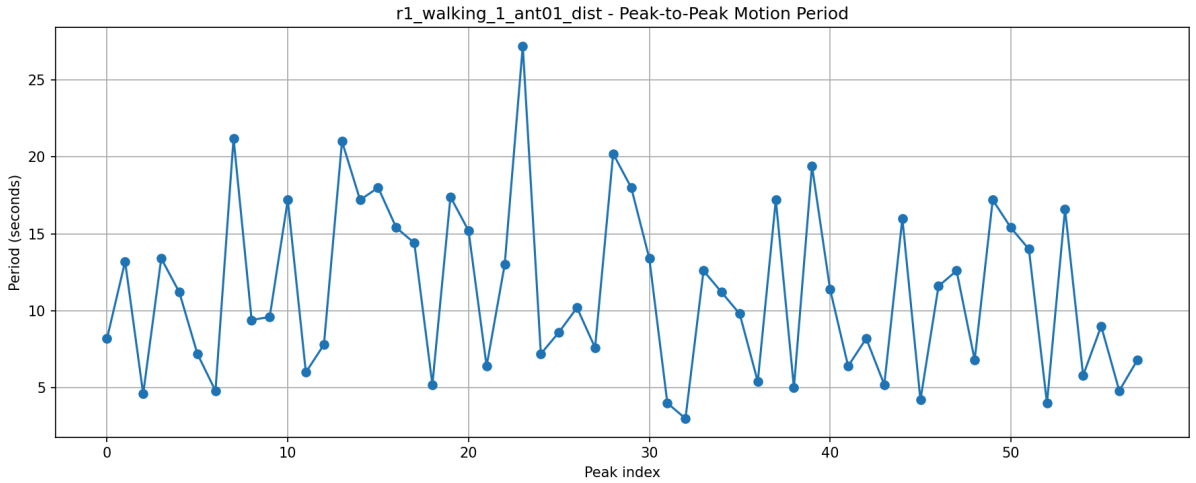
(a)



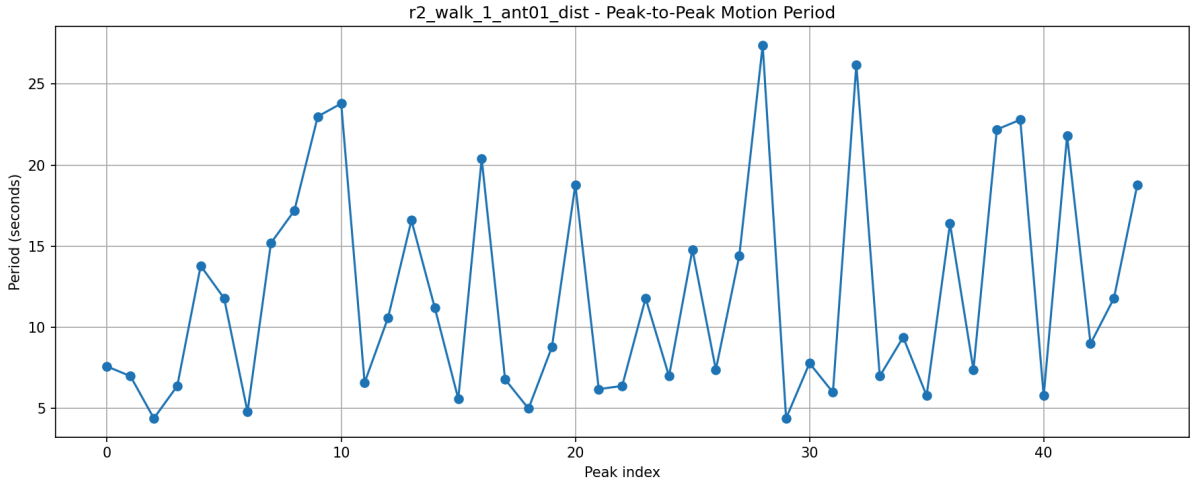
(b)

Figure 5.1. CSI Distance Peaks detected during walking activity for Antenna Pair 0 to 1. (a) Room 1. (b) Room 2.

At the same time, several larger intervals can also be observed. In particular, around peak indices 21 to 23, the motion period increases significantly and reaches approximately 27 seconds. This large interval indicates that a dominant motion event occurred after a relatively long pause compared to the surrounding intervals. Such behavior may correspond to slower movement, a temporary stop, a change in direction, or a longer walking path before the next strong channel perturbation was produced. Since the CSI distance captures large scale channel changes rather than individual footsteps, these large intervals likely correspond to broader movement transitions inside the environment.



(a)



(b)

Figure 5.2. Peak to Peak Motion Period obtained from detected CSI distance peaks during walking activity for Antenna Pair 0 to 1. (a) Room 1. (b) Room 2.

Conversely, smaller motion periods are visible around peak indices 31 and 32, where the periods decrease to approximately 4 and 3 seconds. These short intervals indicate that dominant channel disturbances occurred in rapid succession. This may happen when the subject moves more quickly through a particular region of the room or when the walking trajectory causes rapid changes in the dominant multipath components. Therefore, the coexistence of both short and long periods demonstrates that the walking behavior is quasi periodic rather than strictly periodic.

The CSI Distance Peaks plot for Room 1 in Figure 5.1 (a) also reveals that many detected peaks reach values between approximately 5×10^{13} and 7×10^{13}. These strong amplitudes indicate that the walking activity produces substantial temporal channel variations. Compared

to the static activity signals shown previously, the difference is significant, confirming that walking generates a much stronger perturbation of the wireless propagation environment.

A similar behavior can be observed in Room 2 in Figure 5.1 (b). The CSI distance signal again exhibits repeated dominant peaks corresponding to continuous body movement. However, the overall peak amplitudes are slightly lower compared to Room 1, with many peaks concentrated around 4×10^{13} to 5.5×10^{13}. This suggests that the propagation characteristics and multipath structure of Room 2 produce slightly weaker CSI distance responses to motion. Nevertheless, the repeated oscillatory pattern remains clearly visible, confirming that walking activity still produces dominant temporal structures in the CSI distance signal.

The Peak to Peak Motion Period plot for Room 2 in Figure 5.2 (b) shows a broader spread of motion periods. While many intervals again remain between approximately 5 and 17 seconds, several larger intervals appear throughout the recording. Around peak index 28, the motion period reaches approximately 27 seconds, while another large interval near peak index 32 reaches approximately 26 seconds. Similar to Room 1, these large periods likely correspond to slower movement transitions, pauses, or changes in walking direction before the next dominant motion event occurs.

Shorter periods are also visible in Room 2, particularly around peak indices 29 and 30 where the period decreases to approximately 4 to 6 seconds. These short intervals indicate rapid successive motion disturbances. Compared to Room 1, the alternation between short and long periods appears slightly more pronounced, suggesting that the motion pattern in Room 2 is somewhat less stable over time.

Table 1 summarizes the quantitative results obtained from the periodicity analysis.

The results show that both rooms exhibit similar mean motion periods, approximately 11 to 12 seconds. This indicates that the dominant repetition behavior is primarily determined by the walking activity itself rather than by the room environment alone. In both cases, the walking activity produces repeated large scale channel disturbances with comparable temporal structure.

Dataset	**Number of Detected Peaks**	**Mean peak interval** $\mu_{\Delta n}$	**Standard deviation** $\sigma_{\Delta n}$	**Mean motion period** μ_T **(s)**	**Motion Period Standard Deviation (s)**	**Regularity coefficient** CV
Room 1	59	56.29	27.41	11.26	5.48	0.49
Room 2	46	60.38	33.16	12.08	6.63	0.55

Table 1. Statistical Summary of CSI Distance Periodicity Analysis for Walking Activity Using Antenna Pair 0 to 1

However, Room 2 exhibits a larger standard deviation and a higher regularity coefficient compared to Room 1. This indicates that the temporal spacing between dominant motion events is more variable and less stable in Room 2. The increased variability may be caused by differences in room geometry, multipath propagation conditions, walking trajectory, or the relative position between the subject and the transmitter receiver pair.

Overall, the periodicity analysis demonstrates that the CSI distance signal contains meaningful temporal information about human walking activity. The CSI Distance Peaks plots reveal repeated dominant channel disturbances generated by movement, while the Peak to Peak Motion Period plots characterize how regularly these disturbances occur over time. Therefore, the CSI distance not only captures the presence and intensity of motion, but also provides insight into the temporal dynamics and regularity of human movement inside indoor wireless environments.

Chapter 6

Feature Extraction and Activity Classification

This chapter describes how the processed beamforming outputs are transformed into compact numerical descriptors suitable for machine learning classification. Starting from the CSI distance and optimal beam phase sequences generated in Chapter 4, a second sliding-window stage is applied to extract statistical and temporal features. These features are then used to train a Random Forest classifier under several experimental setups designed to evaluate both in-room performance and cross-room generalization.

6.1 Feature Extraction and Dataset Construction

The output of the beamforming stage consists of two time series for each recording and antenna pair: the CSI distance sequence $d(n)$ and the corresponding optimal beamforming phase sequence $\theta(n)$. Although these sequences already summarize motion-sensitive channel behavior, they are still too long and too noisy to be used directly as classification inputs. For this reason, a second sliding-window aggregation stage is applied on top of the beamformed outputs.

Let W_2 denote the second-level window length. In this work, multiple values were evaluated, namely $W_2 \in \{10, 20, 30, 40, 50\}$, in order to study how the amount of temporal aggregation affects classification performance. A smaller W_2 preserves more fine-grained temporal detail, while a larger W_2 provides smoother features but may suppress short motion variations. The second window slides through the distance and theta sequences with unit stride, so that each new sample is formed from the next overlapping segment of the signal.

For a window starting at index j, the CSI distance segment is

$$D_j = \{d(j),\ d(j+1),\ \ldots,\ d(j+W_2-1)\}$$

and the corresponding optimal beam phase segment is

$$\Theta_j = \{\theta(j),\ \theta(j+1),\ \ldots,\ \theta(j+W_2-1)\}.$$

From each window, a compact feature vector is extracted. The first feature is the mean CSI distance:

$$\mu_d^{(j)} = \frac{1}{W_2} \sum_{n=j}^{j+W_2-1} d(n),$$

which captures the average motion intensity inside the window.

The second feature is the standard deviation of the distance:

$$\sigma_d^{(j)} = \sqrt{\frac{1}{W_2} \sum_{n=j}^{j+W_2-1} (d(n) - \mu_d^{(j)})^2},$$

which measures how strongly the motion response fluctuates around its local mean.

The third feature is the maximum CSI distance value in the window:

$$d_{max}^{(j)} = max(d(n)), \quad n \in D_j,$$

which emphasizes dominant motion peaks.

The fourth feature is the variance of the beamforming phase:

$$\sigma_\theta^{2\,(j)} = Var(\Theta_j),$$

where the optimal beam sequence is first unwrapped before variance computation.

The resulting feature vector for window j is therefore:

$$\bar{x}^j = [\mu_d^{(j)}, \sigma_d^{(j)}, d_{max}^{(j)}, \sigma_\theta^{2\,(j)}]^T.$$

The corresponding activity label is:

$$y^{(j)} \in \{0,\ 1,\ 2,\ 3\},$$

where the mapping used in the implementation is:

$$empty\ = 0,\ sitting\ = 1,\ standing\ = 2,\ and\ walking\ = 3.$$

The final feature dataset therefore contains the following columns:

f_1 – Mean CSI distance

f_2 – Standard deviation of CSI distance

f_3 – Maximum CSI distance in the window

f_4 – Variance of optimal beam phase θ

f_5 – Activity label

Each dataset row is saved as a NumPy array containing these four features followed by the class label.

The feature extraction is repeated independently for each antenna pair, each room, and each second-window size W_2. Three dataset partitions are generated for every antenna pair and W_2 value. The first partition contains only Room 1 recordings, the second contains only Room 2 recordings, and the third combines both rooms into a single dataset. This design makes it possible to evaluate both room-specific learning and cross-room transfer.

The room-specific datasets are useful because they preserve the spatial characteristics of each environment. The combined dataset, on the other hand, allows the classifier to learn from both rooms simultaneously and provides a broader training distribution. In the code, these datasets are stored separately under folders named r1, r2, and r1_r2. The antenna pair is also kept separate so that the effect of spatial configuration can be evaluated independently.

The implementation of the feature construction pipeline is provided in Appendix A.3 (feature_extraction.py).

6.2 Random Forest Classifier

Random Forest is an ensemble learning method that combines multiple decision trees trained on bootstrap samples of the training data. Each tree is built using a random subset of features at each split, which reduces correlation between trees and improves generalization. The final prediction is obtained by majority voting across the individual trees.

This classifier is well suited to the present problem for several reasons. First, the extracted features are tabular and low-dimensional, which matches the strengths of tree-based models. Second, the relationship between CSI distance features, beam phase variance, and human activity is not strictly linear. Random Forest can model nonlinear interactions without requiring strong assumptions about the feature distribution. Third, the model is relatively robust to noise and outliers, which is useful because CSI-based measurements inevitably contain residual fluctuations even after preprocessing.

The classifier is trained using the extracted feature vectors $\bar{x}^{j}$ and their corresponding labels $y^{(j)}$. Before training, the feature values are standardized using statistics computed on the training set only:

$$x' = \frac{x - \mu_{train}}{\sigma_{train}}.$$

This scaling step is applied consistently across all experiments. While tree-based models do not strictly require feature normalization, the standardization keeps the experimental pipeline uniform and avoids numerical imbalance across different feature ranges.

The Random Forest implementation also uses class weighting to account for possible class imbalance in the generated windows. This helps the model avoid bias toward the most frequent class when the number of samples differs across activities or recordings.

6.3 Experimental Setups

The classification experiments were designed to assess performance under different data availability and generalization conditions. Each setup is repeated for every antenna pair and every second-window length W_2. This allows the influence of temporal aggregation and spatial antenna selection to be studied systematically.

The first setup is the mixed training and testing scenario. In this case, the combined dataset from both rooms is randomly split into training and testing subsets. This setup gives an upper-bound estimate of classification performance because the model can learn from both room geometries before testing. It is useful for measuring how separable the activities are when the classifier has access to representative data from the full environment distribution.

The second and third setups are cross-room transfer experiments. In the Room 1 to Room 2 setting, the classifier is trained only on Room 1 data and tested on Room 2 data. The reverse setup trains on Room 2 and tests on Room 1. These experiments are important because they evaluate robustness to environmental change. A model that performs well in cross-room transfer is less dependent on the specific geometry and multipath structure of a single room, which is a desirable property for practical deployment.

The fourth and fifth setups are same-room baselines. In Room 1 to Room 1 and Room 2 to Room 2, the classifier is trained and tested within the same room using a random split. These experiments measure how well the learned features distinguish the four activities when the training and testing distributions are closely matched. They serve as a reference point for comparing against the more challenging cross-room scenarios.

All setups are evaluated separately for each antenna pair, since the selected pair determines the beamforming response and therefore affects the quality of the extracted features. In practice, different antenna combinations may emphasize different propagation paths and different sensitivity regions of the room. Evaluating all pairs allows the thesis to compare how spatial diversity influences classification performance.

The complete experimental workflow is therefore organized as follows:

- For each W_2, features are extracted from the CSI distance and theta sequences.
- The resulting datasets are separated by room and antenna pair.

- A Random Forest classifier is then trained under the five setups described above.
- Finally, the prediction accuracy and confusion matrices are computed for each configuration.

This experimental design makes it possible to compare not only the effect of room transfer, but also the influence of temporal window size and antenna pair selection on activity recognition performance.

The implementation of the classifier and evaluation pipeline is provided in Appendix A.4 (random_forest.py).

6.4 Classification Results and Discussion

The classification results confirm that the proposed feature set captures meaningful activity-dependent structure in the beamformed CSI domain. Across all experiments, the strongest performances are obtained in the same-room settings, followed by the mixed setup, while the cross-room experiments are considerably more challenging. This behavior is expected because the feature distributions learned in one room transfer only partially to another room with different geometry, reflection structure, and antenna-channel relationships.

A clear improvement is observed as the second window size W_2 increases. For the mixed setup, the average accuracy rises from about 68.9% at $W_2 = 10$ to 93.8% at $W_2 = 50$. Room 1 to Room 1 improves from about 73.0% to 94.5%, and Room 2 to Room 2 increases from about 73.9% to 96.3%. This shows that larger temporal aggregation produces more stable features and helps the Random Forest classifier better capture the activity-related dynamics. In contrast, the cross-room settings do not benefit from larger windows: Room 1 to Room 2 stays around 46–47%, and Room 2 to Room 1 remains around 41–44%. This indicates that longer temporal smoothing improves within-room consistency but does not solve the domain shift between rooms.

6.4.1 Accuracy Comparison Across Experimental Setups

Table 2 summarizes the average classification accuracy obtained for each experimental setup using different second-window sizes.

Setup	$W_2 = 10$	$W_2 = 20$	$W_2 = 30$	$W_2 = 40$	$W_2 = 50$
Mixed	68.9%	79.4%	86.7%	91.2%	93.8%
r1→r1	73.0%	83.6%	89.5%	92.7%	94.5%
r2→r2	73.9%	85.8%	91.7%	94.8%	96.3%
r1→r2	46.1%	46.4%	46.8%	47.0%	47.3%
r2→r1	41.2%	42.0%	42.8%	43.5%	44.1%

Table 2. Average classification accuracy for all experimental setups and second-window sizes.

The results show that same-room training and testing achieve the highest performance, while cross-room transfer remains substantially more difficult. The increasing performance with larger W_2 values indicate that temporal aggregation improves feature stability and suppresses short-term CSI fluctuations.

6.4.2 Top Performing Configurations

Table 3 presents the highest-performing configurations obtained from the Random Forest experiments.

The results show that the highest accuracies are consistently achieved in the same-room experiments, particularly in the r2→r2 setup. Larger second-window sizes, especially $W_2 = 50$, dominate the top configurations, confirming that longer temporal aggregation improves feature stability and class separability. Antenna pairs 0 – 1 and 0 – 2 repeatedly appear among the best-performing cases, indicating that these antenna combinations capture highly discriminative CSI dynamics.

Rank	Setup	Antenna Pair	W_2	Accuracy	Mean Error
1	r2→r2	0 – 1	50	96.9%	3.1%
2	r2→r2	1 – 3	50	96.6%	3.4%
3	r2→r2	0 – 2	50	96.5%	3.5%
7	r1→r1	0 – 2	50	95.4%	4.6%
8	r2→r2	0 – 2	40	95.1%	4.9%
9	r1→r1	0 – 3	50	94.8%	5.2%
10	r2→r2	0 – 1	40	94.8%	5.2%
14	Mixed	0 – 1	50	94.4%	5.6%
15	Mixed	0 – 2	50	94.4%	5.6%

Table 3. Top-performing Random Forest configurations.

6.4.3 Average Performance by Antenna Pair

Table 4 shows the average accuracy achieved by each antenna pair across all experiments.

The differences between antenna pairs are relatively moderate, which indicates that the proposed pipeline is robust across different spatial antenna configurations. Nevertheless, antenna pairs 0 – 1 and 0 – 2 consistently outperform the others, especially in same-room experiments.

Antenna Pair	Average Accuracy
0 – 1	71.9%
0 – 2	71.6%
1 – 3	71.1%
0 – 3	70.8%
2 – 3	69.7%
1 – 2	69.2%

Table 4. Average classification accuracy by antenna pair.

6.4.4 Confusion Matrix Analysis

The confusion matrices provide additional insight into the class-level behavior of the classifier. Figures 6.1–6.7 present representative examples of both high-performing and low-performing experimental setups.

The best-performing confusion matrix was obtained in the r2→r2 setup using antenna pair 0 – 1 with $W_2 = 50$ (Figure 6.1). The classifier achieved class accuracies of 96.9% for empty, 94.1% for sitting, 97.0% for standing, and 99.7% for walking. Most samples are concentrated along the diagonal of the matrix, indicating highly reliable classification performance within the same room environment.

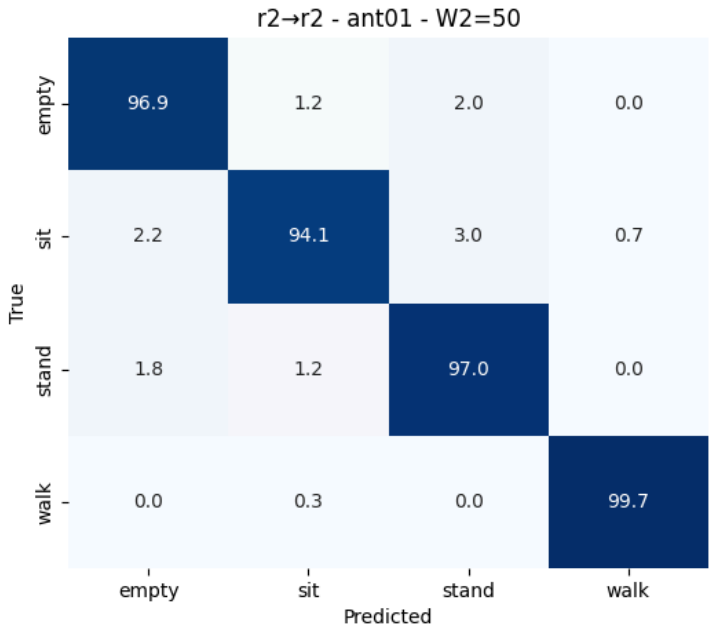


Figure 6.1. Confusion matrix: r2→r2, antenna pair 0 – 1, $W_2 = 50$.

A similarly high-performance result was obtained using antenna pair 1 – 3 in the same experimental setup (Figure 6.2). The classifier reached 96.8% accuracy for empty, 92.3% for sitting, 98.1% for standing, and 99.2% for walking. Minor confusion is observed mainly between the sitting and standing classes.

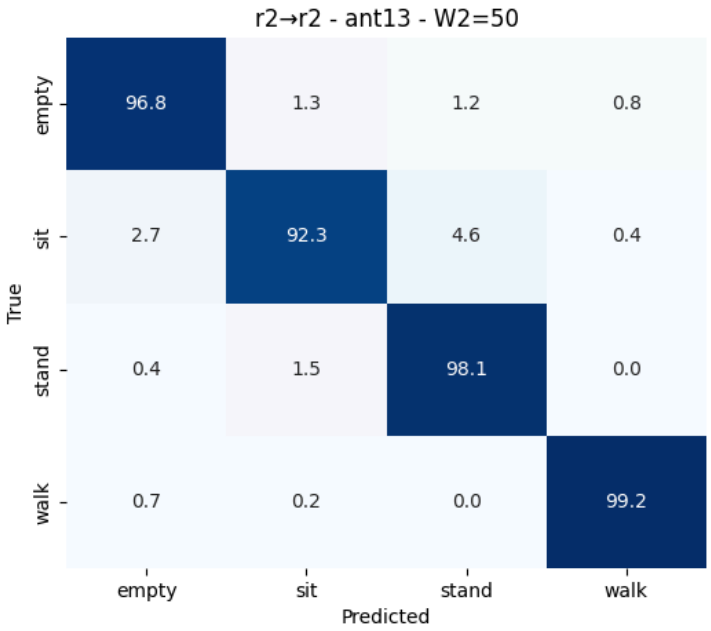


Figure 6.2. Confusion matrix: r2→r2, antenna pair 1 – 3, $W_2 = 50$.

Room 1 experiments also demonstrated high classification accuracy. Using antenna pair 0 – 2 with $W_2 = 50$, the classifier achieved 93.2% for empty, 91.3% for sitting, 96.8% for standing, and 97.7% for walking (Figure 6.3).

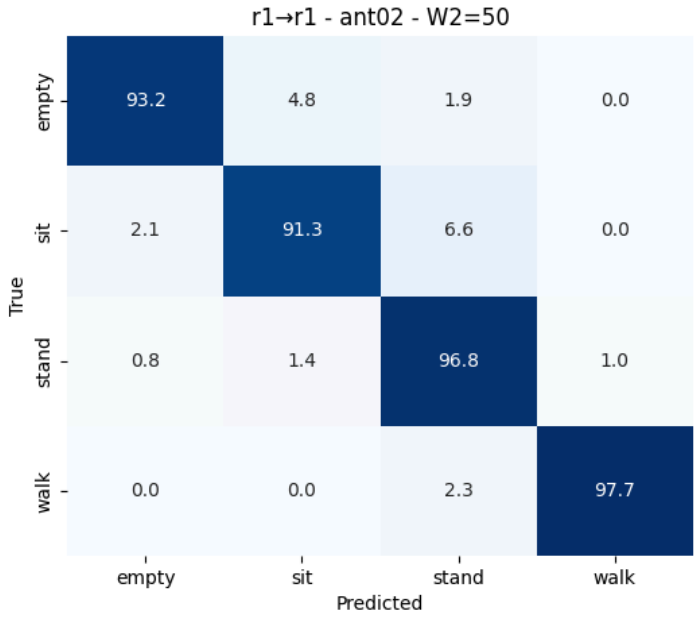


Figure 6.3. Confusion matrix: r1→r1, antenna pair 0 – 2, $W_2 = 50$.

The mixed setup also achieved high classification performance despite combining samples from two different rooms (Figure 6.4). Using antenna pair 0 – 1 with $W_2 = 50$, the classifier obtained 89.8% accuracy for empty, 91.6% for sitting, 94.9% for standing, and 99.2% for walking.

Compared to the same-room experiments, a moderate increase in confusion between empty, sitting, and standing activities can be observed. This behavior is expected because the classifier must generalize across different room geometries, multipath propagation conditions, and environmental characteristics. Nevertheless, the model still maintains high overall performance, indicating that the selected features preserve substantial robustness across environments.

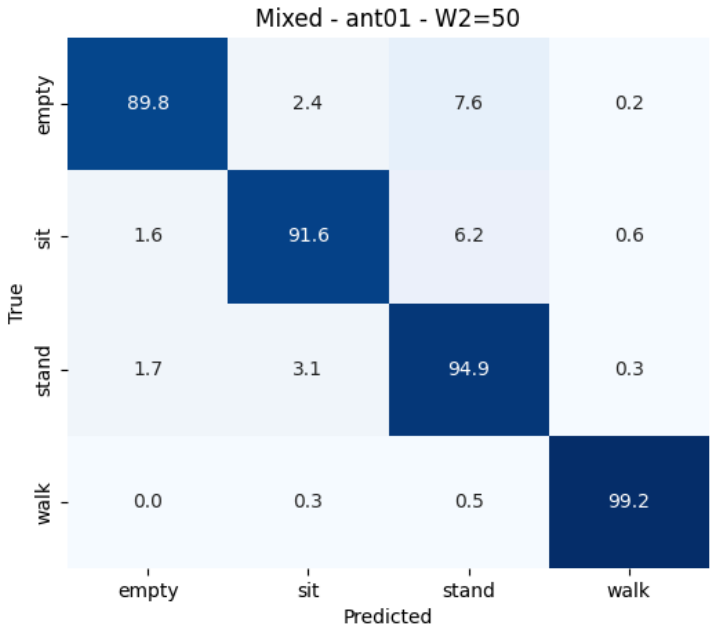


Figure 6.4. Confusion matrix: mixed, antenna pair 0 – 1, $W_2 = 50$.

The cross-room experiments represent the most challenging scenario because the classifier is trained and tested on entirely different environments. Figures 6.5–6.6 illustrate representative cross-room confusion matrices, including the worst-performing experimental case.

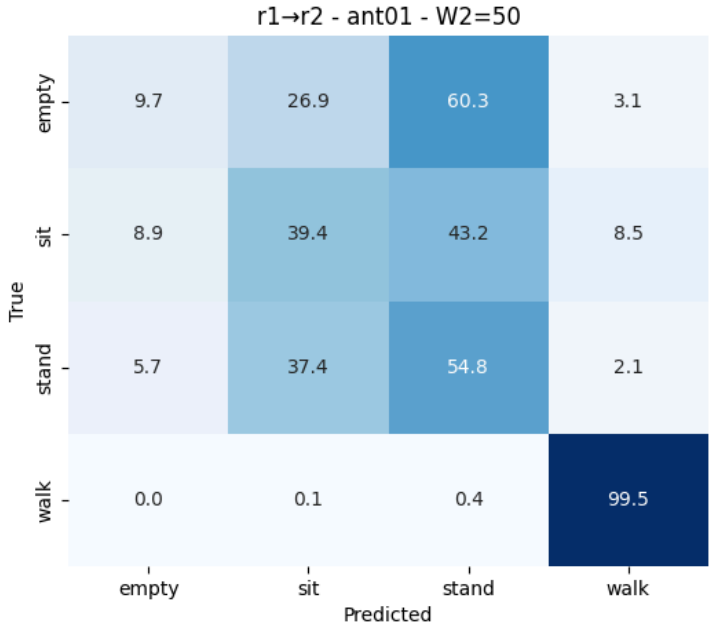


Figure 6.5. Confusion matrix: r1→r2, antenna pair 0 – 1, $W_2 = 50$.

In the r1→r2 setup using antenna pair 0 – 1 and $W_2 = 50$, the classifier shows significant confusion between the empty, sitting, and standing classes. For example, many empty samples are classified as standing, while sitting and standing are frequently misclassified between each other. However, the walking class remains highly distinguishable with 99.5% accuracy.

The worst-performing confusion matrix was obtained in the r2→r1 setup using antenna pair 2 – 3 with $W_2 = 50$ (Figure 6.6), corresponding to the lowest overall accuracy observed in the experiments. Severe confusion is visible among all activity classes. In particular, only 7.4% of sitting samples and 22.4% of standing samples were correctly classified. Most sitting and standing activities were incorrectly predicted as empty, while 33.3% of walking samples were also classified as empty.

These results indicate that antenna pair selection strongly influences classification robustness, especially in cross-room scenarios. Antenna pair 2 – 3 appears to capture less discriminative CSI dynamics compared to pairs such as 0 – 1 or 1 – 3. Furthermore, the results confirm that static activities are considerably more sensitive to environmental changes than dynamic activities such as walking, whose CSI signatures remain more stable across different rooms.

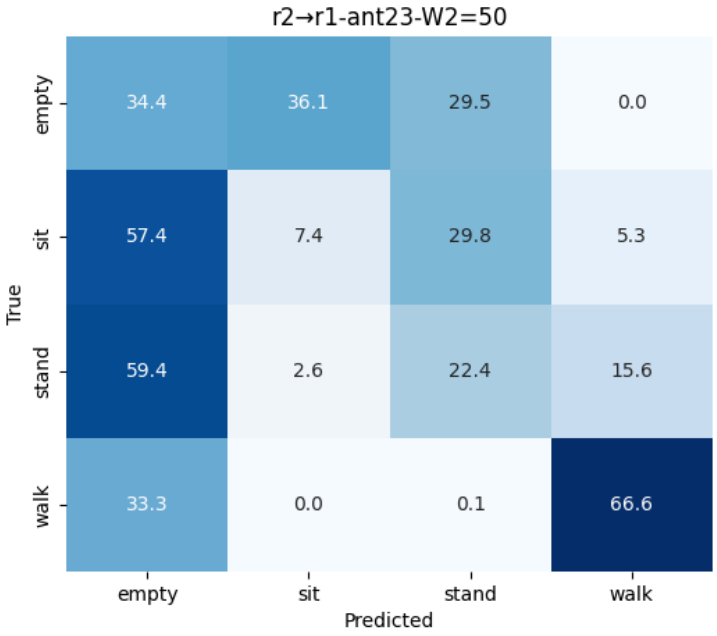


Figure 6.6. Confusion matrix: r2→r1, antenna pair 2 – 3, $W_2 = 50$.

Chapter 7

Conclusions

This thesis investigated WiFi-based human activity recognition using Channel State Information (CSI) and digital receive beamforming on commodity IEEE 802.11ac devices. The work addressed several major challenges associated with CSI-based sensing, including unstable CSI phase measurements, synchronization-induced distortions, wideband subband discontinuities, environmental sensitivity, and robust activity classification across different indoor environments. The proposed framework combined CSI phase preprocessing, receive beamforming, CSI distance analysis, temporal motion analysis, feature extraction, and machine learning classification into a complete sensing pipeline.

The first contribution of the thesis was the development of a CSI preprocessing pipeline for 80 MHz commodity WiFi measurements. The analysis demonstrated that raw CSI phase measurements are heavily corrupted by hardware-related impairments such as PLL-induced antenna phase offsets, Sampling Frequency Offset (SFO), Symbol Timing Offset (STO), and subband stitching discontinuities. To mitigate these effects, the proposed framework introduced pilot subcarrier reconstruction, PLL bias compensation, and segmentwise Linear Regression Transformation (LRT). The preprocessing results showed substantially improved phase continuity and inter-antenna consistency, which are essential prerequisites for reliable beamforming-based sensing.

The thesis further demonstrated that receive digital beamforming can effectively enhance motion-sensitive channel variations. By combining CSI measurements from multiple antennas using adaptive complex weights, the proposed method emphasized dynamic multipath reflections associated with human movement while suppressing static environmental components. The resulting beamformed CSI distance signals clearly separated walking activity from static activities such as empty-room, sitting, and standing conditions. The experiments also showed that different antenna pairs provide different levels of sensitivity and robustness, highlighting the importance of spatial diversity in WiFi sensing systems.

A temporal analysis of CSI distance signals during walking activity was also performed. The periodicity analysis demonstrated that walking generates repeated dominant peaks in the CSI

distance sequence corresponding to recurring motion-induced channel perturbations. The extracted motion periods revealed quasi-periodic behavior with average motion cycles of approximately 11–12 seconds across both rooms. Although environmental differences introduced variability in the temporal statistics, the overall periodic structure remained observable in both environments. These results confirmed that CSI distance signals contain not only motion intensity information but also meaningful temporal dynamics related to human movement patterns.

The final part of the thesis focused on feature extraction and activity classification using Random Forest classifiers. Statistical CSI-distance features and beam-angle variance features were extracted using a second-level sliding window approach. Multiple classification scenarios were evaluated, including same-room, mixed-room, and cross-room training/testing configurations.

The experimental results demonstrated that the proposed framework achieves high classification performance in same-room scenarios. The best-performing configuration reached 96.9% classification accuracy using antenna pair $0 - 1$ and $W_2 = 50$ in the r2→r2 setup, while several additional same-room configurations exceeded 94% accuracy. These results indicate that the extracted beamforming-based features provide effective separability between empty, sitting, standing, and walking activities when the training and testing environments are similar.

The mixed-room experiments also achieved high performance, with accuracies above 94% for the best configurations. This indicates that the selected features preserve substantial robustness when data from different rooms are combined during training and testing. However, the cross-room experiments revealed a significant performance degradation, especially for static activities such as empty, sitting, and standing. The confusion matrices showed that these activities become difficult to separate when the model is trained and tested in different propagation environments. In contrast, walking remained substantially more robust across rooms because dynamic motion generates stronger and more consistent channel perturbations than static postures.

The confusion matrix analysis further demonstrated that environmental changes strongly affect CSI-based sensing systems. Same-room experiments produced highly diagonal confusion matrices, while cross-room setups showed severe class overlap, particularly

between empty, sitting, and standing activities. The worst-performing configuration, obtained using antenna pair 2 – 3 in the r2→r1 setup, confirmed that antenna selection significantly influences cross-environment robustness. These observations suggest that environmental dependence remains one of the primary limitations of CSI-based HAR systems using commodity WiFi hardware.

Overall, the thesis demonstrated that beamforming-enhanced CSI processing combined with statistical feature extraction and Random Forest classification provides an effective framework for passive WiFi-based human activity recognition. The proposed methodology successfully integrates signal preprocessing, spatial filtering, temporal motion analysis, and machine learning classification into a coherent sensing pipeline capable of operating on commodity IEEE 802.11ac hardware.

Several directions for future work can be identified. First, more advanced domain adaptation or transfer learning techniques could be investigated to improve cross-room generalization and reduce environmental sensitivity. Second, additional temporal and spectral features, including Doppler-based representations or deep-learning feature extraction methods, could further enhance classification performance. Third, adaptive beamforming strategies using larger antenna arrays or dynamic beam selection could improve motion sensitivity and robustness. Finally, extending the framework toward finer-grained activities, multi-person scenarios, and real-time implementation would further increase the practical applicability of WiFi-based passive sensing systems.

Figures

Tables

References

[1] S. Yousefi, H. Narui, S.Dayal, S. Ermon, and S. Valaee, “A Survey of Human Activity Recognition Using WiFi CSI,” arXiv preprint arXiv:1708.07129, 2017.

[2] J. Liu, F. Wang, Zhe Li , R. Xiong, T. Mi , R. C. Qiu, “A Wi-Fi Signal-Based Human Activity Recognition Using Intel 5300,” arXiv preprint arXiv:2311.05921, 2023.

[3] K. Xu, J. Wang, H. Zhu, D. Zheng, “Self-Supervised Learning for WiFi CSI-Based Human Activity Recognition: A Systematic Study,” arXiv preprint arXiv:2308.02412, 2023.

[4] G. Diaz, I. Sobron, I. Eizmendi, I. landa, M. Velez, “Channel Phase Processing in Wireless Networks for Human Activity Recognition”, arXiv preprint arXiv:2303.16873, 2023

[5] N. Keerativoranan, A. Haniz, K. Saito and J. Takada, “Mitigation of CSI Temporal Phase Rotation with B2B Calibration,” Sensors, vol. 18, no. 11, p. 3796, 2018.

[6] Q. Wang, H. Wang, L. Huang, C. Shi, J. Chen and R. Zhang, "WiVir: Exploiting WiFi-based Virtual Antenna Array for Passive Stationary Human Localization," 2025 IEEE International Symposium on Circuits and Systems (ISCAS), London, United Kingdom, 2025, pp. 1-5, doi: 10.1109/ISCAS56072.2025.11043246

[7] P. White, “Millimeter-Wave Beamforming: Antenna Array Design,” Rohde & Schwarz / IEEE MTT-S tutorial, 2020. (MTT-S)

[8] P. Loureiro, F. Guiomar, and P. Monteiro, “Visible Light Communications: A Survey on Recent High-Capacity Demonstrations and Digital Modulation Techniques,” Photonics, vol. 10. p. 993. 10.3390/photonics10090993, 2023.

[9] P. Almers, E. Bonek, A. Burr, N. Czink, M. Debbah, V. Degli-Esposti, H. Hofstetter, P. Kyösti, D. Laurenson, G. Matz, A. Molisch, C. Oestges, and H. Özcelik, “Survey of Channel and Radio Propagation Models for Wireless MIMO Systems,” EURASIP Journal on Wireless Communications and Networking. 10.1155/2007/19070, 2007.

[10] B. Allen and M. Ghavami, “Adaptive Array Systems: Fundamentals and Applications”, Wiley, 2005.

[11] L. C. Godara, “Applications of Antenna Arrays to Mobile Communications, Part II: Beam-Forming and Direction-of-Arrival Considerations,” Proceedings of the IEEE, vol. 85, no. 8, pp. 1195–1245, 1997.

[12] H. Krim and M. Viberg, “Two Decades of Array Signal Processing Research,” IEEE Signal Processing Magazine, vol. 13, no. 4, pp. 67–94, 1996.

[13] Y. Ma, G. Zhou and S. Wang, “WiFi Sensing with Channel State Information: A Survey,”

ACM Computing Surveys, vol. 52, no. 3, 2019.

[14] Y. Yousefi, H. Narui, S. Dayal, S. Ermon and S. Valaee, “A Survey on Behaviour Recognition Using WiFi Channel State Information,” arXiv preprint arXiv:1708.07129, 2017.

[15] D. Zhang, Y. Hu, Y. Chen and B. Zeng, “Calibrating Phase Offsets for Commodity WiFi,” IEEE Systems Journal, vol. 14, no. 2, pp. 2268–2279, 2020.

[16] Y. Wang, J. Yang, G. Sun and X. Wang, “Ranging and Fingerprinting-Based Indoor Wi-Fi Localization Using Channel State Information,” IEEE Communications Letters, 2016.

[17] X. Yang, J. Wang, M. Zhou, “Wi-Fi Indoor Localization Based on Channel State Information,” Wireless Networks, 2024.

[18] Hewlett-Packard Enterprise, “802.11ac In-Depth”, White Paper, 2013.

[19] R. Fallani, “Human Activity Recognition based on Wi-Fi CSI Data and Digital Receive Beamforming”, Master Thesis, Università degli Studi di Roma Tor Vergata, 2023.

[20] X. Dong, W.-S. Lu and A. C. K. Soong, “Linear Interpolation in Pilot Symbol Assisted Channel Estimation for OFDM,” IEEE Transactions on Wireless Communications, vol. 6, no. 5, pp. 1910–1920, 2007.

[21] S. Coleri, M. Ergen, A. Puri and A. Bahai, “Channel Estimation Techniques Based on Pilot Arrangement in OFDM Systems,” IEEE Transactions on Broadcasting, vol. 48, no. 3, pp. 223–229, 2002.

[22] A. Zubow, P. Gawłowicz and F. Dressler, “On Phase Offsets of 802.11ac Commodity WiFi,” arXiv preprint arXiv:2005.03755, 2020.

[23] N. Keerativoranan, A. Haniz, K. Saito and J. Takada, “Mitigation of CSI Temporal Phase Rotation with B2B Calibration Method for Fine-Grained Motion Detection Analysis on Commodity Wi-Fi Devices,” Sensors, vol. 18, no. 11, p. 3795, 2018.

[24] M. De Sanctis, R. Fallani, T. Rossi, E. Cianca, M. Ruggieri, and V. Poulkov, “WiFi Sensing Through Digital Receive Beamforming and CSI,” in *2025 18th International Conference on Telecommunications (ConTEL)*, 2025.

Repository and Source Code

The implementation developed for this thesis, including CSI preprocessing, beamforming, feature extraction, and classification scripts, is available at:

https://github.com/aselya9185/har-wifi-csi

The repository contains the Python implementations referenced throughout Appendix A-B, together with experiment scripts and visualization utilities.

Appendix A

Python Implementation

This appendix presents the main Python scripts developed for CSI preprocessing, beamforming, feature extraction, temporal analysis, and activity classification. The implementation was developed using NumPy, SciPy, Scikit-learn, Pandas, and Matplotlib.

A.1 CSI Extraction and Reconstruction Files

Utility functions used throughout the CSI preprocessing and visualization stages. The functions support subcarrier reconstruction, phase unwrapping, and standardized plotting of CSI measurements.

utils.py

```
import numpy as np
import matplotlib.ticker as ticker

# ========================
# INDEX MAPPING
# ========================

NON_RECONSTRUCT = [0,1,2,3,4,5,128,251,252,253,254,255]
REMOVED                                                                    =
[0,1,2,3,4,5,25,53,89,117,127,128,129,139,167,203,231,251,252,253,254,255]

def get_target_indices():
    mask = np.ones(256, dtype=bool)
    mask[NON_RECONSTRUCT] = False
    return np.where(mask)[0]

def get_data_indices():
    all_idx = np.arange(256)
    mask = np.ones(256, dtype=bool)
```

```
    mask[REMOVED] = False

    return all_idx[mask]  # 234

# ==========================
# AXIS
# ==========================

def k_axis():
    return np.arange(-128, 128)


# ==========================
# RECONSTRUCTION
# ==========================

def build_full(x, indices):
    A, T, _ = x.shape
    full = np.full((A, T, 256), np.nan)
    full[:, :, indices] = x
    return full


# ==========================
# UNWRAP (NaN SAFE)
# ==========================

def unwrap_with_nan_full(x):
    out = x.copy()
    for a in range(x.shape[0]):
        for t in range(x.shape[1]):
            valid = ~np.isnan(x[a, t])
            if np.sum(valid) > 1:
                out[a, t, valid] = np.unwrap(x[a, t, valid])
    return out


def unwrap_with_nan_single(x):
    out = x.copy()
    for a in range(x.shape[0]):
        valid = ~np.isnan(x[a])
```

```
        if np.sum(valid) > 1:
            out[a, valid] = np.unwrap(x[a, valid])
    return out


# =========================
# PLOTTING
# =========================

def plot_dataset(ax, data):
    k = k_axis()
    for a in range(data.shape[0]):
            ax.plot(k,  np.ma.masked_invalid(data[a]),  label=f"Antenna  {a}",
linewidth=1)

    ax.grid(True)
    ax.legend(fontsize=8)


def set_pi_ticks(ax):
    ax.yaxis.set_major_locator(ticker.MultipleLocator(2*np.pi))
    ax.yaxis.set_major_formatter(
        ticker.FuncFormatter(lambda val, pos: f"{val/np.pi:.0g}π" if val != 0
else "0")
    )
```

Reconstruction of removed pilot and null subcarriers using linear interpolation in order to restore CSI frequency continuity.

interpolation.py

```
import numpy as np
import os
import matplotlib.pyplot as plt
from utils import (
   k_axis,
   set_pi_ticks,
   get_target_indices,
   build_full
)

# =========================
# 1. DEFINE CONSTANTS
# =========================

REMOVED                                                                    =
[0,1,2,3,4,5,25,53,89,117,127,128,129,139,167,203,231,251,252,253,254,255]
PILOTS = [25, 53, 89, 117, 127, 129, 139, 167, 203, 231]

# =========================
# 2. INDEX MAPPING
# =========================

all_idx = np.arange(256)

mask = np.ones(256, dtype=bool)
mask[REMOVED] = False
data_indices = all_idx[mask]    # 234

target_indices = get_target_indices()  # 244

# =========================
# 3. INTERPOLATION
# =========================
```

```
def interpolate_pilots_packet(csi_packet):
    full_amp = np.full(256, np.nan)
    full_phase = np.full(256, np.nan)

    # extract
    amp = np.abs(csi_packet)
    phase = np.unwrap(np.angle(csi_packet))

    # place known
    full_amp[data_indices] = amp
    full_phase[data_indices] = phase

    # interpolate pilots
    missing = sorted(PILOTS)

    i = 0
    while i < len(missing):

        start = missing[i]
        j = i

        while j + 1 < len(missing) and missing[j+1] == missing[j] + 1:
            j += 1

        end = missing[j]

        left = start - 1
        while left >= 0 and np.isnan(full_amp[left]):
            left -= 1

        right = end + 1
        while right < 256 and np.isnan(full_amp[right]):
            right += 1

        if left >= 0 and right < 256:
            for p in range(start, end + 1):
                ratio = (p - left) / (right - left)

                        full_amp[p] = full_amp[left] + (full_amp[right] -
full_amp[left]) * ratio
```

```
                    full_phase[p] = full_phase[left] + (full_phase[right] -
full_phase[left]) * ratio
        else:
            for p in range(start, end + 1):
                full_amp[p] = np.nan
                full_phase[p] = np.nan

        i = j + 1

    return full_amp[target_indices], full_phase[target_indices]

# =========================
# 4. PROCESS CSI
# =========================

def process_csi_array(csi):
    A, T, _ = csi.shape

    amp_out = np.full((A, T, len(target_indices)), np.nan)
    phase_out = np.full((A, T, len(target_indices)), np.nan)

    for a in range(A):
        for t in range(T):
             amp_out[a, t], phase_out[a, t] = interpolate_pilots_packet(csi[a,
t])

    return amp_out, phase_out

# =========================
# 5. SAVE
# =========================

def process_directory(input_dir, output_dir):

    amp_dir = os.path.join(output_dir, "amplitude")
    phase_dir = os.path.join(output_dir, "phase")

    os.makedirs(amp_dir, exist_ok=True)
    os.makedirs(phase_dir, exist_ok=True)

    for filename in os.listdir(input_dir):
```

```
        if filename.endswith(".npy"):
            print(f"Processing: {filename}")

            csi = np.load(os.path.join(input_dir, filename))

            amp, phase = process_csi_array(csi)

            base = filename.replace(".npy", "")

            np.save(os.path.join(amp_dir, f"{base}_amp.npy"), amp)
            np.save(os.path.join(phase_dir, f"{base}_phase.npy"), phase)

# ==========================
# 6. HELPERS
# ==========================

def build_full_before(csi):
    A, T, _ = csi.shape
    full = np.full((A, T, 256), np.nan, dtype=np.complex128)
    full[:, :, data_indices] = csi
    return full

# ==========================
# 7. PLOTTING
# ==========================

def plot_dataset(ax, data, is_phase=False):
    k = k_axis()

    for a in range(data.shape[0]):
            ax.plot(k,  np.ma.masked_invalid(data[a]),  label=f"Antenna  {a}",
linewidth=1)

    for pilot in PILOTS:
        ax.axvline(x=pilot - 128, color='pink', linestyle='--', linewidth=0.7)

    ax.grid(True)
    ax.legend(fontsize=8)

# ==========================
# 8. PHASE UNWRAP WITH NAN HElPER
```

```
# ==========================

def unwrap_with_nan(x):
    out = x.copy()
    for a in range(x.shape[0]):
        valid = ~np.isnan(x[a])
        out[a, valid] = np.unwrap(x[a, valid])
    return out

# ==========================
# 9. CORE PLOT
# ==========================

def plot_pair(f1, f2, title):

    PACKET_IDX = 1000

    # BEFORE
     csi1_b = build_full_before(np.load(os.path.join("dataset/saved_csi_raw",
f1)))[:, PACKET_IDX, :]
     csi2_b = build_full_before(np.load(os.path.join("dataset/saved_csi_raw",
f2)))[:, PACKET_IDX, :]

    # AFTER
        amp1  =  np.load(f"dataset/interpolation/amplitude/{f1.replace('.npy',
'_amp.npy')}")
         phase1  =  np.load(f"dataset/interpolation/phase/{f1.replace('.npy',
'_phase.npy')}")

        amp2  =  np.load(f"dataset/interpolation/amplitude/{f2.replace('.npy',
'_amp.npy')}")
         phase2  =  np.load(f"dataset/interpolation/phase/{f2.replace('.npy',
'_phase.npy')}")

    amp1 = build_full(amp1, target_indices)[:, PACKET_IDX, :]
    phase1 = build_full(phase1, target_indices)[:, PACKET_IDX, :]

    amp2 = build_full(amp2, target_indices)[:, PACKET_IDX, :]
    phase2 = build_full(phase2, target_indices)[:, PACKET_IDX, :]

    phase1 = unwrap_with_nan(phase1)
```

```
    phase2 = unwrap_with_nan(phase2)

    # ========================
    # BEFORE
    # ========================
    fig, axs = plt.subplots(2,2, figsize=(12,8), sharex=True, sharey='row')

    axs[0,0].set_title("Room 1")
    axs[0,1].set_title("Room 2")

        fig.text(0.02,  0.75,  "Amplitude",  va='center',  rotation='vertical',
fontsize=12)
         fig.text(0.02,   0.25,   "Phase",   va='center',   rotation='vertical',
fontsize=12)

    plot_dataset(axs[0,0], np.abs(csi1_b))
    plot_dataset(axs[0,1], np.abs(csi2_b))

    phase1_b = unwrap_with_nan(np.angle(csi1_b))
    phase2_b = unwrap_with_nan(np.angle(csi2_b))

    plot_dataset(axs[1, 0], phase1_b)
    set_pi_ticks(axs[1, 0])

    plot_dataset(axs[1, 1], phase2_b)
    set_pi_ticks(axs[1, 1])

    fig.suptitle(f"{title} - BEFORE Interpolation")
    plt.tight_layout(rect=[0.06, 0, 1, 1])
    plt.show()

    # ========================
    # AFTER
    # ========================
    fig, axs = plt.subplots(2, 2, figsize=(12,8), sharex=True, sharey='row')

    axs[0,0].set_title("Room 1")
    axs[0,1].set_title("Room 2")

        fig.text(0.02,  0.75,  "Amplitude",  va='center',  rotation='vertical',
fontsize=12)
```

```
        fig.text(0.02,   0.25,   "Phase",   va='center',   rotation='vertical',
fontsize=12)

    plot_dataset(axs[0,0], amp1)
    plot_dataset(axs[0,1], amp2)

    plot_dataset(axs[1, 0], phase1)
    set_pi_ticks(axs[1, 0])

    plot_dataset(axs[1, 1], phase2)
    set_pi_ticks(axs[1, 1])

    fig.suptitle(f"{title} - AFTER Interpolation")
    plt.tight_layout(rect=[0.06, 0, 1, 1])
    plt.show()

# ==========================
# 10. RUN
# ==========================

process_directory("dataset/saved_csi_raw", "dataset/interpolation")

pairs = [
    ("r1_empty.npy", "r2_empty_1.npy", "Empty"),
    ("r1_sitting_1.npy", "r2_sit_1.npy", "Sitting"),
    ("r1_standing_1.npy", "r2_standing_1.npy", "Standing"),
    ("r1_walking_1.npy", "r2_walk_1.npy", "Walking"),
]

for f1, f2, title in pairs:
    plot_pair(f1, f2, title)
```

Compensation of antenna-dependent constant phase offsets caused by independent RF chains and PLL initialization.

PLL_bias.py

```
import numpy as np
import matplotlib.pyplot as plt
import os
import glob
from utils import (
    get_target_indices,
    build_full,
    unwrap_with_nan_full,
    unwrap_with_nan_single,
    plot_dataset,
    set_pi_ticks
)

# ==========================
# 0. DIRECTORIES
# ==========================
input_dir = "dataset/interpolation/phase"
output_dir = "dataset/phase_processing/PLL"
os.makedirs(output_dir, exist_ok=True)

PACKET_IDX = 1000

# ==========================
# 1. INDEX MAPPING
# ==========================

all_idx = np.arange(256)

target_indices = get_target_indices() # 244

# ==========================
# 2. PLL BIAS REMOVAL
# ==========================

def remove_per_antenna_bias_full(phases):
    out = phases.copy()
```

```
    for t in range(phases.shape[1]):

        valid = ~np.isnan(phases[0, t])
        if not np.any(valid):
            continue

        ref_idx = np.where(valid)[0][0]

        for a in range(phases.shape[0]):
            if not np.isnan(phases[a, t, ref_idx]):
                bias = phases[a, t, ref_idx]
                valid_a = ~np.isnan(phases[a, t])
                out[a, t, valid_a] = phases[a, t, valid_a] - bias

    return out

# ========================
# 3. PROCESS ALL FILES
# ========================

files = glob.glob(os.path.join(input_dir, "*.npy"))

for file_path in files:
    filename = os.path.basename(file_path)
    print(f"Processing {filename}")

    phase = np.load(file_path)

    phase_unwrapped = unwrap_with_nan_full(phase)
    phase_pll = remove_per_antenna_bias_full(phase_unwrapped)

    np.save(os.path.join(output_dir, filename), phase_pll)

print("All files processed and saved.")

# ========================
# 4. LOAD EXAMPLES
# ========================

phase_empty = np.load(os.path.join(input_dir, "r1_empty_phase.npy"))
```

```
phase_walk = np.load(os.path.join(input_dir, "r1_walking_1_phase.npy"))

phase_empty_pll = np.load(os.path.join(output_dir, "r1_empty_phase.npy"))
phase_walk_pll = np.load(os.path.join(output_dir, "r1_walking_1_phase.npy"))

empty_full = build_full(phase_empty, target_indices)
walk_full = build_full(phase_walk, target_indices)

empty_pll_full = build_full(phase_empty_pll, target_indices)
walk_pll_full = build_full(phase_walk_pll, target_indices)

empty_before = unwrap_with_nan_single(empty_full[:, PACKET_IDX, :])
walk_before = unwrap_with_nan_single(walk_full[:, PACKET_IDX, :])

empty_after = unwrap_with_nan_single(empty_pll_full[:, PACKET_IDX, :])
walk_after = unwrap_with_nan_single(walk_pll_full[:, PACKET_IDX, :])

combined = np.concatenate([
   empty_before.flatten(),
   walk_before.flatten(),
   empty_after.flatten(),
   walk_after.flatten()
])

valid = combined[~np.isnan(combined)]

y_min = np.min(valid)
y_max = np.max(valid)

margin = 0.05 * (y_max - y_min)
y_min -= margin
y_max += margin

# =========================
# 5. PLOT BEFORE
# =========================

fig_before, axs_before = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

fig_before.suptitle("Before PLL Bias Removal", fontsize=14)
```

```
axs_before[0].set_title("Empty - BEFORE PLL")
axs_before[1].set_title("Walking - BEFORE PLL")

plot_dataset(axs_before[0], empty_before)
plot_dataset(axs_before[1], walk_before)

for ax in axs_before:
   ax.set_ylim(y_min, y_max)
   ax.set_xlabel("Subcarrier Index")
   ax.set_ylabel("Phase")
   set_pi_ticks(ax)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

# =========================
# 6. PLOT AFTER
# =========================

fig_after, axs_after = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

fig_after.suptitle("After PLL Bias Removal", fontsize=14)

axs_after[0].set_title("Empty - AFTER PLL")
axs_after[1].set_title("Walking - AFTER PLL")

plot_dataset(axs_after[0], empty_after)
plot_dataset(axs_after[1], walk_after)

for ax in axs_after:
   ax.set_ylim(y_min, y_max)
   ax.set_xlabel("Subcarrier Index")
   ax.set_ylabel("Phase")
   set_pi_ticks(ax)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

Segmentwise Linear Regression Transformation used for SFO/STO mitigation and phase slope removal across OFDM subbands.

LRT.py

```
import numpy as np
import matplotlib.pyplot as plt
import os
import glob
from utils import (
    build_full,
    get_target_indices,
    unwrap_with_nan_single,
    plot_dataset,
    set_pi_ticks
)
from matplotlib.lines import Line2D

# ==========================
# 0. DIRECTORIES
# ==========================
input_dir = "dataset/phase_processing/PLL"
output_dir = "dataset/phase_processing/LRT"
os.makedirs(output_dir, exist_ok=True)

PACKET_IDX = 1000

# ==========================
# 1. INDEX SETUP
# ==========================
target_indices = get_target_indices()  # 244

# segment boundaries (on 244 domain)
segments_bounds = [
    (0, 58),
    (58, 122),
    (122, 185),
    (185, 244)
]
# ==========================
# 2. LRT FUNCTION
```

```
# ==========================
def apply_lrt_segmented(phase):
    # phase: (A, T, 244)
    A, T, K = phase.shape
    corrected = np.full_like(phase, np.nan)

    for a in range(A):
        for t in range(T):
            for (start, end) in segments_bounds:
                seg = phase[a, t, start:end]

                valid = ~np.isnan(seg)
                if np.sum(valid) < 2:
                    continue

                x = np.arange(start, end)[valid]
                y = seg[valid]

                # linear regression
                slope, intercept = np.polyfit(x, y, 1)
                trend = slope * x + intercept

                corrected[a, t, start:end][valid] = y - trend

    return corrected

# ==========================
# 3. DRAW STITCHING LINES
# ==========================
def plot_stitching_lines(ax, target_indices, segments_bounds):
    for i in range(1, len(segments_bounds)):

        prev_end = segments_bounds[i-1][1]
        curr_start = segments_bounds[i][0]

        # map both indices
        k_prev = target_indices[prev_end - 1]
        k_curr = target_indices[curr_start]

        # midpoint → visually correct boundary
        k_plot = ((k_prev + k_curr) / 2) - 128
```

```
        ax.axvline(x=k_plot, color='deeppink', linestyle='--', linewidth=0.7)

# ==========================
# 4. PROCESS ALL FILES
# ==========================
files = glob.glob(os.path.join(input_dir, "*_phase.npy"))

for file_path in files:
    filename = os.path.basename(file_path)
    print(f"Processing {filename}")

    phase = np.load(file_path)

    # LRT
    phase_lrt = apply_lrt_segmented(phase)

    # Wrap to [-π, π]
    phase_lrt = (phase_lrt + np.pi) % (2 * np.pi) - np.pi

    np.save(os.path.join(output_dir, filename), phase_lrt)

print("All files processed.")

# ==========================
# 5. LOAD EXAMPLES
# ==========================
phase_empty = np.load(os.path.join(input_dir, "r1_empty_phase.npy"))
phase_walk = np.load(os.path.join(input_dir, "r1_walking_1_phase.npy"))

phase_empty_lrt = np.load(os.path.join(output_dir, "r1_empty_phase.npy"))
phase_walk_lrt = np.load(os.path.join(output_dir, "r1_walking_1_phase.npy"))

# ==========================
# 6. REBUILD FULL 256 FOR PLOTTING
# ==========================
empty_full = build_full(phase_empty, target_indices)
walk_full = build_full(phase_walk, target_indices)

empty_lrt_full = build_full(phase_empty_lrt, target_indices)
walk_lrt_full = build_full(phase_walk_lrt, target_indices)
```

```
# ==========================
# 7. EXTRACT PACKET
# ==========================
empty_before = unwrap_with_nan_single(empty_full[:, PACKET_IDX, :])
walk_before = unwrap_with_nan_single(walk_full[:, PACKET_IDX, :])

empty_after = empty_lrt_full[:, PACKET_IDX, :]
walk_after = walk_lrt_full[:, PACKET_IDX, :]

# ==========================
# 8. GLOBAL Y LIMITS
# ==========================
combined = np.concatenate([
   empty_before.flatten(),
   walk_before.flatten()
])

valid = combined[~np.isnan(combined)]

y_min = np.min(valid)
y_max = np.max(valid)

margin = 0.05 * (y_max - y_min)
y_min -= margin
y_max += margin

# ==========================
# 9. PLOT BEFORE
# ==========================
fig_before, axs_before = plt.subplots(2, 1, figsize=(6, 10), sharex=True)

fig_before.suptitle("Before LRT (After PLL)", fontsize=14)

axs_before[0].set_title("Empty")
axs_before[1].set_title("Walking")

plot_dataset(axs_before[0], empty_before)
plot_dataset(axs_before[1], walk_before)

plot_stitching_lines(axs_before[0], target_indices, segments_bounds)
```

```
plot_stitching_lines(axs_before[1], target_indices, segments_bounds)

stitch_legend = Line2D(
    [0], [0],
    color='deeppink',
    linestyle='--',
    lw=1.2,
    label='Subband stitching points'
)

fig_before.legend(
    handles=[stitch_legend],
    loc='upper center',
    bbox_to_anchor=(0.5, 0.96),
    frameon=False
)

for ax in axs_before:
    ax.set_ylim(y_min, y_max)
    set_pi_ticks(ax)
    ax.set_ylabel("Phase")

axs_before[1].set_xlabel("Subcarrier Index")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

# =========================
# 10. PLOT AFTER
# =========================
fig_after, axs_after = plt.subplots(2, 1, figsize=(6, 10), sharex=True)

fig_after.suptitle("After LRT (Slope Removed)", fontsize=14)

axs_after[0].set_title("Empty")
axs_after[1].set_title("Walking")

plot_dataset(axs_after[0], empty_after)
plot_dataset(axs_after[1], walk_after)

plot_stitching_lines(axs_after[0], target_indices, segments_bounds)
```

```
plot_stitching_lines(axs_after[1], target_indices, segments_bounds)

fig_after.legend(
   handles=[stitch_legend],
   loc='upper center',
   bbox_to_anchor=(0.5, 0.96),
   frameon=False
)

for ax in axs_after:
   ax.set_ylim(-np.pi, np.pi)

   ax.set_yticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi])
   ax.set_yticklabels([
       r"$-\pi$", r"$-\frac{\pi}{2}$", "0",
       r"$\frac{\pi}{2}$", r"$\pi$"
   ])

   ax.set_ylabel("Phase")

axs_after[1].set_xlabel("Subcarrier Index")

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

A.2 Beamforming and CSI Distance Processing

Computation of beamformed CSI distance using receive digital beamforming and Euclidean distance analysis.

csi_distance.py

```
import numpy as np
import matplotlib.pyplot as plt
import os

# =========================
# DIRECTORIES
# =========================
input_dir = "dataset/reconstructed_csi"
dist_dir = "dataset/csi_distance"
theta_dir = "dataset/theta"

plot_dist_dir = "plots/csi_distance_new"
plot_theta_dir = "plots/optimal_beam_new"
plot_compare_dir = "plots/csi_distance_compare"

os.makedirs(dist_dir, exist_ok=True)
os.makedirs(theta_dir, exist_ok=True)

# =========================
# PARAMETERS
# =========================
W = 16
theta_vals = np.linspace(-np.pi, np.pi, 180, endpoint=False)

files = sorted([
    f for f in os.listdir(input_dir)
    if f.endswith("_reconstructed.npy")
])



antenna_pairs = [
    (0, 1),
```

```
    (0, 2),
    (0, 3),
    (1, 2),
    (1, 3),
    (2, 3)
]

# store for comparison plots
comparison_data = {}

# =========================
# HELPERS
# =========================
def angle_diff(a, b):
    return np.angle(np.exp(1j * (a - b)))

def set_wrapped_pi_ticks(ax):
    ax.set_yticks([-np.pi, -np.pi/2, 0, np.pi/2, np.pi])
    ax.set_yticklabels([
        r"$-\pi$", r"$-\frac{\pi}{2}$", r"$0$", r"$\frac{\pi}{2}$", r"$\pi$"
    ])

# =========================
# MAIN PROCESS
# =========================
for filename in files:

    name = filename.replace("_reconstructed.npy", "")
    path = os.path.join(input_dir, filename)

    if not os.path.exists(path):
        print(f"Missing file: {path}")
        continue

    # create folders for plots
    os.makedirs(os.path.join(plot_dist_dir, name), exist_ok=True)
    os.makedirs(os.path.join(plot_theta_dir, name), exist_ok=True)

    csi = np.load(path)
    packets = csi.shape[1]
```

```
for (ant1, ant2) in antenna_pairs:

    print(f"  Antennas: {ant1}-{ant2}")

    h1 = csi[ant1, :, :]
    h2 = csi[ant2, :, :]

    best_distances = []
    best_thetas = []

    prev_theta = 0.0

    for start in range(0, packets - W):

        window_h1 = h1[start:start + W]
        window_h2 = h2[start:start + W]

        candidates = []

        # scan all theta
        for theta in theta_vals:
            w2 = np.exp(1j * theta)

            ybf = window_h1 + w2 * window_h2

            diff = ybf[1:] - ybf[:-1]
            d = np.sqrt(np.sum(np.abs(diff) ** 2, axis=1))
            mean_d = np.mean(d)

            candidates.append((theta, mean_d))

        # sort by distance (descending)
        candidates = sorted(candidates, key=lambda x: -x[1])

        # take top-K to avoid noise
        top_k = candidates[:5]

        # choose theta closest to previous
        best_theta, best_distance = min(
            top_k,
            key=lambda x: abs(angle_diff(x[0], prev_theta))
```

```
            )

            prev_theta = best_theta

            best_distances.append(best_distance)
            best_thetas.append(best_theta)

        best_distances = np.array(best_distances)
        best_thetas = np.array(best_thetas)

        best_thetas_wrapped = (best_thetas + np.pi) % (2 * np.pi) - np.pi

        # ==========================
        # SAVE
        # ==========================
           np.save(os.path.join(dist_dir, f"{name}_ant{ant1}{ant2}_dist.npy"),
best_distances)
         np.save(os.path.join(theta_dir, f"{name}_ant{ant1}{ant2}_theta.npy"),
best_thetas_wrapped)

        # ==========================
        # STORE FOR COMPARISON
        # ==========================
        key = f"{name}_ant{ant1}{ant2}"
        comparison_data[key] = best_distances

        # ==========================
        # DISTANCE PLOT
        # ==========================
        plt.figure(figsize=(12, 6), dpi=150)

        plt.plot(best_distances)
        plt.title(f"{name} - (Ant {ant1}-{ant2})")
        plt.xlabel("Window index")
        plt.ylabel("Mean CSI distance")
        plt.ylim(0, 8e13)
        plt.grid(True)
        plt.tight_layout()

        save_path = os.path.join(
            plot_dist_dir, name, f"{name}_{ant1}-{ant2}.png"
```

```
        )
        plt.savefig(save_path)
        plt.close()

        # ========================
        # THETA PLOT
        # ========================
        plt.figure(figsize=(12, 6), dpi=150)

        plt.plot(best_thetas_wrapped)
        plt.title(f"{name} - (Ant {ant1}-{ant2})")
        plt.xlabel("Window index")
        plt.ylabel("θ")
        set_wrapped_pi_ticks(plt.gca())
        plt.grid(True)
        plt.tight_layout()

        save_path = os.path.join(
            plot_theta_dir, name, f"{name}_{ant1}-{ant2}.png"
        )
        plt.savefig(save_path)
        plt.close()


# ========================
# CSI DISTANCE COMPARISON PLOTS
# ========================

def plot_comparison(room_prefix, activity_names):
    os.makedirs(os.path.join(plot_compare_dir, room_prefix), exist_ok=True)

    for (ant1, ant2) in antenna_pairs:

        plt.figure(figsize=(12, 6), dpi=150)

        for name in activity_names:
            key = f"{name}_ant{ant1}{ant2}"
            if key in comparison_data:
                plt.plot(
                    comparison_data[key],
                    label=name,
```

```
                    linewidth=0.9,
                )

        plt.title(f"{room_prefix.upper()} - Ant {ant1}-{ant2}")
        plt.xlabel("Window index")
        plt.ylabel("CSI distance")
        plt.ylim(0, 8e13)

        plt.legend()
        plt.grid(True)
        plt.tight_layout()

        save_path = os.path.join(
            plot_compare_dir,
            room_prefix,
            f"{room_prefix}_ant_{ant1}-{ant2}.png"
        )

        plt.savefig(save_path)
        plt.close()


# R1 comparison
plot_comparison("r1", [
    "r1_empty",
    "r1_sitting_1",
    "r1_standing_1",
    "r1_walking_1"
])

# R2 comparison
plot_comparison("r2", [
    "r2_empty_1",
    "r2_sit_1",
    "r2_standing_1",
    "r2_walk_1"
])
```

Temporal periodicity analysis of walking activity using CSI distance peak detection and motion period estimation.

analyze_walking_period.py

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os

from scipy.signal import find_peaks


# ==========================
# DIRECTORIES
# ==========================
input_dir = "dataset/csi_distance"

plot_root = "plots/walking_peak_analysis"
csv_root = "plots/walking_peak_analysis/csv"

os.makedirs(plot_root, exist_ok=True)
os.makedirs(csv_root, exist_ok=True)

target_ant = "ant01"

# estimated effective sampling frequency
fs = 5.0  # Hz

# smoothing window
smooth_window = 5

# peak detection parameters
peak_prominence = 0.6e13
peak_distance  = 15   # minimum samples between peaks (to avoid detecting
multiple peaks in one step)
height = 2e13

# ==========================
# HELPERS
# ==========================
```

```
def moving_average(x, w=5):
    return np.convolve(x, np.ones(w) / w, mode='same')


def load_walking_files():
    files = sorted([
        f for f in os.listdir(input_dir)
        if f.endswith("_dist.npy")
        and target_ant in f
        and ("walk" in f or "walking" in f)
    ])
    return files


def detect_peaks(signal):
    peaks, properties = find_peaks(
        signal,
        prominence=peak_prominence,
        distance=peak_distance,
        height=height
    )
    return peaks, properties


# ==========================
# MAIN ANALYSIS
# ==========================
summary_results = []

files = load_walking_files()

print("\nWalking datasets found:")
for f in files:
    print(f)

for filename in files:

    print(f"\nProcessing: {filename}")

    path = os.path.join(input_dir, filename)
```

```
signal = np.load(path)

smooth_signal = moving_average(signal, smooth_window)

# DETECT PEAKS
peaks, properties = detect_peaks(smooth_signal)

peak_values = smooth_signal[peaks]

# PEAK INTERVALS
delta_n = np.diff(peaks)

delta_t = delta_n / fs

# METRICS
mean_delta_n = np.mean(delta_n) if len(delta_n) > 0 else np.nan
std_delta_n = np.std(delta_n) if len(delta_n) > 0 else np.nan

mean_period = np.mean(delta_t) if len(delta_t) > 0 else np.nan
std_period = np.std(delta_t) if len(delta_t) > 0 else np.nan

regularity_cv = (
    std_delta_n / mean_delta_n
    if mean_delta_n > 0 else np.nan
)

n_peaks = len(peaks)

# SAVE SUMMARY
summary_results.append([
    filename,
    n_peaks,
    mean_delta_n,
    std_delta_n,
    mean_period,
    std_period,
    regularity_cv
])

# CREATE OUTPUT FOLDER
dataset_name = filename.replace(".npy", "")
```

```
dataset_plot_dir = os.path.join(plot_root, dataset_name)

os.makedirs(dataset_plot_dir, exist_ok=True)

# ============================================================
# PLOT 1: SIGNAL + DETECTED PEAKS
# ============================================================
plt.figure(figsize=(14, 6), dpi=150)

plt.plot(signal, alpha=0.4, label="Raw signal")

plt.plot(
    smooth_signal,
    linewidth=2,
    label="Smoothed signal"
)

plt.plot(
    peaks,
    peak_values,
    "ro",
    markersize=5,
    label="Detected peaks"
)

plt.title(f"{dataset_name} - CSI Distance Peaks")
plt.xlabel("Window index")
plt.ylabel("Mean CSI distance")

plt.grid(True)
plt.legend()
plt.tight_layout()

plt.savefig(
    os.path.join(dataset_plot_dir, "signal_with_peaks.png")
)

plt.close()

# ============================================================
```

```
# PLOT 2: INTERVALS Δnk
# ============================================================
plt.figure(figsize=(12, 5), dpi=150)

plt.plot(delta_n, marker='o')

plt.title(f"{dataset_name} - Peak Intervals Δn")
plt.xlabel("Peak index")
plt.ylabel("Δn (samples)")

plt.grid(True)
plt.tight_layout()

plt.savefig(
    os.path.join(dataset_plot_dir, "peak_intervals.png")
)

plt.close()

# ============================================================
# PLOT 3: INTERVALS IN SECONDS
# ============================================================
plt.figure(figsize=(12, 5), dpi=150)

plt.plot(delta_t, marker='o')

plt.title(f"{dataset_name} - Peak-to-Peak Motion Period")
plt.xlabel("Peak index")
plt.ylabel("Period (seconds)")

plt.grid(True)
plt.tight_layout()

plt.savefig(
    os.path.join(dataset_plot_dir, "motion_period_seconds.png")
)

plt.close()

# ============================================================
# PLOT 4: HISTOGRAM OF Δn
```

```
# ============================================================
plt.figure(figsize=(10, 5), dpi=150)

plt.hist(delta_n, bins=15)

plt.title(f"{dataset_name} - Histogram of Δn")
plt.xlabel("Δn (samples)")
plt.ylabel("Count")

plt.grid(True)
plt.tight_layout()

plt.savefig(
    os.path.join(dataset_plot_dir, "histogram_delta_n.png")
)

plt.close()

# ============================================================
# PLOT 5: HISTOGRAM OF PERIODS
# ============================================================
plt.figure(figsize=(10, 5), dpi=150)

plt.hist(delta_t, bins=15)

plt.title(f"{dataset_name} - Histogram of Periods")
plt.xlabel("Period (seconds)")
plt.ylabel("Count")

plt.grid(True)
plt.tight_layout()

plt.savefig(
    os.path.join(dataset_plot_dir, "histogram_periods.png")
)

plt.close()

# ============================================================
# SAVE PEAKS CSV
# ============================================================
```

```
    peaks_df = pd.DataFrame({
        "peak_index": peaks,
        "peak_value": peak_values
    })

    peaks_df.to_csv(
        os.path.join(dataset_plot_dir, "detected_peaks.csv"),
        index=False
    )

    # ============================================================
    # SAVE INTERVALS CSV
    # ============================================================
    intervals_df = pd.DataFrame({
        "delta_n_samples": delta_n,
        "delta_t_seconds": delta_t
    })

    intervals_df.to_csv(
        os.path.join(dataset_plot_dir, "peak_intervals.csv"),
        index=False
    )

    print(f"Detected peaks: {n_peaks}")
    print(f"Mean Δn: {mean_delta_n:.2f}")
    print(f"Mean period: {mean_period:.2f} s")
    print(f"Regularity CV: {regularity_cv:.3f}")


# ==========================
# FINAL SUMMARY TABLE
# ==========================
summary_df = pd.DataFrame(summary_results, columns=[
    "dataset",
    "num_peaks",
    "mean_delta_n",
    "std_delta_n",
    "mean_period_seconds",
    "std_period_seconds",
    "regularity_cv"
])
```

```
summary_csv_path = os.path.join(
   csv_root,
   "walking_peak_summary.csv"
)

summary_df.to_csv(summary_csv_path, index=False)

print("\n====================================")
print("Walking peak analysis completed.")
print("====================================")

print("\nSummary:")
print(summary_df)
```

A.3 Feature Extraction and Dataset Generation

Extraction of statistical and beamforming-related features from CSI distance and theta sequences using second-level temporal windowing.

feature_extraction.py

```
import numpy as np
import os
from collections import defaultdict

# ==========================
# DIRECTORIES
# ==========================
dist_dir = "dataset/csi_distance"
theta_dir = "dataset/theta"
output_root = "dataset/features"

os.makedirs(output_root, exist_ok=True)

# ==========================
# PARAMETERS
# ==========================
W2_values = [10, 20, 30, 40, 50]

antenna_pairs = [
    "ant01", "ant02", "ant03",
    "ant12", "ant13", "ant23"
]

labels_map = {
    "empty": 0,
    "sit": 1,
    "sitting": 1,
    "standing": 2,
    "walk": 3,
    "walking": 3
}

# ==========================
```

```
# HELPERS
# ========================
def get_label(name):
    for key in labels_map:
        if key in name:
            return labels_map[key]
    raise ValueError(f"Unknown label in {name}")

def get_room(name):
    if name.startswith("r1"):
        return "r1"
    elif name.startswith("r2"):
        return "r2"
    else:
        return "unknown"

# ========================
# SAVE
# ========================
def save_dataset(data, base_out, subfolder, filename):
    if len(data) == 0:
        return

    os.makedirs(os.path.join(base_out, subfolder), exist_ok=True)

    arr = np.array(data)
    np.save(
        os.path.join(base_out, subfolder, filename),
        arr
    )

# ========================
# LOAD FILE INDEX
# ========================
files = sorted([f for f in os.listdir(dist_dir) if f.endswith("_dist.npy")])

# group files by antenna pair
pair_files = defaultdict(list)

for f in files:
    for pair in antenna_pairs:
```

```
        if pair in f:
            pair_files[pair].append(f)

# ========================
# MAIN FEATURE EXTRACTION
# ========================
for W2 in W2_values:

    print(f"\nProcessing W2 = {W2}")

    base_out = os.path.join(output_root, f"w2_{W2}")

    for pair in antenna_pairs:

        print(f"  Pair: {pair}")

        # containers per split
        data_r1 = []
        data_r2 = []
        data_all = []

        for dist_file in pair_files[pair]:

            name = dist_file.replace(f"_{pair}_dist.npy", "")

            theta_file = f"{name}_{pair}_theta.npy"

            dist_path = os.path.join(dist_dir, dist_file)
            theta_path = os.path.join(theta_dir, theta_file)

            if not os.path.exists(theta_path):
                continue

            dist = np.load(dist_path)
            theta = np.load(theta_path)

            # unwrap for stability
            theta = np.unwrap(theta)

            label = get_label(name)
            room = get_room(name)
```

```
            T = len(dist)

            for start in range(0, T - W2):

                d_win = dist[start:start + W2]
                t_win = theta[start:start + W2]

                features = [
                    np.mean(d_win), # mean distance
                     np.std(d_win),  # variability
                     np.max(d_win),  # peaks
                     np.var(t_win),  # beam stability
                ]

                sample = features + [label]

                # store per split
                if room == "r1":
                    data_r1.append(sample)
                elif room == "r2":
                    data_r2.append(sample)

                data_all.append(sample)

        save_dataset(data_r1, base_out, "r1", f"r1_{pair}.npy")
        save_dataset(data_r2, base_out, "r2", f"r2_{pair}.npy")
        save_dataset(data_all, base_out, "r1_r2", f"r1_r2_{pair}.npy")

print("\nAll feature datasets saved.")

# =========================
# DEBUG: CHECK ONE DATASET
# =========================
sample_path = os.path.join(
   output_root,
   "w2_20",         # pick any W2
   "r1_r2",         # pick split
   "r1_r2_ant01.npy"  # pick antenna
)
```

```
if os.path.exists(sample_path):
    data = np.load(sample_path)

    print("\n=== SAMPLE DATASET ===")
    print("Path:", sample_path)
    print("Shape:", data.shape)
    print("\nFirst 5 rows:")
    print(data[:5])
else:
    print("\nSample file not found:", sample_path)

# =========================
# DEBUG: WALKING SAMPLES
# =========================
if os.path.exists(sample_path):
    data = np.load(sample_path)

    labels = data[:, -1].astype(int)

    walking_data = data[labels == 3]

    print("\n=== WALKING SAMPLES (label = 3) ===")
    print("Total walking samples:", walking_data.shape[0])

    print("\nFirst 5 walking rows:")
    print(walking_data[:5])
```

A.4 Classification and Evaluation

Random Forest classification experiments for same-room, mixed-room, and cross-room activity recognition.

random_forest.py

```
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# ==========================
# PARAMETERS
# ==========================
W2_values = [10, 20, 30, 40, 50]

antenna_pairs = [
    "ant01", "ant02", "ant03",
    "ant12", "ant13", "ant23"
]

base_dir = "dataset/features"
plot_root = "plots"

os.makedirs(plot_root, exist_ok=True)

# ==========================
# HELPERS
# ==========================
def load_dataset(path):
    if not os.path.exists(path):
        return None
    return np.load(path)
```

```
def prepare_xy(data):
    X = data[:, :-1]
    y = data[:, -1].astype(int)
    return X, y

def scale_data(X_train, X_test):
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
    return X_train, X_test

def run_rf(X_train, y_train, X_test):
    model = RandomForestClassifier(class_weight="balanced", random_state=42)
    model.fit(X_train, y_train)
    return model.predict(X_test)

def save_conf_matrix(y_true, y_pred, save_path, title):
    cm = confusion_matrix(y_true, y_pred)
    cm_percent = cm.astype("float") / cm.sum(axis=1, keepdims=True) * 100

    plt.figure(figsize=(6, 5))
    sns.heatmap(cm_percent, annot=True, fmt=".1f", cmap="Blues", cbar=False)

    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(title)

    plt.xticks([0.5,1.5,2.5,3.5], ['empty','sit','stand','walk'])
    plt.yticks([0.5,1.5,2.5,3.5], ['empty','sit','stand','walk'])

    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    plt.savefig(save_path)
    plt.close()

# ==========================
# MAIN LOOP
# ==========================
results = []

for W2 in W2_values:
```

```
    print(f"\n=== W2 = {W2} ===")

    for pair in antenna_pairs:

        print(f"Pair: {pair}")

        path_r1 = os.path.join(base_dir, f"w2_{W2}", "r1", f"r1_{pair}.npy")
        path_r2 = os.path.join(base_dir, f"w2_{W2}", "r2", f"r2_{pair}.npy")
                  path_all  =  os.path.join(base_dir,  f"w2_{W2}",  "r1_r2",
f"r1_r2_{pair}.npy")

        data_r1 = load_dataset(path_r1)
        data_r2 = load_dataset(path_r2)
        data_all = load_dataset(path_all)

        if data_r1 is None or data_r2 is None or data_all is None:
            continue

        # ==========================
        # SETUP 1: MIXED
        # ==========================
        X, y = prepare_xy(data_all)

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.5, random_state=42
        )

        X_train, X_test = scale_data(X_train, X_test)

        y_pred = run_rf(X_train, y_train, X_test)

        acc = accuracy_score(y_test, y_pred)
        err = 1 - acc

        results.append(["mixed", pair, W2, acc, err])

        save_conf_matrix(
            y_test,
            y_pred,
            f"{plot_root}/confusion_matrix/w2_{W2}/mixed/{pair}.png",
            f"Mixed - {pair} - W2={W2}"
```

```
)

# ========================
# SETUP 2: r1 -> r2
# ========================
X_train, y_train = prepare_xy(data_r1)
X_test, y_test = prepare_xy(data_r2)

X_train, X_test = scale_data(X_train, X_test)

y_pred = run_rf(X_train, y_train, X_test)

acc = accuracy_score(y_test, y_pred)
err = 1 - acc

results.append(["r1_to_r2", pair, W2, acc, err])

save_conf_matrix(
    y_test,
    y_pred,
    f"{plot_root}/confusion_matrix/w2_{W2}/r1_to_r2/{pair}.png",
    f"r1→r2 - {pair} - W2={W2}"
)

# ========================
# SETUP 3: r2 -> r1
# ========================
X_train, y_train = prepare_xy(data_r2)
X_test, y_test = prepare_xy(data_r1)

X_train, X_test = scale_data(X_train, X_test)

y_pred = run_rf(X_train, y_train, X_test)

acc = accuracy_score(y_test, y_pred)
err = 1 - acc

results.append(["r2_to_r1", pair, W2, acc, err])

save_conf_matrix(
    y_test,
```

```
    y_pred,
    f"{plot_root}/confusion_matrix/w2_{W2}/r2_to_r1/{pair}.png",
    f"r2→r1-{pair}-W2={W2}"
)

# ==========================
# SETUP 4: r1 -> r1
# ==========================
X, y = prepare_xy(data_r1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=42
)

X_train, X_test = scale_data(X_train, X_test)

y_pred = run_rf(X_train, y_train, X_test)

acc = accuracy_score(y_test, y_pred)
err = 1 - acc

results.append(["r1_to_r1", pair, W2, acc, err])

save_conf_matrix(
    y_test,
    y_pred,
    f"{plot_root}/confusion_matrix/w2_{W2}/r1_to_r1/{pair}.png",
    f"r1→r1 - {pair} - W2={W2}"
)

# ==========================
# SETUP 5: r2 -> r2
# ==========================
X, y = prepare_xy(data_r2)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=42
)

X_train, X_test = scale_data(X_train, X_test)
```

```
        y_pred = run_rf(X_train, y_train, X_test)

        acc = accuracy_score(y_test, y_pred)
        err = 1 - acc

        results.append(["r2_to_r2", pair, W2, acc, err])

        save_conf_matrix(
            y_test,
            y_pred,
            f"{plot_root}/confusion_matrix/w2_{W2}/r2_to_r2/{pair}.png",
            f"r2→r2 - {pair} - W2={W2}"
        )

# ==========================
# SAVE SUMMARY TABLE FIGURE
# ==========================
df = pd.DataFrame(results, columns=[
   "setup", "antenna_pair", "W2", "accuracy", "mean_error"
])

os.makedirs(f"{plot_root}/accuracy_error", exist_ok=True)

# ---- SAVE CSV ----
csv_path = f"{plot_root}/accuracy_error/summary_table.csv"
df.to_csv(csv_path, index=False)

# TOP 30 BEST CONFIGS
best_df = df.sort_values(by="accuracy", ascending=False).head(30)

best_path = f"{plot_root}/accuracy_error/best_configurations.csv"
best_df.to_csv(best_path, index=False)

# ==========================
# BEST ANTENNA (AVERAGE)
# ==========================
best_antenna_avg = (
   df.groupby("antenna_pair")["accuracy"]
   .mean()
   .sort_values(ascending=False)
)
```

```
best_antenna_avg_path = f"{plot_root}/accuracy_error/best_antenna_avg.csv"
best_antenna_avg.to_csv(best_antenna_avg_path)

print("\nBest antenna pairs (average accuracy):")
print(best_antenna_avg)

print("\nAll experiments completed and plots saved.")
```

Appendix B

Additional Experimental Results

Table B.1 reports the complete classification accuracy results for all experimental setups, antenna pairs, and temporal window sizes.

summary_table.csv

setup	antenna_pair	W2	accuracy	mean_error
mixed	ant01	10	0.7055019274268060	0.2944980725731940
r1_to_r2	ant01	10	0.5313095444309720	0.4686904555690280
r2_to_r1	ant01	10	0.4912815602479460	0.5087184397520540
r1_to_r1	ant01	10	0.7272085874111780	0.2727914125888220
r2_to_r2	ant01	10	0.7714359951498350	0.2285640048501650
mixed	ant02	10	0.6975373538500750	0.30246264614992500
r1_to_r2	ant02	10	0.47665858305906800	0.523341416940932
r2_to_r1	ant02	10	0.4821347578491160	0.5178652421508850
r1_to_r1	ant02	10	0.7455525878143430	0.25444741218565700
r2_to_r2	ant02	10	0.7382643339684740	0.2617356660315260
mixed	ant03	10	0.6972187709070060	0.3027812290929940
r1_to_r2	ant03	10	0.4541832669322710	0.545816733067729
r2_to_r1	ant03	10	0.4514186362949150	0.5485813637050850
r1_to_r1	ant03	10	0.7505417527591590	0.24945824724084100
r2_to_r2	ant03	10	0.7472717824354760	0.25272821756452400
mixed	ant12	10	0.6706170951607250	0.3293829048392750
r1_to_r2	ant12	10	0.3950718863675730	0.6049281136324270

r2_to_r1	ant12	10	0.3940180416267700	0.6059819583732300
r1_to_r1	ant12	10	0.7216650708058260	0.27833492919417400
r2_to_r2	ant12	10	0.7299497661527800	0.2700502338472200
mixed	ant13	10	0.6907833954570070	0.3092166045429930
r1_to_r2	ant13	10	0.5106097349731510	0.4893902650268490
r2_to_r1	ant13	10	0.44053318550622400	0.5594668144937760
r1_to_r1	ant13	10	0.7269566093836620	0.2730433906163380
r2_to_r2	ant13	10	0.741555517062186	0.25844448293781400
mixed	ant23	10	0.673133900410972	0.326866099589028
r1_to_r2	ant23	10	0.4674779144292400	0.5325220855707600
r2_to_r1	ant23	10	0.39502595373683400	0.6049740462631660
r1_to_r1	ant23	10	0.7105780375951220	0.2894219624048780
r2_to_r2	ant23	10	0.7052658929499390	0.29473410705006100
mixed	ant01	20	0.8083019109094400	0.19169808909056100
r1_to_r2	ant01	20	0.5412393533808450	0.4587606466191550
r2_to_r1	ant01	20	0.4744036789973720	0.5255963210026280
r1_to_r1	ant01	20	0.8192844147968470	0.18071558520315300
r2_to_r2	ant01	20	0.8700677907178860	0.1299322092821140
mixed	ant02	20	0.8182718732025310	0.1817281267974690
r1_to_r2	ant02	20	0.4801407961063790	0.5198592038936210
r2_to_r1	ant02	20	0.47468162522741100	0.5253183747725900
r1_to_r1	ant02	20	0.842581362441884	0.15741863755811600
r2_to_r2	ant02	20	0.8620719624543720	0.13792803754562800
mixed	ant03	20	0.8108583114974120	0.1891416885025880
r1_to_r2	ant03	20	0.44211715626629600	0.5578828437337040

r2_to_r1	ant03	20	0.44986860723670900	0.5501313927632910
r1_to_r1	ant03	20	0.8454618960986460	0.15453810390135400
r2_to_r2	ant03	20	0.866069876586129	0.133930123413871
mixed	ant12	20	0.7831533201252640	0.21684667987473600
r1_to_r2	ant12	20	0.39479402051103800	0.6052059794889620
r2_to_r1	ant12	20	0.39774105518496100	0.6022589448150400
r1_to_r1	ant12	20	0.8122094198504140	0.1877905801495860
r2_to_r2	ant12	20	0.8416478359116980	0.15835216408830200
mixed	ant13	20	0.7934747874992010	0.20652521250079900
r1_to_r2	ant13	20	0.5139057882843730	0.4860942117156270
r2_to_r1	ant13	20	0.4219476450373960	0.5780523549626040
r1_to_r1	ant13	20	0.8193854861532240	0.18061451384677600
r2_to_r2	ant13	20	0.857204936554841	0.14279506344515900
mixed	ant23	20	0.7890969514922990	0.21090304850770100
r1_to_r2	ant23	20	0.4664957413523380	0.5335042586476620
r2_to_r1	ant23	20	0.3747220537699620	0.6252779462300380
r1_to_r1	ant23	20	0.8124115625631700	0.1875884374368300
r2_to_r2	ant23	20	0.8274813140969930	0.17251868590300700
mixed	ant01	30	0.8840667970127250	0.11593320298727500
r1_to_r2	ant01	30	0.5198412698412700	0.4801587301587300
r2_to_r1	ant01	30	0.461663203770334	0.538336796229666
r1_to_r1	ant01	30	0.886231186337607	0.113768813662393
r2_to_r2	ant01	30	0.9234257805686380	0.07657421943136230
mixed	ant02	30	0.8899644219366010	0.11003557806339900
r1_to_r2	ant02	30	0.4773678702250130	0.522632129774987

r2_to_r1	ant02	30	0.4844169665028130	0.5155830334971870
r1_to_r1	ant02	30	0.901079410125171	0.09892058987482900
r2_to_r2	ant02	30	0.9171463457177740	0.08285365428222570
mixed	ant03	30	0.8719510240712840	0.12804897592871600
r1_to_r2	ant03	30	0.44017094017094000	0.5598290598290600
r2_to_r1	ant03	30	0.43305630162671700	0.5669436983732830
r1_to_r1	ant03	30	0.8997618203010190	0.10023817969898100
r2_to_r2	ant03	30	0.9152276295133440	0.0847723704866562
mixed	ant12	30	0.8710215070995870	0.12897849290041400
r1_to_r2	ant12	30	0.4097331240188380	0.5902668759811620
r2_to_r1	ant12	30	0.39157756043176400	0.6084224395682360
r1_to_r1	ant12	30	0.8877514822885520	0.11224851771144800
r2_to_r2	ant12	30	0.9038897610326180	0.0961102389673818
mixed	ant13	30	0.8664380268598350	0.1335619731401650
r1_to_r2	ant13	30	0.511250654107797	0.488749345892203
r2_to_r1	ant13	30	0.3955303299042210	0.6044696700957790
r1_to_r1	ant13	30	0.8781229412659000	0.12187705873410000
r2_to_r2	ant13	30	0.9192394906680620	0.0807605093319379
mixed	ant23	30	0.8684573223500750	0.13154267764992500
r1_to_r2	ant23	30	0.4584423512994940	0.5415576487005060
r2_to_r1	ant23	30	0.3490092738053010	0.6509907261946990
r1_to_r1	ant23	30	0.8852176557036440	0.11478234429635600
r2_to_r2	ant23	30	0.8994418280132570	0.10055817198674300
mixed	ant01	40	0.9165380658436210	0.08346193415637860
r1_to_r2	ant01	40	0.5286626991072990	0.4713373008927010

r2_to_r1	ant01	40	0.444049191991056	0.5559508080089440
r1_to_r1	ant01	40	0.922400650472609	0.077599349527391
r2_to_r2	ant01	40	0.9478382636093120	0.052161736390687900
mixed	ant02	40	0.9266975308641980	0.07330246913580250
r1_to_r2	ant02	40	0.46770523367757700	0.5322947663224230
r2_to_r1	ant02	40	0.5031761357861570	0.49682386421384300
r1_to_r1	ant02	40	0.9333265575769900	0.06667344242301040
r2_to_r2	ant02	40	0.9513390512865400	0.048660948713460500
mixed	ant03	40	0.9150270061728400	0.0849729938271605
r1_to_r2	ant03	40	0.4370733415018380	0.562926658498162
r2_to_r1	ant03	40	0.4167852424026830	0.5832147575973170
r1_to_r1	ant03	40	0.9288545583900800	0.07114544160991980
r2_to_r2	ant03	40	0.9467005076142130	0.053299492385786800
mixed	ant12	40	0.9060570987654320	0.09394290123456790
r1_to_r2	ant12	40	0.40854192193243500	0.5914580780675650
r2_to_r1	ant12	40	0.3947809736761870	0.6052190263238130
r1_to_r1	ant12	40	0.9233153775790220	0.07668462242097770
r2_to_r2	ant12	40	0.9361106248906000	0.06388937510939960
mixed	ant13	40	0.9026813271604940	0.09731867283950610
r1_to_r2	ant13	40	0.5053387012077720	0.4946612987922280
r2_to_r1	ant13	40	0.37447911373107000	0.6255208862689300
r1_to_r1	ant13	40	0.9130501067181620	0.08694989328183760
r2_to_r2	ant13	40	0.9452126728513920	0.054787327148608500
mixed	ant23	40	0.90625	0.09375
r1_to_r2	ant23	40	0.4608786977069840	0.5391213022930160

r2_to_r1	ant23	40	0.33710234779957300	0.6628976522004270
r1_to_r1	ant23	40	0.9182843784937490	0.08171562150625070
r2_to_r2	ant23	40	0.9363731839663930	0.06362681603360750
mixed	ant01	50	0.9442419942597310	0.0557580057402689
r1_to_r2	ant01	50	0.5089583699279820	0.4910416300720180
r2_to_r1	ant01	50	0.4447841818274470	0.5552158181725530
r1_to_r1	ant01	50	0.9447587015237220	0.05524129847627780
r2_to_r2	ant01	50	0.9689091867205340	0.031090813279466100
mixed	ant02	50	0.9437582637298850	0.056241736270115100
r1_to_r2	ant02	50	0.4715879149833130	0.5284120850166870
r2_to_r1	ant02	50	0.5028283137135000	0.49717168628650100
r1_to_r1	ant02	50	0.9536768078275490	0.04632319217245070
r2_to_r2	ant02	50	0.9648691375373270	0.035130862462673500
mixed	ant03	50	0.9378567512657620	0.06214324873423850
r1_to_r2	ant03	50	0.4382575092218510	0.5617424907781490
r2_to_r1	ant03	50	0.42088365693319100	0.5791163430668090
r1_to_r1	ant03	50	0.9479182591856500	0.05208174081435050
r2_to_r2	ant03	50	0.9642543474442300	0.035745652555770200
mixed	ant12	50	0.933632171305105	0.06636782869489500
r1_to_r2	ant12	50	0.4095819427366940	0.5904180572633060
r2_to_r1	ant12	50	0.3935687713397540	0.6064312286602460
r1_to_r1	ant12	50	0.9428221984406050	0.05717780155939460
r2_to_r2	ant12	50	0.9581942736694190	0.04180572633058140
mixed	ant13	50	0.9324067206294950	0.0675932793705053
r1_to_r2	ant13	50	0.5091779378183730	0.4908220621816270

r2_to_r1	ant13	50	0.37249656015899700	0.6275034398410030
r1_to_r1	ant13	50	0.9374713346583090	0.06252866534169090
r2_to_r2	ant13	50	0.9664500263481470	0.03354997365185320
mixed	ant23	50	0.9341481505369410	0.06585184946305910
r1_to_r2	ant23	50	0.4505533110837870	0.5494466889162130
r2_to_r1	ant23	50	0.32637721041634800	0.6736227895836520
r1_to_r1	ant23	50	0.9429750802629570	0.05702491973704330
r2_to_r2	ant23	50	0.9567890391709120	0.043210960829088400